

# 图论

Morning Flower and Evening Oath

XCPC vector 模板分册

# 目录

|                            |    |
|----------------------------|----|
| 使用约定                       | v  |
| 第一章 DFS (图论)               | 1  |
| 1.1 顶点划分: 各诱导子图全为奇度        | 1  |
| 第二章 树上问题                   | 4  |
| 2.1 树的直径                   | 4  |
| 2.1.1 树上点集直径摘要             | 4  |
| 2.2 最近公共祖先                 | 7  |
| 2.2.1 离线 Tarjan LCA        | 7  |
| 2.2.2 欧拉序与 ST 表 LCA        | 10 |
| 2.2.3 倍增 LCA、祖先与最大边权       | 13 |
| 2.2.4 两条树路径的交集             | 18 |
| 2.3 树链剖分                   | 20 |
| 2.3.1 重链剖分与 LCA            | 20 |
| 2.4 树上启发式合并                | 29 |
| 2.4.1 树上启发式合并: 子树众数颜色和     | 29 |
| 2.5 虚树                     | 32 |
| 2.5.1 虚树: 关键点压缩与路径信息       | 32 |
| 2.6 树分治                    | 36 |
| 2.6.1 点分治: 树上距离点对计数        | 36 |
| 2.7 动态树分治                  | 40 |
| 2.7.1 动态点分树: 点权赋值与距离球求和    | 40 |
| 2.7.2 动态点分树: 最近活动点与点号方案    | 45 |
| 2.7.3 动态点分树: 有符号边权活动点直径与端点 | 49 |
| 第三章 最短路问题                  | 55 |
| 3.1 最短路                    | 55 |
| 3.1.1 Johnson: 负边全源最短路     | 55 |
| 3.1.2 非负权最短路               | 58 |
| 3.1.3 顶点开放时间的单位边最短路        | 60 |
| 第四章 生成树问题                  | 62 |
| 4.1 最小生成树                  | 62 |
| 4.1.1 曼哈顿最小生成树             | 62 |
| 4.1.2 严格与非严格次小生成树、换边方案     | 65 |
| 4.1.3 曼哈顿最小生成树: 闭锥扫描与平局割证明 | 70 |
| 4.2 最小树形图                  | 71 |
| 4.2.1 有向最小树形图              | 71 |
| 第五章 连通性相关                  | 74 |
| 5.1 强连通分量                  | 74 |
| 5.1.1 Tarjan 强连通分量与缩点      | 74 |
| 5.1.2 函数图分解、跳转与到达          | 76 |

|            |                     |            |
|------------|---------------------|------------|
| 5.1.3      | 强连通分量               | 82         |
| 5.2        | 双连通分量               | 85         |
| 5.2.1      | 点双与边双：共享 lowlink 核心 | 85         |
| 5.2.2      | 边双缩点森林（保留桥编号）       | 89         |
| 5.2.3      | 属于简单奇环的全部顶点         | 91         |
| 5.2.4      | 边双缩点的大小与内部边数        | 94         |
| 5.2.5      | 无向边定向与强连通定向         | 97         |
| 5.3        | 割点和桥                | 98         |
| 5.3.1      | 由点双分解计算删点连通块数       | 98         |
| 5.3.2      | 连通无向图最少加边方案         | 101        |
| 5.3.3      | 割点与桥                | 102        |
| 5.4        | 圆方树                 | 105        |
| 5.4.1      | 由点双构造圆方森林           | 105        |
| <b>第六章</b> | <b>环计数问题</b>        | <b>107</b> |
| 6.1        | 无向简单图三元环枚举          | 107        |
| <b>第七章</b> | <b>2-SAT</b>        | <b>109</b> |
| 7.1        | 2-SAT 与任意可行方案       | 109        |
| 7.2        | 递归染色与回滚求最小赋值        | 112        |
| <b>第八章</b> | <b>欧拉图</b>          | <b>115</b> |
| 8.1        | 有向图字典序最小欧拉路         | 115        |
| 8.2        | 字典序最小单词链            | 118        |
| 8.3        | 无向图欧拉路与边方向          | 120        |
| 8.4        | 欧拉回路与指定端点定向         | 123        |
| 8.5        | 任意起终点的混合欧拉路径        | 126        |
| <b>第九章</b> | <b>网络流</b>          | <b>127</b> |
| 9.1        | 最大流                 | 127        |
| 9.1.1      | 最大流与方案              | 127        |
| 9.1.2      | 最大流中单位容量边的可行性与必选性   | 130        |
| 9.2        | 最小割                 | 134        |
| 9.2.1      | 正容量最小割的可行边与必选边      | 134        |
| 9.2.2      | 最大权闭合子图与选点方案        | 136        |
| 9.2.3      | Gomory–Hu 最小割树      | 138        |
| 9.2.4      | 最小割树全点对查询           | 140        |
| 9.3        | 费用流                 | 141        |
| 9.3.1      | 势能最小费用最大流           | 141        |
| 9.3.2      | SPFA 最小费用最大流        | 145        |
| 9.3.3      | 允许负环的最小费用最大流        | 149        |
| 9.4        | 上下界网络流              | 152        |
| 9.4.1      | 上下界可行循环流            | 152        |
| 9.4.2      | 有源汇上下界最值流与边流量       | 154        |
| 9.5        | Stoer–Wagner 算法     | 158        |
| 9.5.1      | 无向全局最小割             | 158        |

|                                 |            |
|---------------------------------|------------|
| <b>第十章 图的匹配</b>                 | <b>161</b> |
| 10.1 二分图最大匹配                    | 161        |
| 10.1.1 Hopcroft–Karp 最大匹配与最小点覆盖 | 161        |
| 10.1.2 二分图最大匹配的可行边与必选边          | 164        |
| 10.2 二分图最大权匹配                   | 165        |
| 10.2.1 矩形二分图最小权匹配               | 165        |
| 10.2.2 二分图最大权匹配（缺边与可选匹配）        | 167        |
| 10.3 一般图最大匹配                    | 169        |
| 10.3.1 一般图最大基数匹配：带花树            | 169        |
| <b>第十一章 Prüfer 序列</b>           | <b>173</b> |
| 11.1 Prüfer 双射与有标号树计数           | 173        |
| 11.1.1 双射为何成立                   | 173        |
| 11.1.2 Cayley、指定度数与叶子数          | 173        |
| 11.1.3 加权 Cayley 与连接已有连通块       | 174        |
| <b>第十二章 矩阵树定理</b>               | <b>175</b> |
| 12.1 带权矩阵树：无向树与内外向树形图           | 175        |
| 12.2 任意正模数带权矩阵树                 | 177        |
| 12.3 带权矩阵树定理与方向约定               | 180        |
| 12.3.1 为什么这个余子式恰好计数             | 180        |
| 12.3.2 既有接口与验证记录                | 180        |
| 12.4 任意正模数的矩阵树计数                | 181        |
| <b>第十三章 LGV 引理</b>              | <b>182</b> |
| 13.1 LGV: DAG 路径权值行列式           | 182        |
| <b>第十四章 支配树</b>                 | <b>184</b> |
| 14.1 支配树: Lengauer–Tarjan       | 184        |
| <b>第十五章 配套知识与验证范围</b>           | <b>188</b> |
| 15.1 树路径交：投影与单点交                | 188        |
| 15.2 非负权树的点集直径摘要                | 189        |
| 15.3 函数图的入环链与到达查询               | 190        |
| 15.4 杭州 2023 H: 随机事件顺序与函数图      | 191        |
| 15.5 杭州 2023 G: 开放时间的单位边最短路     | 192        |
| 15.6 SCC 模板题与速度榜口径              | 192        |
| 15.7 SPFA 费用流与残量方案              | 192        |
| 15.8 2-SAT 编号方向与双连通定义           | 193        |
| 15.9 排列约束与最大流单位边分类              | 194        |
| 15.10 点分治、启发式合并与虚树用法            | 194        |
| 15.11 严格次小生成树与换边                | 195        |
| 15.12 闭合集、树形图与全局割的完整用法          | 195        |
| <b>第一部分 跨类依赖</b>                | <b>196</b> |
| <b>附录 A 数论</b>                  | <b>197</b> |
| A.1 模算术简介                       | 197        |
| A.1.1 静态模整数                     | 197        |

|             |                     |            |
|-------------|---------------------|------------|
| A.2         | 卢卡斯定理               | 199        |
| A.2.1       | Lucas: 小素数模的大下标组合数  | 199        |
| <b>附录 B</b> | <b>组合数学</b>         | <b>201</b> |
| B.1         | 排列组合                | 201        |
| B.1.1       | 阶乘组合数               | 201        |
| <b>附录 C</b> | <b>线性代数</b>         | <b>203</b> |
| C.1         | 行列式                 | 203        |
| C.1.1       | 素数模行列式              | 203        |
| C.1.2       | 任意正模数行列式: 欧几里得消元    | 205        |
| <b>附录 D</b> | <b>并查集</b>          | <b>207</b> |
| D.1         | 并查集                 | 207        |
| D.1.1       | 并查集                 | 207        |
| <b>附录 E</b> | <b>树状数组</b>         | <b>209</b> |
| E.1         | 树状数组第 $k$ 小与组合依赖    | 209        |
| <b>附录 F</b> | <b>线段树</b>          | <b>211</b> |
| F.1         | 线段树基础               | 211        |
| F.1.1       | 通用线段树与左右边界搜索        | 211        |
| F.1.2       | 线段树: 区间乘、加、赋值与模意义求和 | 215        |

# 使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

# 第一章 DFS (图论)

## 1.1 顶点划分：各诱导子图全为奇度

代码依赖：dsu (第 207 页, D.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`odd_induced_partition(n,edges)` 接收  $0..n-1$  的无向简单图，禁止自环、重边。返回 `optional<vector<int>>`：无解为 `nullopt`，否则每点一个正组号，同组顶点的诱导子图中每个点度数均为奇数。不删选组内边，不要求组数最少。充要条件是每个连通分量点数为偶数；空图返回空划分。先递归 DFS 取得原深度和后序，再合并未完成分组；内部点已为奇度、根为偶度，奇数个子组可封口，偶数时留一个可上挂的子组给祖先。level 只记录活动根的挂接深度，原 `depth` 不修改。祖先邻点按原深度排序，以二分寻找最近的严格祖先，避免反复扫描高出度根的全部邻居。时间  $O((n+m) \log(n+1))$ ，空间  $O(n+m)$ ，递归栈  $O(n)$ ；节点数、边数须可由 `int` 表示。依赖 dsu 的按调用方向连接根语义。

```
1 optional<vector<int>> odd_induced_partition(int n, const vector<pair<int, int>> &edges)
2 {
3     vector<vector<int>> g(n), up(n);
4     for (auto [u, v] : edges)
5     {
6         assert(0 <= u && u < n && 0 <= v && v < n && u != v);
7         g[u].push_back(v);
8         g[v].push_back(u);
9     }
10    vector<int> depth(n, -1), order;
11    auto dfs = [&](auto &&self, int u, int h) -> int
12    {
13        depth[u] = h;
14        int size = 1;
15        for (int v : g[u])
16            if (depth[v] == -1) size += self(self, v, h + 1);
17        order.push_back(u);
18        return size;
19    };
20    for (int u = 0; u < n; u++)
21        if (depth[u] == -1 && dfs(dfs, u, 0) % 2) return nullopt;
22    for (auto [u, v] : edges)
23    {
24        if (depth[u] > depth[v]) swap(u, v);
25        up[v].push_back(u);
26    }
27    for (auto &a : up)
28        sort(a.begin(), a.end(), [&](int u, int v) { return depth[u] < depth[v]; });
```

合并未完成分组（接上页同一函数）：

```
29     dsu d(n);
30     vector<int> level = depth, color(n);
31     int count = 0;
32     for (int u : order)
33     {
34         vector<int> children;
35         for (int v : g[u])
36             if (level[v] == level[u] + 1 && !color[d.find(v)]) children.push_back(v);
37         if (children.empty()) continue;
38         int skip = -1, best = -1;
39         if (children.size() % 2 == 0)
40             for (int v : children)
41             {
42                 auto it = lower_bound(up[v].begin(),
43                                     up[v].end(),
44                                     depth[u],
45                                     [&](int w, int h) { return depth[w] < h; });
46                 if (it != up[v].begin() && depth[*prev(it)] > best)
47                 {
48                     skip = v;
49                     best = depth[*prev(it)];
50                 }
51             }
52         if (skip != -1) level[skip] = best + 1;
53         if (children.size() % 2 || skip != -1) color[d.find(u)] = ++count;
54         for (int v : children)
55             if (v != skip) d.merge(u, v);
56     }
57     for (int u = 0; u < n; u++) color[u] = color[d.find(u)];
58     return color;
59 }
```

## 应用题补充：奇度诱导子图划分（本地接口示例）

题意：输入无向简单图，点号减 1 后调用；输出同组顶点的诱导子图全部为奇度的顶点分组，无解输出 -1。组号本身无唯一答案，验证时按原图所有同组边计算度数。本例是本地接口演示，链接为存在性定理来源，不是 OJ 记录。

所需模板：[dsu](#)（第 207 页，D.1.1 节）、[odd\\_induced\\_partition](#)（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://people.maths.ox.ac.uk/scott/Papers/odds2.pdf>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<pair<int, int>> edges(m);
8     for (auto &[u, v] : edges)
9     {
10         cin >> u >> v;
11         u--;
12         v--;
13     }
14     auto color = odd_induced_partition(n, edges);
15     if (!color)
16         cout << -1 << '\n';
17     else
18     {
19         for (int x : *color) cout << x << ' ';
20         cout << '\n';
21     }
22 }
```

## 第二章 树上问题

### 2.1 树的直径

#### 2.1.1 树上点集直径摘要

默认空点集,  $a=b=-1$ 、 $length=0$ 。insert( $v,distance$ ) 插入顶点, 重复插入合法; merge( $other,distance$ ) 合并两个点集摘要, 允许自合并; farthest( $v,distance$ ) 返回 optional<pair<int,long long>> (最远顶点、距离), 空集返回 nullopt, 距离并列时可返回任意最远点。distance( $u,v$ ) 回调必须精确返回同一棵固定非负权树的距离, 点号为非负整数; 所有距离  $\leq LLONG\_MAX$ 。每次插入或最远查询最多两次距离调用, 合并最多四次, 摘要只占  $O(1)$  空间。保存  $a$ 、 $b$  为点集中某对直径端点, 不保留全部点; 字段只读, 不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合, 重置用默认构造。

```
1 struct TreeDiameter
2 {
3     using ll = long long;
4     int a = -1, b = -1;
5     ll length = 0;
6
7     // distance is the metric of one fixed tree with nonnegative edge weights.
8     template <class Distance> void insert(int v, const Distance &distance)
9     {
10         assert(v >= 0);
11         if (a == -1)
12         {
13             a = v;
14             b = v;
15             return;
16         }
17         ll x = distance(a, v), y = distance(b, v);
18         if (x >= y && x > length)
19         {
20             b = v;
21             length = x;
22         }
23         else if (y > length)
24         {
25             a = v;
26             length = y;
27         }
28     }
```

合并与最远点查询（接上页同一结构体）：

```
30     template <class Distance> void merge(TreeDiameter other, const Distance &distance)
31     {
32         if (other.a == -1) return;
33         insert(other.a, distance);
34         insert(other.b, distance);
35     }
36
37     template <class Distance>
38     optional<pair<int, ll>> farthest(int v, const Distance &distance) const
39     {
40         if (a == -1) return nullopt;
41         ll x = distance(a, v), y = distance(b, v);
42         return x >= y ? make_pair(a, x) : make_pair(b, y);
43     }
44 };
```

## 应用题补充：Codeforces 379F New Year Tree

题意：初始 1 连接 2、3、4；每次给指定叶子添两个新叶子，输出直径边数。 $q \leq 500000$ ，最终点数  $4+2q$ 。先离线建最终树并做 HLD，再按编号顺序 insert；添叶不会改变旧点间距离，满足固定非负权树前提。只维护已出现顶点。 $O(q \log q)$  时间、 $O(q)$  空间，DFS 递归；只调用 insert 及 length, merge/farthest 由接口测试验证。

所需模板：HLD（第 20 页，2.3.1 节）、TreeDiameter（第 4 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://codeforces.com/problemset/problem/379/F>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int q;
6     cin >> q;
7     int n = 4 + 2 * q;
8     HLD tree(n);
9     for (int v = 2; v <= 4; v++) tree.add(1, v);
10    for (int i = 0; i < q; i++)
11    {
12        int v;
13        cin >> v;
14        tree.add(v, 5 + 2 * i);
15        tree.add(v, 6 + 2 * i);
16    }
17    tree.build();
18    auto dis = [&](int u, int v)
19    {
20        return tree.dep[u] + tree.dep[v] - 2 * tree.dep[tree.lca(u, v)];
21    };
22    TreeDiameter cur;
23    for (int v = 1; v <= 4; v++) cur.insert(v, dis);
24    for (int i = 0; i < q; i++)
25    {
26        cur.insert(5 + 2 * i, dis);
27        cur.insert(6 + 2 * i, dis);
28        cout << cur.length << '\n';
29    }
30    return 0;
31 }

```

## 2.2 最近公共祖先

### 2.2.1 离线 Tarjan LCA

构造 `OfflineLCA(n)`, `add(u,v)` 加无向树边, `add_query(u,v)` 登记查询并返回从 0 开始的编号, `run(root=1)` 后读取 `answer[id]`。输入必须是非空树, 点号  $1..n$ ; 允许同点、重复查询及零个查询。`run` 可重复调用或换根, 不清空查询; 追加查询后需重新 `run`, 旧答案在重算前不可按新根使用。`DFS` 与并查集查找均递归。并查集按大小合并并压缩路径, `ancestor[find(u)]` 保存已合并部分对应的当前树祖先; 不能直接拿并查集代表当 `LCA`。每个子树完成后合并到父点并更新 `ancestor`; 查询另一端已完成时, 从其集合读取祖先。总时间  $O((n+q) \alpha(n))$ , 空间  $O(n+q)$ 。全部点对和根的父链对拍、50 万点链及 P3379、POJ1470 完整驱动本地验证, 在线待补。

```

1 struct OfflineLCA
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<vector<pair<int, int>>> queries;
6     vector<int> parent, size, ancestor, done, answer;
7
8     OfflineLCA(int n)
9         : n(n),
10          g(n + 1),
11          queries(n + 1),
12          parent(n + 1),
13          size(n + 1),
14          ancestor(n + 1),
15          done(n + 1)
16     {
17     }
18
19     void add(int u, int v)
20     {
21         assert(1 <= u && u <= n && 1 <= v && v <= n);
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     int add_query(int u, int v)
27     {
28         assert(1 <= u && u <= n && 1 <= v && v <= n);
29         int id = answer.size();
30         answer.push_back(0);
31         queries[u].push_back({v, id});
32         queries[v].push_back({u, id});
33         return id;
34     }

```

```

36     int find(int u)
37     {
38         if (parent[u] != u) parent[u] = find(parent[u]);
39         return parent[u];
40     }
41
42     void merge(int u, int v)
43     {
44         u = find(u);
45         v = find(v);
46         if (size[u] < size[v]) swap(u, v);
47         parent[v] = u;
48         size[u] += size[v];
49     }
50
51     void dfs(int u, int p)
52     {
53         for (int v : g[u])
54             if (v != p)
55             {
56                 dfs(v, u);
57                 merge(u, v);
58                 ancestor[find(u)] = u;
59             }
60         done[u] = 1;
61         for (auto [v, id] : queries[u])
62             if (done[v]) answer[id] = ancestor[find(v)];
63     }
64
65     // Input must be a nonempty tree. Query IDs are 0-based.
66     void run(int root = 1)
67     {
68         assert(1 <= root && root <= n);
69         iota(parent.begin(), parent.end(), 0);
70         iota(ancestor.begin(), ancestor.end(), 0);
71         fill(size.begin(), size.end(), 1);
72         fill(done.begin(), done.end(), 0);
73         fill(answer.begin(), answer.end(), 0);
74         dfs(root, 0);
75     }
76 };

```

## 模板题使用：Luogu P3379

题意：给定以  $\text{root}$  为根的树，回答两点的最近公共祖先。先登记全部查询，再  $\text{run}(\text{root})$ ， $\text{answer}$  按查询加入顺序输出；点号为  $1..n$ 。

所需模板：[OfflineLCA](#)（第 7 页，2.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3379>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, root;
6     cin >> n >> m >> root;
7     OfflineLCA graph(n);
8     for (int i = 1; i < n; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     for (int i = 0; i < m; i++)
15     {
16         int u, v;
17         cin >> u >> v;
18         graph.add_query(u, v);
19     }
20     graph.run(root);
21     for (int u : graph.answer) cout << u << '\n';
22     return 0;
23 }
```

### 2.2.2 欧拉序与 ST 表 LCA

构造 EulerLCA(n), add(u,v,w=1) 加非负权无向树边, build(root=1) 预处理; lca(u,v) 返回最近公共祖先, distance(u,v) 仍返回无权路径边数; weighted\_distance(u,v) 返回 long long 带权距离。边权非负, 所有树上路径的权和须  $\leq \text{LLONG\_MAX}$ 。输入必须为非空树, 点号 1..n; 允许同点查询和重复 build 换根。只可在 build 后查询, 加边会清除 ready 标志。递归 DFS 在进入顶点及从每个儿子返回时记录顶点, euler 长度为 2n-1, first[u] 是从 0 开始的首次出现位置; depth[root]=0。两点首次出现位置间深度最小的顶点就是 LCA, 用 ST 表保存该顶点。预处理  $O(n \log n)$  时间和空间, 每次查询  $O(1)$ 。ST 内部下标从 0 开始; first、depth、euler、root\_dist 可直接读取; root\_dist 为当前根的带权距离。50 万点链、全部小树根与点对以及 P3379、POJ1330 双风格完整程序本地验证; 上述无权题在线待补; QOJ8235 的带权应用已双 AC。

```

1 struct EulerLCA
2 {
3     using ll = long long;
4     int n;
5     bool ready = false;
6     vector<vector<int>> g, st;
7     vector<vector<ll>> cost;
8     vector<ll> root_dist;
9     vector<int> first, depth, euler, lg;
10
11     EulerLCA(int n)
12         : n(n), g(n + 1), cost(n + 1), root_dist(n + 1), first(n + 1), depth(n + 1)
13     {
14     }
15
16     void add(int u, int v, ll w = 1)
17     {
18         assert(1 <= u && u <= n && 1 <= v && v <= n);
19         assert(w >= 0);
20         g[u].push_back(v);
21         g[v].push_back(u);
22         cost[u].push_back(w);
23         cost[v].push_back(w);
24         ready = false;
25     }
26
27     int shallower(int u, int v) const { return depth[u] <= depth[v] ? u : v; }

```

递归欧拉序与 RMQ (接上页同一结构体):

```

29 void dfs(int u, int parent)
30 {
31     first[u] = euler.size();
32     euler.push_back(u);
33     for (int i = 0; i < int(g[u].size()); i++)
34     {
35         int v = g[u][i];
36         if (v != parent)
37         {
38             depth[v] = depth[u] + 1;
39             root_dist[v] = root_dist[u] + cost[u][i];
40             dfs(v, u);
41             euler.push_back(u);
42         }
43     }
44 }
45
46 // Input must be a nonempty tree. Rebuilding may change the root.
47 void build(int root = 1)
48 {
49     assert(1 <= root && root <= n);
50     euler.clear();
51     depth[root] = 0;
52     root_dist[root] = 0;
53     dfs(root, 0);
54     int m = euler.size();
55     lg.assign(m + 1, 0);
56     for (int i = 2; i <= m; i++) lg[i] = lg[i / 2] + 1;
57     st.assign(lg[m] + 1, {});
58     st[0] = euler;
59     for (int k = 1; k <= lg[m]; k++)
60     {
61         st[k].resize(m - (1 << k) + 1);
62         for (int i = 0; i + (1 << k) <= m; i++)
63             st[k][i] = shallower(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
64     }
65     ready = true;
66 }

```

LCA、边数距离与带权距离（接上页同一结构体）：

```

68     int lca(int u, int v) const
69     {
70         assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
71         int l = first[u], r = first[v];
72         if (l > r) swap(l, r);
73         int k = lg[r - l + 1];
74         return shallower(st[k][l], st[k][r - (1 << k) + 1]);
75     }
76
77     int distance(int u, int v) const
78     {
79         int z = lca(u, v);
80         return depth[u] + depth[v] - 2 * depth[z];
81     }
82
83     ll weighted_distance(int u, int v) const
84     {
85         int z = lca(u, v);
86         return (root_dist[u] - root_dist[z]) + (root_dist[v] - root_dist[z]);
87     }
88 };

```

## 模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。加完所有边后 build(root)，再逐条调用 lca；点号为 1..n。此题只使用无权 LCA 接口。

所需模板：EulerLCA（第 10 页，2.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3379>

```

1  int main()
2  {
3      ios::sync_with_stdio(false);
4      cin.tie(nullptr);
5      int n, q, root;
6      cin >> n >> q >> root;
7      EulerLCA graph(n);
8      for (int i = 1; i < n; i++)
9      {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.build(root);
15     while (q--)
16     {
17         int u, v;
18         cin >> u >> v;
19         cout << graph.lca(u, v) << '\n';
20     }
21     return 0;
22 }

```

### 2.2.3 倍增 LCA、祖先与最大边权

构造 `LiftingLCA(n)`, `add(u,v,w=1)` 加边, `build(root=1)` 预处理非空无向树, 点号 `1..n`。 `jump(u,k)` 返回 `k` 级祖先, `k` 非负, 超过根返回 `0`; `lca(u,v)` 求最近公共祖先; `distance` 返回路径边数, 不是权值和; `max_edge` 返回路径最大边权, 同点返回 `nullopt`, 其他情况返回有值的 `optional<long long>`。支持负权和 `LLONG_MIN`, `LOW` 只是最大值合并的单位元, 不表示空路径。DFS 递归, 根深度为 `0`; `up[k][u]` 与 `mx[k][u]` 是  $2^k$  次方长度向上段的祖先与最大边权, `0` 号节点为哨兵。可重复 `build` 换根; 修改图后必须重建。预处理  $O(n \log n)$  时间与空间, 各查询  $O(\log n)$ 。不需要边权最值时, 可删去 `mx` 和 `max_edge` 相关部分。全小树点对/根、极值权重及 `50` 万点负权链本地验证; P3379、POJ1330 驱动只验证 LCA, 最大边权仍是本地证据。

```

1 struct LiftingLCA
2 {
3     using ll = long long;
4     static constexpr ll LOW = numeric_limits<ll>::lowest();
5     int n, levels;
6     bool ready = false;
7     vector<vector<pair<int, ll>>> g;
8     vector<vector<int>> up;
9     vector<vector<ll>> mx;
10    vector<int> depth;
11
12    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
13    {
14        assert(n > 0);
15        levels = 1;
16        while ((1LL << levels) <= n) levels++;
17        up.assign(levels, vector<int>(n + 1));
18        mx.assign(levels, vector<ll>(n + 1, LOW));
19    }
20
21    void add(int u, int v, ll w = 1)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26        ready = false;
27    }

```

```

29 void dfs(int u, int parent, ll weight)
30 {
31     up[0][u] = parent;
32     mx[0][u] = weight;
33     for (int k = 1; k < levels; k++)
34     {
35         int p = up[k - 1][u];
36         up[k][u] = up[k - 1][p];
37         mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
38     }
39     for (auto [v, w] : g[u])
40         if (v != parent)
41         {
42             depth[v] = depth[u] + 1;
43             dfs(v, u, w);
44         }
45 }
46
47 // Nonempty tree only. Node 0 means no ancestor.
48 void build(int root = 1)
49 {
50     assert(1 <= root && root <= n);
51     depth[root] = 0;
52     dfs(root, 0, LOW);
53     ready = true;
54 }
55
56 int jump(int u, long long steps) const
57 {
58     assert(ready && 1 <= u && u <= n && steps >= 0);
59     if (steps > depth[u]) return 0;
60     for (int k = 0; k < levels; k++)
61         if (steps >> k & 1) u = up[k][u];
62     return u;
63 }

```

```

65  int lca(int u, int v) const
66  {
67      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
68      if (depth[u] < depth[v]) swap(u, v);
69      u = jump(u, depth[u] - depth[v]);
70      if (u == v) return u;
71      for (int k = levels - 1; k >= 0; k--)
72          if (up[k][u] != up[k][v])
73              {
74                  u = up[k][u];
75                  v = up[k][v];
76              }
77      return up[0][u];
78  }
79
80  int distance(int u, int v) const
81  {
82      return depth[u] + depth[v] - 2 * depth[lca(u, v)];
83  }
84
85  optional<ll> max_edge(int u, int v) const
86  {
87      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
88      if (u == v) return nullopt;
89      if (depth[u] < depth[v]) swap(u, v);
90      ll answer = LOW;
91      int delta = depth[u] - depth[v];
92      for (int k = 0; k < levels; k++)
93          if (delta >> k & 1)
94              {
95                  answer = max(answer, mx[k][u]);
96                  u = up[k][u];
97              }
98      if (u == v) return answer;
99      for (int k = levels - 1; k >= 0; k--)
100          if (up[k][u] != up[k][v])
101              {
102                  answer = max(answer, max(mx[k][u], mx[k][v]));
103                  u = up[k][u];
104                  v = up[k][v];
105              }
106      return max(answer, max(mx[0][u], mx[0][v]));
107  }
108 };

```

## 模板题使用：Luogu P3379

题意：给定以  $root$  为根的树，回答两点的最近公共祖先。加完所有边后  $build(root)$ ，再逐条调用  $lca$ ；点号为  $1..n$ 。此题不涉及跳祖先或路径最大边权。

所需模板：LiftingLCA（第 13 页，2.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3379>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q, root;
6     cin >> n >> q >> root;
7     LiftingLCA graph(n);
8     for (int i = 1; i < n; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.build(root);
15     while (q--)
16     {
17         int u, v;
18         cin >> u >> v;
19         cout << graph.lca(u, v) << '\n';
20     }
21     return 0;
22 }
```

## 应用题补充：SPOJ QTREE2 / 路径距离与第 $k$ 个点

题意：多测  $t \leq 25$ 、 $n \leq 10000$ 、边长  $c \leq 100000$ ， $DIST\ u\ v$  查距离， $KTH\ u\ v\ k$  查从  $u$  到  $v$  的第  $k$  个点（1-based 且合法）， $DONE$  结束，每组后空行。根距离  $d$  额外用递归 DFS 求出；切勿把  $h.distance$  的边数当权值和。令  $a$ 、 $b$  为两端到 LCA 的边数， $k \leq a+1$  从  $u$  跳  $k-1$ ，否则从  $v$  跳  $a+b+1-k$ 。预处理  $O(n \log n)$ 、查询  $O(\log n)$ 、空间  $O(n \log n)$ ；本用法所有根距离、差和答案须在 `long long` 范围。

所需模板：LiftingLCA（第 13 页，2.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.spoj.com/problems/QTREE2/>

## SPOJ QTREE2 / 路径距离与第 k 个点：使用代码

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int tests;
6     cin >> tests;
7     while (tests--)
8     {
9         int n;
10        cin >> n;
11        LiftingLCA h(n);
12        for (int i = 1; i < n; i++)
13        {
14            int u, v;
15            long long w;
16            cin >> u >> v >> w;
17            h.add(u, v, w);
18        }
19        h.build();
20        vector<long long> d(n + 1);
21        auto dfs = [&](auto &&self, int u, int p) -> void
22        {
23            for (auto [v, w] : h.g[u])
24            {
25                if (v == p) continue;
26                d[v] = d[u] + w;
27                self(self, v, u);
28            }
29        };
30        dfs(dfs, 1, 0);
31        string cmd;
32        while (cin >> cmd && cmd != "DONE")
33        {
34            int u, v;
35            cin >> u >> v;
36            int p = h.lca(u, v);
37            if (cmd == "DIST") cout << (d[u] - d[p]) + (d[v] - d[p]) << '\n';
38            else
39            {
40                int k;
41                cin >> k;
42                int a = h.depth[u] - h.depth[p];
43                int b = h.depth[v] - h.depth[p];
44                int ans = k <= a + 1 ? h.jump(u, k - 1) : h.jump(v, a + b + 1 - k);
45                cout << ans << '\n';
46            }
47        }
48        cout << '\n';
49    }
50 }
```

### 2.2.4 两条树路径的交集

`path_intersection(a,b,c,d,depth,lca)` 返回路径 `a-b` 与 `c-d` 的公共路径，字段 `u`、`v` 是交集端点，`vertices` 是公共顶点数；空交返回 `{0,0,0}`，单点交的 `u=v`。点号从 1 开始，`depth` 数组与 `lca` 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能带带权根距离。支持数组或 `vector` 深度表，回调接收两个点并返回 `LCA`。将 `c`、`d` 分别投影到 `a-b` 得 `p`、`q`；投影是 `lca(a,b)`、`lca(a,x)`、`lca(b,x)` 中最深者。若 `p≠q`，`c-d` 必经过 `p-q`；若 `p=q`，检验 `p` 是否在 `c-d` 上即可区分单点交与空交。不依赖顶点编号大小。常数次 `LCA` 调用、 $O(1)$  额外空间；使用 `EulerLCA` 时为  $O(1)$ ，`HLD/倍增` 时为  $O(\log n)$ 。公共边数为  $\max(0, \text{vertices}-1)$ 。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```

1 struct PathIntersection
2 {
3     int u, v, vertices;
4 };
5
6 // Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
7 template <class Depth, class Lca>
8 PathIntersection
9 path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
10 {
11     int z = lca(a, b);
12     auto project = [&](int x)
13     {
14         int p = z;
15         for (int v : {lca(a, x), lca(b, x)})
16             if (depth[v] > depth[p]) p = v;
17         return p;
18     };
19     auto distance = [&](int u, int v)
20     {
21         return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
22     };
23     int p = project(c), q = project(d);
24     if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
25     return {p, q, int(distance(p, q) + 1)};
26 }

```

## 应用题补充：Luogu P3398

题意：树上两条路径是否有公共顶点， $n, q \leq 100000$ 。点号  $1..n$ ，EulerLCA 的 depth 与 lca 必须来自同一根，depth 是边数而非带权距离。vertices 非零输出 Y，否则 N；此题只使用是否相交，交集端点和顶点数另由本地路径点集验证。EulerLCA 预处理  $O(n \log n)$ ，每次查询  $O(1)$ ，DFS 递归。

所需模板：EulerLCA（第 10 页，2.2.2 节）、path\_intersection（第 18 页，2.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3398>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7     EulerLCA tree(n);
8     for (int i = 1; i < n; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         tree.add(u, v);
13     }
14     tree.build();
15     auto lca = [&](int u, int v)
16     {
17         return tree.lca(u, v);
18     };
19     while (q--)
20     {
21         int a, b, c, d;
22         cin >> a >> b >> c >> d;
23         auto result = path_intersection(a, b, c, d, tree.depth, lca);
24         cout << (result.vertices ? 'Y' : 'N') << '\n';
25     }
26     return 0;
27 }
```

## 2.3 树链剖分

### 2.3.1 重链剖分与 LCA

`add(u,v)` 建无向树；`build(root)` 后读取 `fa`、`dep`、`siz`、`son`、`top`、`dfn`、`rk`。子树对应  $[dfn[u], dfn[u] + siz[u] - 1]$ 。`lca(u,v)` 为  $O(\log n)$ ，`path(u,v,work,edge)` 分解路径；`work(l,r)` 仅适用于可交换聚合，边权模式排除 LCA 位置。输入必须是连通无环树，建树  $O(n)$ ，两遍递归 DFS；支持改变根后重复 `build`，`dfn` 会重新编号。`path_ordered(u, v, work, edge)` 的 `edge` 默认为 `false`；按真实 `u` 到 `v` 顺序回调 `work(l, r, rev)`：1-based 闭区间  $l \leq r$ ，`rev=true` 按 `r` 到 `l` 访问，否则按 `l` 到 `r`。支持非交换聚合， $O(\log n)$  段与额外空间；结合线段树通常每次查询  $O(\log^2 n)$ 。`edge=true` 仅删除 LCA 对应位置，仍须自行处理边算子的方向；不自动求逆。`u=v` 时点模式一次单点回调，边模式无回调；改变根后须重建依赖 `dfn` 的结构。

```

1 struct HLD
2 {
3     int n, timer = 0;
4     vector<vector<int>>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6
7     HLD(int n)
8         : n(n),
9           g(n + 1),
10          fa(n + 1),
11          dep(n + 1),
12          siz(n + 1),
13          son(n + 1),
14          top(n + 1),
15          dfn(n + 1),
16          rk(n + 1)
17     {
18     }
19
20     void add(int u, int v)
21     {
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     void dfs1(int u, int p)
27     {
28         fa[u] = p;
29         siz[u] = 1;
30         son[u] = 0;
31         for (int v : g[u])
32         {
33             if (v == p) continue;
34             dep[v] = dep[u] + 1;
35             dfs1(v, u);
36             siz[u] += siz[v];
37             if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
38         }
39     }

```

重编号、LCA 与可交换路径（接上页同一结构体）：

```

41 void dfs2(int u, int t)
42 {
43     top[u] = t;
44     dfn[u] = ++timer;
45     rk[timer] = u;
46     if (son[u]) dfs2(son[u], t);
47     for (int v : g[u])
48     {
49         if (v != fa[u] && v != son[u]) dfs2(v, v);
50     }
51 }
52
53 // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
54 void build(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     timer = 0;
58     dep[root] = 0;
59     dfs1(root, root);
60     dfs2(root, root);
61     assert(timer == n);
62 }
63
64 int lca(int u, int v) const
65 {
66     while (top[u] != top[v])
67     {
68         if (dep[top[u]] < dep[top[v]]) swap(u, v);
69         u = fa[top[u]];
70     }
71     return dep[u] < dep[v] ? u : v;
72 }
73
74 // Commutative operations only; edge=true excludes the LCA's position.
75 template <class F> void path(int u, int v, F work, bool edge = false) const
76 {
77     while (top[u] != top[v])
78     {
79         if (dep[top[u]] < dep[top[v]]) swap(u, v);
80         work(dfn[top[u]], dfn[u]);
81         u = fa[top[u]];
82     }
83     if (dep[u] > dep[v]) swap(u, v);
84     if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
85 }

```

按真实方向分解路径（接上页同一结构体）：

```
87 // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
88 template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
89 {
90     vector<pair<int, int>> down;
91     while (top[u] != top[v])
92     {
93         if (dep[top[u]] >= dep[top[v]])
94         {
95             work(dfn[top[u]], dfn[u], true);
96             u = fa[top[u]];
97         }
98         else
99         {
100             down.emplace_back(dfn[top[v]], dfn[v]);
101             v = fa[top[v]];
102         }
103     }
104     if (dep[u] >= dep[v])
105     {
106         if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
107     }
108     else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
109     for (auto it = down.rbegin(); it != down.rend(); ++it)
110         work(it->first, it->second, false);
111 }
112 };
```

## 模板题使用：Luogu P3384

题意：给定有根点权树，支持路径加、路径和、子树加、子树和，答案模  $\text{mod}$ 。先按  $\text{dfn}$  重排初值再构造线段树； $\text{path}$  回调给出 1-based 闭区间，子树区间为  $[\text{dfn}[u], \text{dfn}[u] + \text{siz}[u] - 1]$ 。

所需模板：[HLD](#)（第 20 页，2.3.1 节）、[AffineSegTree](#)（第 215 页，F.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3384>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, root, mod;
6     cin >> n >> m >> root >> mod;
7     vector<long long> values(n + 1);
8     for (int u = 1; u <= n; u++) cin >> values[u];
9     HLD h(n);
10    for (int i = 1; i < n; i++)
11    {
12        int u, v;
13        cin >> u >> v;
14        h.add(u, v);
15    }
16    h.build(root);
17    vector<long long> ordered(n);
18    for (int u = 1; u <= n; u++) ordered[h.dfn[u] - 1] = values[u];
19    AffineSegTree seg(ordered, mod);
20    while (m--)
21    {
22        int op, u;
23        cin >> op >> u;
24        if (op == 1)
25        {
26            int v;
27            long long x;
28            cin >> v >> x;
29            h.path(u, v, [&](int l, int r) { seg.update(l, r, 1, x); });
30        }
31        else if (op == 2)
32        {
33            int v;
34            cin >> v;
35            long long ans = 0;
36            h.path(u, v, [&](int l, int r) { ans = (ans + seg.query(l, r)) % mod; });
37            cout << ans << '\n';
38        }
39        else if (op == 3)
40        {
41            long long x;
42            cin >> x;
43            seg.update(h.dfn[u], h.dfn[u] + h.siz[u] - 1, 1, x);
44        }
45        else
46            cout << seg.query(h.dfn[u], h.dfn[u] + h.siz[u] - 1) << '\n';
47    }
48 }
```

## 模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树， $1 \leq N, Q \leq 200000$ 。点函数  $f(x) = a * x + b$ ，对 998244353 取模；题面  $1 \leq a, c < \text{mod}, 0 \leq b, d, x < \text{mod}$ 。  
 $0 \leq p < d$  改单点函数， $1 \leq u, v \leq x$  按  $u$  到  $v$ （含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间  $[l, r]$  对应 segtree 半开  $[l-1, r)$ 。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改  $O(\log N)$ ，路径  $O(\log^2 N)$ ，空间  $O(N)$ ；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 20 页，2.3.1 节）、segtree（第 211 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://judge.yosupo.jp/problem/vertex\\_set\\_path\\_composite](https://judge.yosupo.jp/problem/vertex_set_path_composite)

```

1 using S = array<long long, 4>;
2 const int mod = 998244353;
3
4 S op(S a, S b)
5 {
6     return {b[0] * a[0] % mod,
7             (b[0] * a[1] + b[1]) % mod,
8             a[2] * b[2] % mod,
9             (a[2] * b[3] + a[3]) % mod};
10 }
11
12 S e()
13 {
14     return {1, 0, 1, 0};
15 }
16
17 int main()
18 {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21     int n, q;
22     cin >> n >> q;
23     vector<S> a(n), val(n);
24     for (auto &x : a)
25     {
26         cin >> x[0] >> x[1];
27         x[2] = x[0];
28         x[3] = x[1];
29     }
30     HLD h(n);
31     for (int i = 1; i < n; i++)
32     {
33         int u, v;
34         cin >> u >> v;
35         h.add(u + 1, v + 1);
36     }
37     h.build();
38     for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
39     segtree<S, op, e> s(val);

```

继续处理操作（接上页同一 main）：

```
40     while (q--)  
41     {  
42         int t;  
43         cin >> t;  
44         if (t == 0)  
45         {  
46             int p;  
47             long long c, d;  
48             cin >> p >> c >> d;  
49             s.set(h.dfn[p + 1] - 1, {c, d, c, d});  
50         }  
51         else  
52         {  
53             int u, v;  
54             long long x;  
55             cin >> u >> v >> x;  
56             h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)  
57             {  
58                 S z = s.prod(l - 1, r);  
59                 int k = rev ? 2 : 0;  
60                 x = (z[k] * x + z[k + 1]) % mod;  
61             });  
62             cout << x << '\n';  
63         }  
64     }  
65 }
```

## 接口用法演示：路径点权增减与点查 (kuangbin 3.3.1 源码模型)

演示约定：读到 EOF：n,m,q、n 个初值、m 条树边，要求  $m=n-1$ 。I u v x/D u v x 给路径所有点加/减 x，Q u 查点值。1-based，初值保留在 a，Fenwick 仅记录增量差分；HLD 闭区间 [l,r] 在 l 加 x、r+1 减 x，包含 LCA 与两端。演示  $n,q \leq 50000$ 、| 初值  $\leq 10^{12}$ 、 $0 \leq x \leq 10^6$ ，全部中间值用 long long。每次更新  $O(\log^2 n)$ 、点查  $O(\log n)$ 、空间  $O(n)$ 。原 HDU3966 题面未核验。

所需模板：HLD (第 20 页, 2.3.1 节)、Fenwick (第 209 页, E.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://github.com/kuangbin/ACM-ICPC>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, q;
6     while (cin >> n >> m >> q)
7     {
8         vector<long long> a(n + 1);
9         for (int u = 1; u <= n; u++) cin >> a[u];
10        HLD h(n);
11        for (int i = 0; i < m; i++)
12        {
13            int u, v;
14            cin >> u >> v;
15            h.add(u, v);
16        }
17        h.build();
18        auto bit = Fenwick<long long>(n);
19        while (q--)
20        {
21            char op;
22            int u;
23            cin >> op >> u;
24            if (op == 'Q') cout << a[u] + bit.sum(h.dfn[u]) << '\n';
25            else
26            {
27                int v;
28                long long x;
29                cin >> v >> x;
30                if (op == 'D') x = -x;
31                h.path(u, v, [&](int l, int r)
32                {
33                    bit.add(l, x);
34                    bit.add(r + 1, -x);
35                });
36            }
37        }
38    }
39 }
```

## 应用题补充：SPOJ QTREE / 单边修改与路径最大边权

题意：多测  $t \leq 20$ 、 $n \leq 10000$ 。输入边按  $1..n-1$  编号；CHANGE  $i\ x$  修改第  $i$  条边，QUERY  $u\ v$  查路径最大边权，DONE 结束本组。把边存到较深端点的  $dfn-1$ ；path 的  $edge=true$  排除 LCA，闭  $[l,r]$  转为 segtree 的半开  $[l-1,r)$ 。空路径约定输出 0，非空路径支持 signed64 负权及最小值；题面仅写  $c \leq 10^6$ ，未写修改值下界。预处理  $O(n)$ 、修改  $O(\log n)$ 、查询  $O(\log^2 n)$ 、空间  $O(n)$ ，递归 DFS。

所需模板：HLD（第 20 页，2.3.1 节）、segtree（第 211 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.spoj.com/problems/QTREE/>

```
1 auto op(long long x, long long y)
2 {
3     return max(x, y);
4 }
5
6 auto e()
7 {
8     return numeric_limits<long long>::lowest();
9 }
```

使用入口（接上页配置函数）：

```

11 int main()
12 {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15     int tests;
16     cin >> tests;
17     while (tests--)
18     {
19         int n;
20         cin >> n;
21         HLD h(n);
22         vector<int> a(n), b(n), child(n);
23         vector<long long> w(n), base(n, e());
24         for (int i = 1; i < n; i++)
25         {
26             cin >> a[i] >> b[i] >> w[i];
27             h.add(a[i], b[i]);
28         }
29         h.build();
30         for (int i = 1; i < n; i++)
31         {
32             child[i] = h.dep[a[i]] > h.dep[b[i]] ? a[i] : b[i];
33             base[h.dfn[child[i]] - 1] = w[i];
34         }
35         auto seg = segtree<long long, op, e>(base);
36         string cmd;
37         while (cin >> cmd && cmd != "DONE")
38         {
39             int u;
40             cin >> u;
41             if (cmd == "CHANGE")
42             {
43                 long long x;
44                 cin >> x;
45                 seg.set(h.dfn[child[u]] - 1, x);
46             }
47             else
48             {
49                 int v;
50                 cin >> v;
51                 long long ans = e();
52                 h.path(u, v, [&](int l, int r)
53                 {
54                     ans = max(ans, seg.prod(l - 1, r));
55                 }, true);
56                 cout << (u == v ? 0 : ans) << '\n';
57             }
58         }
59     }
60 }

```

## 2.4 树上启发式合并

### 2.4.1 树上启发式合并：子树众数颜色和

SubtreeColors( $n$ ), 点号  $1..n$ , add( $u,v$ ) 建无向树; 要求图连通且无环,  $n=0$  可空运行。run(colors, root=1) 传入长度  $n+1$  的 int 颜色数组, colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和, max\_freq[u] 为该频次; 支持负颜色与稀疏编号, 内部离散化, 和用 long long。重复 run 可更换颜色或根。重儿子信息保留, 轻儿子通过 DFS 序区间加入; 清空整棵临时子树后再重置最大值, 不尝试逐次删除时维护众数。 $O(n \log n)$  时间,  $O(n)$  空间, 全部 DFS 递归。insert 是内部增量统计, 可按题目修改统计量, 但相应清空规则也必须调整。

```

1 struct SubtreeColors
2 {
3     int n, best = 0;
4     long long sum = 0;
5     vector<vector<int>>> g;
6     vector<int> siz, heavy, tin, tout, order, color, value, freq;
7     vector<int> max_freq;
8     vector<long long> answer;
9
10    SubtreeColors(int n) : n(n), g(n + 1) {}
11
12    void add(int u, int v)
13    {
14        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
15        g[u].push_back(v);
16        g[v].push_back(u);
17    }
18
19    void prepare(int u, int p)
20    {
21        siz[u] = 1;
22        tin[u] = order.size();
23        order.push_back(u);
24        for (int v : g[u])
25        {
26            if (v == p) continue;
27            prepare(v, u);
28            siz[u] += siz[v];
29            if (siz[v] > siz[heavy[u]]) heavy[u] = v;
30        }
31        tout[u] = (int)order.size() - 1;
32    }
33
34    void insert(int u)
35    {
36        int c = color[u];
37        freq[c]++;
38        if (freq[c] > best)
39        {
40            best = freq[c];
41            sum = value[c];
42        }
43        else if (freq[c] == best)
44            sum += value[c];
45    }

```

启发式合并与运行入口（接上页同一结构体）：

```

47 void solve(int u, int p, bool keep)
48 {
49     for (int v : g[u])
50     {
51         if (v != p && v != heavy[u]) solve(v, u, false);
52     }
53     if (heavy[u]) solve(heavy[u], u, true);
54     for (int v : g[u])
55     {
56         if (v == p || v == heavy[u]) continue;
57         for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
58     }
59     insert(u);
60     max_freq[u] = best;
61     answer[u] = sum;
62     if (!keep)
63     {
64         for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
65         best = 0;
66         sum = 0;
67     }
68 }
69
70 // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
71 void run(const vector<int> &colors, int root = 1)
72 {
73     assert((int)colors.size() == n + 1);
74     siz.assign(n + 1, 0);
75     heavy = siz;
76     tin = siz;
77     tout = siz;
78     color = siz;
79     max_freq = siz;
80     answer.assign(n + 1, 0);
81     order.clear();
82     value.assign(colors.begin() + 1, colors.end());
83     sort(value.begin(), value.end());
84     value.erase(unique(value.begin(), value.end()), value.end());
85     freq.assign(value.size(), 0);
86     best = 0;
87     sum = 0;
88     if (!n) return;
89     assert(1 <= root && root <= n);
90     for (int u = 1; u <= n; u++)
91         color[u] =
92             lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
93     prepare(root, 0);
94     solve(root, 0, false);
95 }
96 };

```

## 应用题补充：Codeforces 600E

题意：以 1 为根，对每个子树输出出现次数最多的所有颜色编号之和；并列颜色都计入。颜色数组 1-based，run 后直接读 answer，和用 long long。n 至 10 万、颜色在 1..n；比赛应用，不替代正式模板题。

所需模板：SubtreeColors（第 29 页，2.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://codeforces.com/problemset/problem/600/E>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n;
6     std::cin >> n;
7     std::vector<int> colors(n + 1);
8     for (int u = 1; u <= n; u++) std::cin >> colors[u];
9     SubtreeColors tree(n);
10    for (int i = 1; i < n; i++)
11    {
12        int u, v;
13        std::cin >> u >> v;
14        tree.add(u, v);
15    }
16    tree.run(colors);
17    for (int u = 1; u <= n; u++) std::cout << tree.answer[u] << (u == n ? '\n' : ' ');
18 }
```

## 2.5 虚树

### 2.5.1 虚树：关键点压缩与路径信息

`VirtualTree(n)`,  $n \geq 1$ , 点号  $1..n$ ; `add(u,v,w=1)` 建非负权连通树, 无环和重边。 `prepare(root=1)` 递归预处理, 可换根重建; 根到任一点距离须在 `long long` 范围内。 `build(keys)` 自动去重并补齐必要 LCA, 返回 `vertices` 和 `edges`; 全部编号沿用原树, `vertices` 按 DFS 序排列, 非空时首项即虚树根。 `edges` 为 `(from,to,distance,minimum)`, 分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果, 单点返回单节点; 原树根不会强制加入, 需要时由调用者加入 `keys`。边按后代 DFS 序排列, 可逆序做自底向上的 DP。不分配或清空  $n$  个查询邻接表; 每次仅返回  $O(k)$  结果。预处理  $O(n \log n)$  时间空间, 单次  $O(k \log k + k \log n)$ 。 `lca`、`ancestor` 也可直接使用, 要求有效点号且已经 `prepare`。构造用的祖先链栈属于虚树算法, 原树 DFS 保持递归。

```

1 struct VirtualTree
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll distance, minimum;
9     };
10
11     struct Result
12     {
13         vector<int> vertices;
14         vector<Edge> edges;
15     };
16
17     int n, log = 1, timer = 0;
18     vector<vector<pair<int, ll>>> g;
19     vector<vector<int>> up;
20     vector<vector<ll>> mn;
21     vector<int> depth, tin, tout;
22     vector<ll> dist;
23
24     VirtualTree(int n) : n(n), g(n + 1) {}
25
26     void add(int u, int v, ll w = 1)
27     {
28         assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
29         g[u].push_back({v, w});
30         g[v].push_back({u, w});
31     }

```

预处理与最近公共祖先（接上页同一结构体）：

```

33 void dfs(int u, int p)
34 {
35     tin[u] = ++timer;
36     up[0][u] = p;
37     for (int j = 1; j < log; j++)
38     {
39         up[j][u] = up[j - 1][up[j - 1][u]];
40         mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
41     }
42     for (auto [v, w] : g[u])
43     {
44         if (v == p) continue;
45         assert(dist[u] <= numeric_limits<ll>::max() - w);
46         depth[v] = depth[u] + 1;
47         dist[v] = dist[u] + w;
48         mn[0][v] = w;
49         dfs(v, u);
50     }
51     tout[u] = timer;
52 }
53
54 void prepare(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     log = 1;
58     while ((1LL << log) <= n) log++;
59     up.assign(log, vector<int>(n + 1));
60     mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
61     depth.assign(n + 1, 0);
62     tin = depth;
63     tout = depth;
64     dist.assign(n + 1, 0);
65     timer = 0;
66     dfs(root, root);
67 }
68
69 bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
70
71 int lca(int u, int v) const
72 {
73     if (ancestor(u, v)) return u;
74     if (ancestor(v, u)) return v;
75     for (int j = log - 1; j >= 0; j--)
76     {
77         if (!ancestor(up[j][u], v)) u = up[j][u];
78     }
79     return up[0][u];
80 }

```

压缩路径与虚树构造（接上页同一结构体）：

```

82  Edge path(int u, int v) const
83  {
84      assert(u != v && ancestor(u, v));
85      ll minimum = numeric_limits<ll>::max();
86      int x = v, steps = depth[v] - depth[u];
87      for (int j = 0; j < log; j++)
88      {
89          if (steps >> j & 1)
90          {
91              minimum = min(minimum, mn[j][x]);
92              x = up[j][x];
93          }
94      }
95      return {u, v, dist[v] - dist[u], minimum};
96  }
97
98  // Uses original vertex IDs. Empty input gives an empty result.
99  Result build(vector<int> keys) const
100 {
101     if (keys.empty()) return {};
102     for (int u : keys) assert(1 <= u && u <= n);
103     auto cmp = [&](int u, int v)
104     {
105         return tin[u] < tin[v];
106     };
107     sort(keys.begin(), keys.end(), cmp);
108     keys.erase(unique(keys.begin(), keys.end()), keys.end());
109     int k = keys.size();
110     for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
111     sort(keys.begin(), keys.end(), cmp);
112     keys.erase(unique(keys.begin(), keys.end()), keys.end());
113     Result result{keys, {}};
114     vector<int> st;
115     for (int u : keys)
116     {
117         while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
118         if (!st.empty()) result.edges.push_back(path(st.back(), u));
119         st.push_back(u);
120     }
121     return result;
122 }
123 };

```

## 应用题补充：Luogu P2495 [SDOI2011] 消耗战

题意：每次给若干非根关键点，求删边使它们都与根 1 断开的最小代价。显式加入根构造虚树，逆序遍历虚边，使用路径最小边权与子树代价取小；只清空本次顶点和标记。 $n$  至 25 万、询问及关键点总数至 50 万。省选应用，正式非比赛模板题映射仍待补。

所需模板：VirtualTree（第 32 页，2.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P2495>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n;
6     std::cin >> n;
7     VirtualTree tree(n);
8     for (int i = 1; i < n; i++)
9     {
10         int u, v;
11         long long w;
12         std::cin >> u >> v >> w;
13         tree.add(u, v, w);
14     }
15     tree.prepare();
16     std::vector<int> marked(n + 1);
17     std::vector<long long> dp(n + 1);
18     int q;
19     std::cin >> q;
20     while (q--)
21     {
22         int k;
23         std::cin >> k;
24         std::vector<int> keys(k);
25         for (int &u : keys)
26         {
27             std::cin >> u;
28             marked[u] = 1;
29         }
30         keys.push_back(1);
31         auto result = tree.build(keys);
32         for (int u : result.vertices) dp[u] = 0;
33         for (int i = (int)result.edges.size() - 1; i >= 0; i--)
34         {
35             auto e = result.edges[i];
36             long long cost = marked[e.to] ? e.minimum : std::min(e.minimum, dp[e.to]);
37             dp[e.from] += cost;
38         }
39         std::cout << dp[1] << '\n';
40         for (int u : keys) marked[u] = 0;
41     }
42 }

```

## 2.6 树分治

### 2.6.1 点分治：树上距离点对计数

CentroidPairs( $n$ ), 点号  $1..n$ ;  $\text{add}(u,v,w)$  建树, 允许森林但禁止环、重边、自环, 边权非负; 所有简单路径长度须能用  $\text{long long}$  表示。build() 预处理, 可重复调用; 改变边后须重建。 $\text{count\_leq}(k)$  统计距离  $\leq k$  的无序不同顶点对,  $\text{count\_exact}(k)$  统计距离恰为  $k$  的点对; 负  $k$  返回 0, 零距离只统计不同顶点, 跨连通块不配对。 $\text{parent}[c]$  是点分树父节点, 根为 0。 $\text{all}[c]$  为该层分治块到重心的距离, 含重心距离 0;  $\text{part}[c]$  为去掉重心后各子块的距离。预处理  $O(n \log^2 n)$ , 空间  $O(n \log n)$ , 每次计数  $O(n \log n)$ ; 固定树上多次查询可以复用排序结果。DFS 递归, 距离和比较采用减法避免溢出。不是动态点分树更新模板。

```

1 struct CentroidPairs
2 {
3     using ll = long long;
4     int n;
5     vector<vector<pair<int, ll>>> g;
6     vector<int> siz, removed, parent;
7     vector<vector<ll>> all;
8     vector<vector<vector<ll>>> part;
9
10    CentroidPairs(int n)
11        : n(n),
12          g(n + 1),
13          siz(n + 1),
14          removed(n + 1),
15          parent(n + 1),
16          all(n + 1),
17          part(n + 1)
18    {
19    }
20
21    void add(int u, int v, ll w)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26    }
27
28    void size_dfs(int u, int p)
29    {
30        siz[u] = 1;
31        for (auto [v, w] : g[u])
32        {
33            if (v == p || removed[v]) continue;
34            size_dfs(v, u);
35            siz[u] += siz[v];
36        }
37    }

```

重心与分解（接上页同一结构体）：

```

39  int centroid(int u, int p, int total)
40  {
41      for (auto [v, w] : g[u])
42      {
43          if (v != p && !removed[v] && siz[v] > total / 2)
44              return centroid(v, u, total);
45      }
46      return u;
47  }
48
49  void collect(int u, int p, ll d, vector<ll> &out)
50  {
51      out.push_back(d);
52      for (auto [v, w] : g[u])
53      {
54          if (v == p || removed[v]) continue;
55          assert(d <= numeric_limits<ll>::max() - w);
56          collect(v, u, d + w, out);
57      }
58  }
59
60  void decompose(int entry, int p)
61  {
62      size_dfs(entry, 0);
63      int c = centroid(entry, 0, siz[entry]);
64      removed[c] = 1;
65      parent[c] = p;
66      all[c] = {0};
67      for (auto [v, w] : g[c])
68      {
69          if (removed[v]) continue;
70          part[c].push_back({});
71          auto &d = part[c].back();
72          collect(v, c, w, d);
73          sort(d.begin(), d.end());
74          all[c].insert(all[c].end(), d.begin(), d.end());
75      }
76      sort(all[c].begin(), all[c].end());
77      for (auto [v, w] : g[c])
78      {
79          if (!removed[v]) decompose(v, c);
80      }
81  }

```

重建与距离计数（接上页同一结构体）：

```

83 // Input must be a forest. Rebuild after changing edges.
84 void build()
85 {
86     fill(removed.begin(), removed.end(), 0);
87     for (int u = 1; u <= n; u++)
88     {
89         all[u].clear();
90         part[u].clear();
91     }
92     for (int u = 1; u <= n; u++)
93     {
94         if (!removed[u]) decompose(u, 0);
95     }
96 }
97
98 ll pairs(const vector<ll> &d, ll k) const
99 {
100     int l = 0, r = (int)d.size() - 1;
101     ll answer = 0;
102     while (l < r)
103     {
104         if (d[l] <= k - d[r])
105         {
106             answer += r - l;
107             l++;
108         }
109         else
110             r--;
111     }
112     return answer;
113 }
114
115 // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
116 ll count_leq(ll k) const
117 {
118     if (k < 0) return 0;
119     ll answer = 0;
120     for (int c = 1; c <= n; c++)
121     {
122         answer += pairs(all[c], k);
123         for (const auto &d : part[c]) answer -= pairs(d, k);
124     }
125     return answer;
126 }
127
128 ll count_exact(ll k) const
129 {
130     if (k < 0) return 0;
131     return count_leq(k) - (k ? count_leq(k - 1) : 0);
132 }
133 };

```

## 模板题使用：Luogu P3806

题意：正权树上查询距离为  $k$  的不同顶点对是否存在。先 build 一次，再用 `count_exact` 的结果是否非零输出 AYE/NAY。 $n$  至 1 万、询问至 100、边权至 1 万；该程序查询存在性，底层计数另支持零权和森林。

所需模板：CentroidPairs（第 36 页，2.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3806>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     CentroidPairs tree(n);
8     for (int i = 1; i < n; i++)
9     {
10         int u, v;
11         long long w;
12         std::cin >> u >> v >> w;
13         tree.add(u, v, w);
14     }
15     tree.build();
16     while (m--)
17     {
18         long long k;
19         std::cin >> k;
20         std::cout << (tree.count_exact(k) ? "AYE" : "NAY") << '\n';
21     }
22 }
```

## 2.7 动态树分治

### 2.7.1 动态点分树：点权赋值与距离球求和

代码依赖：Fenwick (第 209 页, E.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

构造 CentroidSum(n), T 默认为 long long;  $n \geq 1$ , 输入是固定连通单位边树。点号  $0..n-1$ ; 先 add(u,v), 再 build(values)。set(u,x) 是点权赋值; query(u,k) 返回到 u 距离  $\leq k$  的全部点权和, 含 u 自身; 负半径返回 0, 半径用 long long。build 可在同一拓扑上重建并替换全部权值, 建树后不再 add 边, 不提供 link/cut。每点保存重心祖先、距离和分支, 使用总距离桶减同支桶容斥; 距离 d 对应 Fenwick 下标 d+1。线性建每层 Fenwick, 构造/空间  $O(n \log n)$ , 修改/查询  $O(\log^2 n)$ 。T 支持零值及加减, 所有桶和、赋值差与容斥中间值均须可表示; 可用负权, 不使用 kth。内部图、路径、桶和辅助函数不能外部改写。原树 DFS 递归深度最坏  $O(n)$ , 不等于重心层数; 最大链测试提供 256MiB 栈, 仍需核对目标评测栈限制。

```

1 template <class T = long long> struct CentroidSum
2 {
3     struct Record
4     {
5         int centroid, distance, branch;
6     };
7     int n;
8     vector<vector<int>> g;
9     vector<int> siz, removed;
10    vector<T> value;
11    vector<vector<Record>> path;
12    vector<Fenwick<T>> all;
13    vector<vector<Fenwick<T>>> part;
14
15    CentroidSum(int n)
16        : n(n), g(n), siz(n), removed(n), value(n), path(n), all(n), part(n)
17    {
18        assert(n > 0);
19    }
20
21    void add(int u, int v)
22    {
23        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
24        g[u].push_back(v);
25        g[v].push_back(u);
26    }

```

子树大小、重心与距离记录（接上页同一结构体）：

```
28 void size_dfs(int u, int p)
29 {
30     siz[u] = 1;
31     for (int v : g[u])
32     {
33         if (v == p || removed[v]) continue;
34         size_dfs(v, u);
35         siz[u] += siz[v];
36     }
37 }
38
39 int centroid(int u, int p, int total)
40 {
41     for (int v : g[u])
42         if (v != p && !removed[v] && siz[v] > total / 2)
43             return centroid(v, u, total);
44     return u;
45 }
46
47 void collect(int u, int p, int d, int c, int b, vector<T> &bucket)
48 {
49     path[u].push_back({c, d, b});
50     if ((int)bucket.size() <= d + 1) bucket.resize(d + 2);
51     bucket[d + 1] += value[u];
52     for (int v : g[u])
53         if (v != p && !removed[v]) collect(v, u, d + 1, c, b, bucket);
54 }
```

线性构建距离桶和递归分解（接上页同一结构体）：

```

56 // Raw distance d is stored at 1-based Fenwick index d+1.
57 void make_bit(Fenwick<T> &bit, vector<T> bucket)
58 {
59     bit.n = (int)bucket.size() - 1;
60     bit.a = move(bucket);
61     for (int i = 1; i <= bit.n; i++)
62         if (i + (i & -i) <= bit.n) bit.a[i + (i & -i)] += bit.a[i];
63 }
64
65 void decompose(int entry)
66 {
67     size_dfs(entry, -1);
68     int c = centroid(entry, -1, siz[entry]);
69     removed[c] = 1;
70     path[c].push_back({c, 0, -1});
71     vector<T> total{T{}, value[c]};
72     for (int v : g[c])
73     {
74         if (removed[v]) continue;
75         vector<T> bucket(1);
76         collect(v, c, 1, c, (int)part[c].size(), bucket);
77         if (total.size() < bucket.size()) total.resize(bucket.size());
78         for (int d = 1; d < (int)bucket.size(); d++) total[d] += bucket[d];
79         part[c].emplace_back();
80         make_bit(part[c].back(), move(bucket));
81     }
82     make_bit(all[c], move(total));
83     for (int v : g[c])
84         if (!removed[v]) decompose(v);
85 }

```

重建、点权赋值与距离球求和（接上页同一结构体）：

```

87 // Rebuild resets all values and indices; do not change the tree after building.
88 void build(const vector<T> &values)
89 {
90     assert((int)values.size() == n);
91     value = values;
92     fill(removed.begin(), removed.end(), 0);
93     for (int u = 0; u < n; u++)
94     {
95         path[u].clear();
96         part[u].clear();
97     }
98     decompose(0);
99 }
100
101 void set(int u, T x)
102 {
103     assert(0 <= u && u < n);
104     T delta = x - value[u];
105     value[u] = x;
106     for (auto [c, d, b] : path[u])
107     {
108         all[c].add(d + 1, delta);
109         if (b != -1) part[c][b].add(d + 1, delta);
110     }
111 }
112
113 T prefix(const Fenwick<T> &bit, long long radius) const
114 {
115     if (radius < 0) return T{};
116     return bit.sum(radius >= bit.n ? bit.n : (int)radius + 1);
117 }
118
119 T query(int u, long long radius) const
120 {
121     assert(0 <= u && u < n);
122     T answer{};
123     if (radius < 0) return answer;
124     for (auto [c, d, b] : path[u])
125     {
126         answer += prefix(all[c], radius - d);
127         if (b != -1) answer -= prefix(part[c][b], radius - d);
128     }
129     return answer;
130 }
131 };

```

## 模板题使用：Luogu P6329 【模板】点分树 / 震波

题意：固定单位边树上修改点权，查询到中心距离不超过  $k$  的权值和。操作 0 查询，操作 1 赋值；每次先将  $x$  及  $y/k$  与最近一次查询答案异或，再把 1-based 顶点  $x$  减一。更新不改变  $last$ ，操作类型不异或。 $n, m \leq 100000$ ，点权/新值在  $1..10000$ ，解码后  $k$  在  $0..n-1$ 。构造后先 build；原树递归 DFS 需要足够栈空间。

所需模板：Fenwick（第 209 页，E.1 节）、CentroidSum（第 40 页，2.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P6329>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     std::vector<long long> value(n);
8     for (auto &x : value) std::cin >> x;
9     CentroidSum tree(n);
10    for (int i = 1; i < n; i++)
11    {
12        int u, v;
13        std::cin >> u >> v;
14        tree.add(u - 1, v - 1);
15    }
16    tree.build(value);
17    long long last = 0;
18    while (m--)
19    {
20        int op;
21        long long x, y;
22        std::cin >> op >> x >> y;
23        x ^= last;
24        y ^= last;
25        if (op == 0)
26        {
27            last = tree.query((int)x - 1, y);
28            std::cout << last << '\n';
29        }
30        else
31            tree.set((int)x - 1, y);
32    }
33 }
```

## 2.7.2 动态点分树：最近活动点与点号方案

CentroidNearest( $n$ ),  $n \geq 1$ , 点号  $0..n-1$ ; 先  $\text{add}(u,v,w=1)$  建固定连通树, 禁止环、重边、自环。边权非负, 所有原树简单路径长度须能用  $\text{long long}$  表示。build() 清空活动集, 可在同一拓扑上重建, 建树后不再 add。set( $u, \text{on}$ ) 启用或停用点, 重复设置无影响; query( $u$ ) 返回  $\text{optional}<\text{pair}<\text{long long}, \text{int}>>$ , 即最近距离和同距离中最小点号, 空集返回  $\text{nullopt}$ , 距离  $\text{LLONG\_MAX}$  合法。active[ $u$ ] 只读, 可用  $\text{set}(u, \text{!active}[u])$  翻转; 其他成员均为内部数据, 不直接修改。每个重心用 set 维护 (距离, 原点号), 删除时精确到点号。查询跳过溢出的重心绕路距离; 最优点与查询点的某个公共重心在原路径上, 因此仍能取得精确答案。构建  $O(n \log n)$ , 单次更新  $O(\log^2 n)$ , 查询  $O(\log n)$ , 空间  $O(n \log n)$ 。所有 DFS 递归, 原树最大递归深度  $O(n)$ 。不支持负边权或动态拓扑。

```

1 struct CentroidNearest
2 {
3     using ll = long long;
4     int n;
5     bool built = false;
6     vector<vector<pair<int, ll>>> g, path;
7     vector<int> siz, removed, active;
8     vector<std::set<pair<ll, int>>> bag;
9
10    CentroidNearest(int n)
11        : n(n), g(n), path(n), siz(n), removed(n), active(n), bag(n)
12    {
13        assert(n > 0);
14    }
15
16    void add(int u, int v, ll w = 1)
17    {
18        assert(!built && 0 <= u && u < n && 0 <= v && v < n);
19        assert(u != v && w >= 0);
20        g[u].push_back({v, w});
21        g[v].push_back({u, w});
22    }
23
24    void size_dfs(int u, int p)
25    {
26        siz[u] = 1;
27        for (auto [v, w] : g[u])
28        {
29            if (v == p || removed[v]) continue;
30            size_dfs(v, u);
31            siz[u] += siz[v];
32        }
33    }
34
35    int centroid(int u, int p, int total)
36    {
37        for (auto [v, w] : g[u])
38            if (v != p && !removed[v] && siz[v] > total / 2)
39                return centroid(v, u, total);
40        return u;
41    }

```

重心距离与递归分解（接上页同一结构体）：

```
43 void collect(int u, int p, ll d, int c)
44 {
45     path[u].push_back({c, d});
46     for (auto [v, w] : g[u])
47     {
48         if (v == p || removed[v]) continue;
49         assert(d <= LLONG_MAX - w);
50         collect(v, u, d + w, c);
51     }
52 }
53
54 void decompose(int entry)
55 {
56     size_dfs(entry, -1);
57     int c = centroid(entry, -1, siz[entry]);
58     removed[c] = 1;
59     collect(c, -1, 0, c);
60     for (auto [v, w] : g[c])
61         if (!removed[v]) decompose(v);
62 }
```

清空重建、活动点增删与最近点查询（接上页同一结构体）：

```

64 // Fixed connected nonnegative-weight tree; rebuild clears all active points.
65 void build()
66 {
67     fill(removed.begin(), removed.end(), 0);
68     fill(active.begin(), active.end(), 0);
69     for (int u = 0; u < n; u++)
70     {
71         path[u].clear();
72         bag[u].clear();
73     }
74     decompose(0);
75     built = true;
76 }
77
78 void set(int u, bool on)
79 {
80     assert(built && 0 <= u && u < n);
81     if (active[u] == on) return;
82     active[u] = on;
83     for (auto [c, d] : path[u])
84     {
85         if (on) bag[c].insert({d, u});
86         else bag[c].erase({d, u});
87     }
88 }
89
90 // Minimum (distance, vertex ID); nullopt means no active point.
91 optional<pair<ll, int>> query(int u) const
92 {
93     assert(built && 0 <= u && u < n);
94     optional<pair<ll, int>> ans;
95     for (auto [c, d] : path[u])
96     {
97         if (bag[c].empty()) continue;
98         auto [x, v] = *bag[c].begin();
99         // A detour may overflow even when every simple path fits.
100         if (x > LLONG_MAX - d) continue;
101         pair<ll, int> now{d + x, v};
102         if (!ans || now < *ans) ans = now;
103     }
104     return ans;
105 }
106 };

```

## 应用题补充：SPOJ QTREE5 / Query on a tree V (Luogu SP2939)

题意：N,Q≤100000，单位边树点号 1..N，初始全黑；先 N 和 N-1 条边，再 Q 条操作。0 u 翻转黑白，1 u 输出最近白点距离（可为自身），没有白点输出-1。内部转换为 0-based，白点对应活动点。原站注明 modified from ZJOI07，按应用用法收录；非独立非比赛模板题。

所需模板：CentroidNearest（第 45 页，2.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.spoj.com/problems/QTREE5/>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n;
6     std::cin >> n;
7     CentroidNearest tree(n);
8     for (int i = 1; i < n; i++)
9     {
10         int u, v;
11         std::cin >> u >> v;
12         tree.add(u - 1, v - 1);
13     }
14     tree.build();
15     int q;
16     std::cin >> q;
17     while (q--)
18     {
19         int op, u;
20         std::cin >> op >> u;
21         --u;
22         if (op == 0)
23             tree.set(u, !tree.active[u]);
24         else
25         {
26             auto ans = tree.query(u);
27             std::cout << (ans ? ans->first : -1) << '\n';
28         }
29     }
30 }
```

### 2.7.3 动态点分树：有符号边权活动点直径与端点

CentroidDiameter( $n$ ), 固定连通无向树、0-based 点号,  $n > 0$  且  $2n$  不溢出  $\text{int}$ 。add( $u, v, w$ ) 仅在首次 build 前; 允许负边权, 所有简单路径权值和必须能用  $\text{long long}$  表示。build() 递归分解并清空活动集, 可反复重建; set( $u, on$ ) 幂等启停点。query() 返回 optional<tuple<long long,int,int>>, 内容为最大距离与任意一对达到最优值的活动端点, 允许两端相同, 所以非空集答案至少 0; 空集 nullopt。各重心按不同分支取最大距离, 只合并跨分支真实路径, 不能套用非负权直径端点摘要; 并列不承诺最小编号。空集预处理  $O(n \log n)$ 、set  $O(\log^2 n)$ 、query  $O(1)$ , 空间  $O(n \log n)$ ; 逐点激活全树  $O(n \log^2 n)$ 。支持值拷贝; active 只读用于反色, 其余字段与 sum/size\_dfs/centroid/collect/decompose/candidate 为内部状态。DFS 保持递归, 树输入合法性与足够栈空间由调用方保证。

```

1 struct CentroidDiameter
2 {
3     using ll = long long;
4     using Item = tuple<ll, int, int>;
5     using Best = tuple<ll, int, int, int>;
6     struct Entry
7     {
8         int c, b;
9         ll d;
10    };
11    int n;
12    bool built = false;
13    vector<vector<pair<int, ll>>> g;
14    vector<vector<Entry>> path;
15    vector<int> siz, removed, active;
16    vector<std::set<pair<ll, int>>> bag;
17    vector<std::set<Item>> top;
18    std::set<Best> best;
19    std::set<int> live;
20
21    CentroidDiameter(int n)
22        : n(n), g(n), path(n), siz(n), removed(n), active(n), top(n)
23    {
24        assert(0 < n && n <= INT_MAX / 2);
25    }
26
27    ll sum(ll a, ll b) const
28    {
29        assert(b >= 0 ? a <= LLONG_MAX - b : a >= LLONG_MIN - b);
30        return a + b;
31    }
32
33    void add(int u, int v, ll w = 1)
34    {
35        assert(!built && 0 <= u && u < n && 0 <= v && v < n && u != v);
36        g[u].push_back({v, w});
37        g[v].push_back({u, w});
38    }

```

递归分解与不同分支的距离（接上页同一结构体）：

```

40 void size_dfs(int u, int p)
41 {
42     siz[u] = 1;
43     for (auto [v, w] : g[u])
44     {
45         if (v == p || removed[v]) continue;
46         size_dfs(v, u);
47         siz[u] += siz[v];
48     }
49 }
50
51 int centroid(int u, int p, int total)
52 {
53     for (auto [v, w] : g[u])
54         if (v != p && !removed[v] && siz[v] > total / 2)
55             return centroid(v, u, total);
56     return u;
57 }
58
59 void collect(int u, int p, ll d, int c, int b)
60 {
61     path[u].push_back({c, b, d});
62     for (auto [v, w] : g[u])
63     {
64         if (v == p || removed[v]) continue;
65         collect(v, u, sum(d, w), c, b);
66     }
67 }
68
69 void decompose(int entry)
70 {
71     size_dfs(entry, -1);
72     int c = centroid(entry, -1, siz[entry]);
73     removed[c] = 1;
74     path[c].push_back({c, (int)bag.size(), 0});
75     bag.emplace_back();
76     for (auto [v, w] : g[c])
77     {
78         if (removed[v]) continue;
79         int b = bag.size();
80         bag.emplace_back();
81         collect(v, c, w, c, b);
82     }
83     for (auto [v, w] : g[c])
84         if (!removed[v]) decompose(v);
85 }

```

清空重建与重心候选（接上页同一结构体）：

```
87 // Fixed tree, signed weights. Rebuild clears all active points.
88 void build()
89 {
90     fill(removed.begin(), removed.end(), 0);
91     fill(active.begin(), active.end(), 0);
92     for (int u = 0; u < n; u++)
93     {
94         path[u].clear();
95         top[u].clear();
96     }
97     bag.clear();
98     best.clear();
99     live.clear();
100    decompose(0);
101    built = true;
102 }
103
104 optional<Best> candidate(int c) const
105 {
106     if (top[c].size() < 2) return nullopt;
107     auto it = top[c].rbegin();
108     auto [x, u, a] = *it++;
109     auto [y, v, b] = *it;
110     if (u > v) swap(u, v);
111     return Best{sum(x, y), u, v, c};
112 }
```

活动点设置与端点查询（接上页同一结构体）：

```

114 void set(int u, bool on)
115 {
116     assert(built && 0 <= u && u < n);
117     if (active[u] == on) return;
118     active[u] = on;
119     if (on) live.insert(u);
120     else live.erase(u);
121     for (auto [c, b, d] : path[u])
122     {
123         if (auto old = candidate(c)) best.erase(*old);
124         if (!bag[b].empty())
125         {
126             auto [x, v] = *bag[b].rbegin();
127             top[c].erase({x, v, b});
128         }
129         if (on) bag[b].insert({d, u});
130         else bag[b].erase({d, u});
131         if (!bag[b].empty())
132         {
133             auto [x, v] = *bag[b].rbegin();
134             top[c].insert({x, v, b});
135         }
136         if (auto now = candidate(c)) best.insert(*now);
137     }
138 }
139
140 // Maximum distance and any attaining pair; equal endpoints are allowed.
141 optional<Item> query() const
142 {
143     assert(built);
144     if (live.empty()) return nullopt;
145     if (!best.empty())
146     {
147         auto [d, u, v, c] = *best.rbegin();
148         if (d > 0) return Item{d, u, v};
149     }
150     int u = *live.begin();
151     return Item{0, u, u};
152 }
153 };

```

## 应用题补充：SPOJ QTREE4 / 有符号边权动态白点直径

题意： $n, q \leq 100000$ ，边权  $-1000..1000$ ，初始全白；C u 反色，A 查白点对的最大距离，两端允许相同。无白点输出 They have disappeared.。题目 1-based，传入模板时减 1；query() 非空时取  $\text{get}<0>$ 。固定树且允许负边，不能改用只支持非负权的 TreeDiameter 摘要。

所需模板：CentroidDiameter（第 49 页，2.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.spoj.com/problems/QTREE4/>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     CentroidDiameter t(n);
8     for (int i = 1; i < n; i++)
9     {
10         int u, v;
11         long long w;
12         cin >> u >> v >> w;
13         t.add(u - 1, v - 1, w);
14     }
15     t.build();
16     for (int u = 0; u < n; u++) t.set(u, true);
17     int q;
18     cin >> q;
19     while (q--)
20     {
21         char op;
22         cin >> op;
23         if (op == 'C')
24         {
25             int u;
26             cin >> u;
27             --u;
28             t.set(u, !t.active[u]);
29         }
30         else
31         {
32             auto ans = t.query();
33             if (ans) cout << get<0>(*ans) << '\n';
34             else cout << "They have disappeared.\n";
35         }
36     }
37 }

```

## 接口用法演示：活动点最大距离与端点方案（接口演示）

演示约定：输入  $n, q, n-1$  条 0-based 带权树边，初始全不活动。0 u on 设置状态（重复设置不变）；1 查询，空集输出  $-1$ ，否则输出最大距离  $d$  及任意最优活动点  $u, v$ 。允许负边和  $u=v$ ，并列不指定点号；所有简单路径和须在 signed64 内。

所需模板：CentroidDiameter（第 49 页，2.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://oi-wiki.org/graph/tree-divide/>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7     CentroidDiameter t(n);
8     for (int i = 1; i < n; i++)
9     {
10         int u, v;
11         long long w;
12         cin >> u >> v >> w;
13         t.add(u, v, w);
14     }
15     t.build();
16     while (q--)
17     {
18         int op;
19         cin >> op;
20         if (op == 0)
21         {
22             int u, on;
23             cin >> u >> on;
24             t.set(u, on);
25         }
26         else
27         {
28             auto ans = t.query();
29             if (!ans) cout << -1 << '\n';
30             else
31             {
32                 auto [d, u, v] = *ans;
33                 cout << d << ' ' << u << ' ' << v << '\n';
34             }
35         }
36     }
37 }
```

## 第三章 最短路问题

### 3.1 最短路

#### 3.1.1 Johnson：负边全源最短路

Johnson(n), 点号 1..n,  $1 \leq n < \text{INT\_MAX}$ , 总边数不超过  $\text{INT\_MAX}$ ; add(u,v,w) 加有向边, 允许重边、自环和完整 long long 边权。build() 用初值全零的 Bellman-Ford 求势能, 等价于超级源点连向所有点; 全图任意位置有负环时返回 false, 否则返回 true。仅成功后可 distances(s), 返回 vector<int128> 的原图距离,  $d[s]=0$ , 不可达为 Johnson::inf; 0 号不用。可按需逐源调用, 不强制存  $n^2$  矩阵。加边会使预处理失效, 再 build 后查询; 可重复 build。重赋权为  $w+h[u]-h[v]$ , 非负; 查询后加  $h[v]-h[s]$  还原。势能与距离全程 int128, 不能无条件转为 long long。预处理  $O(nm+n)$  时间, 单源用惰性优先队列,  $O(n+m \log(m+1))$  时间、 $O(n+m)$  辅助空间; 整个结构  $O(n+m)$ 。g、edges、h 仅供读取, 勿手动修改。

```
1 struct Johnson
2 {
3     using I = __int128_t;
4     static constexpr I inf = I(1) << 120;
5     int n;
6     vector<tuple<int, int, long long>> edges;
7     vector<vector<pair<int, long long>>> g;
8     vector<I> h;
9     bool ready = false;
10
11     Johnson(int n) : n(n), g(n + 1), h(n + 1) { assert(n > 0); }
12
13     void add(int u, int v, long long w)
14     {
15         assert(1 <= u && u <= n && 1 <= v && v <= n);
16         edges.push_back({u, v, w});
17         g[u].push_back({v, w});
18         ready = false;
19     }
```

势能预处理、单源查询与距离还原（接上页同一结构体）：

```

21  bool build()
22  {
23      ready = false;
24      fill(h.begin(), h.end(), 0);
25      for (int step = 0; step < n; step++)
26      {
27          bool changed = false;
28          for (auto [u, v, w] : edges)
29              if (h[v] > h[u] + w)
30              {
31                  h[v] = h[u] + w;
32                  changed = true;
33              }
34          if (!changed)
35          {
36              ready = true;
37              return true;
38          }
39      }
40      return false;
41  }
42
43  vector<I> distances(int s) const
44  {
45      assert(ready && 1 <= s && s <= n);
46      vector<I> d(n + 1, inf);
47      priority_queue<pair<I, int>, vector<pair<I, int>>, greater<pair<I, int>>> q;
48      d[s] = 0;
49      q.push({0, s});
50      while (!q.empty())
51      {
52          auto [value, u] = q.top();
53          q.pop();
54          if (value != d[u]) continue;
55          for (auto [v, w] : g[u])
56          {
57              I next = value + w + h[u] - h[v];
58              if (next < d[v])
59              {
60                  d[v] = next;
61                  q.push({next, v});
62              }
63          }
64      }
65      for (int v = 1; v <= n; v++)
66          if (d[v] != inf) d[v] += h[v] - h[s];
67      return d;
68  }
69  };

```

## 模板题使用：Luogu P5905

题意：有向图可能有负边，若任意位置存在负环输出 -1。否则逐源求距离，按题意把不可达距离换为  $10^9$ ，再求编号加权和。题目规模和边权界保证最终和可转为 long long。

所需模板：Johnson（第 55 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P5905>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     Johnson graph(n);
8     while (m--)
9     {
10         int u, v;
11         long long w;
12         cin >> u >> v >> w;
13         graph.add(u, v, w);
14     }
15     if (!graph.build())
16     {
17         cout << -1 << '\n';
18         return 0;
19     }
20     for (int s = 1; s <= n; s++)
21     {
22         auto d = graph.distances(s);
23         Johnson::I answer = 0;
24         for (int v = 1; v <= n; v++)
25             answer += v * (d[v] == Johnson::inf ? 1000000000 : d[v]);
26         cout << (long long)answer << '\n';
27     }
28 }
```

### 3.1.2 非负权最短路

$\text{add}(u, v, w)$ 、 $\text{run}(s)$ ， $\text{dis}$  为距离， $\text{pre}$  为前驱， $\text{path}(t)$  返回点序列。不可达用  $\text{LLONG\_MAX}$ ；要求所有有限最短距离严格小于该哨兵， $w \geq 0$ 。 $O((V+E)\log V)$ 。

```

1 struct Dijkstra
2 {
3     using ll = long long;
4     static constexpr ll inf = LLONG_MAX;
5     int n;
6     vector<vector<pair<int, ll>>> g;
7     vector<ll> dis;
8     vector<int> pre;
9
10    Dijkstra(int n) : n(n), g(n + 1) {}
11
12    void add(int u, int v, ll w)
13    {
14        assert(w >= 0);
15        g[u].push_back({v, w});
16    }
17
18    void run(int s)
19    {
20        dis.assign(n + 1, inf);
21        pre.assign(n + 1, -1);
22        dis[s] = 0;
23        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
24        q.push({0, s});
25        while (!q.empty())
26        {
27            auto [d, u] = q.top();
28            q.pop();
29            if (d != dis[u]) continue;
30            for (auto [v, w] : g[u])
31                if (w < inf - d && d + w < dis[v])
32                {
33                    dis[v] = d + w;
34                    pre[v] = u;
35                    q.push({dis[v], v});
36                }
37        }
38    }
39
40    vector<int> path(int t) const
41    {
42        if (dis[t] == inf) return {};
43        vector<int> ans;
44        for (; t != -1; t = pre[t]) ans.push_back(t);
45        reverse(ans.begin(), ans.end());
46        return ans;
47    }
48 };

```

## 模板题使用：Luogu P4779

题意：给非负权有向图和源点  $s$ ，输出  $s$  到各点的最短距离。加完边后  $\text{run}(s)$ ，读取  $\text{dis}[1..n]$ ；本题保证源点可达所有点，一般输入的不可达值须按目标题目格式另行转换。

所需模板：Dijkstra（第 58 页，3.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P4779>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m, s;
6     std::cin >> n >> m >> s;
7     Dijkstra graph(n);
8     while (m--)
9     {
10         int u, v;
11         long long w;
12         std::cin >> u >> v >> w;
13         graph.add(u, v, w);
14     }
15     graph.run(s);
16     for (int i = 1; i <= n; i++) std::cout << graph.dis[i] << (i == n ? '\n' : ' ');
17 }
```

### 3.1.3 顶点开放时间的单位边最短路

`release_bfs(open,source,neighbors)` 返回各点最早到达时间,不可达为 -1; 点号  $0..n-1$ ,  $n>0$ , `open[v]` 为非负整数, 源点初始时间为 `open[source]`。允许原地等待, 每条边耗时恰为 1, `neighbors(u,visit)` 枚举所有出邻点并调用 `visit(v)`, 可直接枚举网格而不建边。令  $R=\max(\text{open})$ , 要求  $R+n<\text{INT\_MAX}$  且内存可承受; 时间  $O(n+m+R)$ , 空间  $O(n+R)$ 。第一次发现即可定距, 只适用于单位边和固定开放时间; 不能用于一般带权边、关闭时间或周期可用边。

```

1 template <class Neighbors>
2 vector<int> release_bfs(const vector<int> &open, int source, Neighbors neighbors)
3 {
4     int n = open.size();
5     assert(0 <= source && source < n);
6     int last = *max_element(open.begin(), open.end());
7     assert(last >= 0 && last < INT_MAX - n);
8     vector<vector<int>> pending(last + 1);
9     vector<int> dist(n, -1), queue;
10    queue.reserve(n);
11    dist[source] = open[source];
12    queue.push_back(source);
13    int head = 0, time = 0, waiting = 0;
14    while (head < int(queue.size()) || waiting)
15    {
16        int limit = head < int(queue.size()) ? dist[queue[head]] : INT_MAX;
17        while (time <= last && time <= limit && pending[time].empty()) time++;
18        int u;
19        if (time <= last && time <= limit)
20        {
21            u = pending[time].back();
22            pending[time].pop_back();
23            waiting--;
24        }
25        else
26            u = queue[head++];
27        neighbors(u,
28                [&](int v)
29                {
30                    if (dist[v] != -1) return;
31                    dist[v] = max(dist[u] + 1, open[v]);
32                    if (dist[v] == dist[u] + 1)
33                        queue.push_back(v);
34                    else
35                    {
36                        pending[open[v]].push_back(v);
37                        waiting++;
38                    }
39                });
40    }
41    return dist;
42 }

```

## 应用题补充：QOJ 8236 / ICPC 2023 Hangzhou G —Snake Move

题意：蛇可移动或缩尾，求蛇头到各格最少步数的平方和，模  $2^{64}$ 。初始身体按 0-based  $i$  (不含头) 设  $open=k-i$ ，尾格为 1；源点时间 0，障碍不连边。 $n, m$  至多 3000， $k$  至多  $\min(nm, 100000)$ 。不可达不计，使用 unsigned long long 自然回绕；这是比赛应用，不是独立模板题。

所需模板：[release\\_bfs](#) (第 60 页, 3.1.3 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://codeforces.com/gym/104976/attachments/download/24253/hangzhou\(en\).pdf](https://codeforces.com/gym/104976/attachments/download/24253/hangzhou(en).pdf)

```

1 int main()
2 {
3     int n, m, k;
4     scanf("%d%d%d", &n, &m, &k);
5     vector<int> open(n * m);
6     int source = 0;
7     for (int i = 0; i < k; i++)
8     {
9         int x, y;
10        scanf("%d%d", &x, &y);
11        int u = (x - 1) * m + y - 1;
12        if (i == 0)
13            source = u;
14        else
15            open[u] = k - i;
16    }
17    vector<string> grid(n);
18    vector<char> row(m + 1);
19    for (auto &s : grid)
20    {
21        scanf("%s", row.data());
22        s = row.data();
23    }
24    auto dist =
25        release_bfs(open,
26                    source,
27                    [&](int u, auto visit)
28                    {
29                        int x = u / m, y = u % m;
30                        const int dx[] = {-1, 1, 0, 0};
31                        const int dy[] = {0, 0, -1, 1};
32                        for (int i = 0; i < 4; i++)
33                        {
34                            int a = x + dx[i], b = y + dy[i];
35                            if (a >= 0 && a < n && b >= 0 && b < m && grid[a][b] == '.')
36                                visit(a * m + b);
37                        }
38                    });
39    unsigned long long answer = 0;
40    for (int d : dist)
41        if (d >= 0) answer += 1ULL * d * d;
42    printf("%llu\n", answer);
43 }

```

## 第四章 生成树问题

### 4.1 最小生成树

#### 4.1.1 曼哈顿最小生成树

代码依赖：dsu (第 207 页, D.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

Point 为 `pair<long long, long long>`，坐标支持完整有符号 64 位，点数须  $\leq \text{INT\_MAX}$  且内存可分配。candidates(points) 给出最多  $4n$  条 (weight,u,v) 候选，允许重复边；solve(points) 返回精确 128 位 weight 与原下标边对 edges。空/单点返回 0 且无边，重合点仍为不同顶点；同权最优树不唯一。先转 128 位再变换/加减，输入不变。四方向扫描与 Kruskal 为  $O(n \log(n+1))$ 、 $O(n)$  空间；调用处选较大 dsu 根在前，不改变普通 dsu 语义。候选保证只针对无约束完整 L1 图，不能直接删禁边后声称仍含受限 MST。

```
1 struct ManhattanMST
2 {
3     using I = __int128;
4     using Point = pair<long long, long long>;
5     using Edge = tuple<I, int, int>;
6     struct Result
7     {
8         I weight = 0;
9         vector<pair<int, int>> edges;
10    };
11
12    static vector<Edge> candidates(const vector<Point> &points)
13    {
14        assert(points.size() <= INT_MAX);
15        int n = int(points.size());
16        vector<pair<I, I>> p(points.begin(), points.end());
17        vector<int> id(n);
18        iota(id.begin(), id.end(), 0);
19        vector<Edge> e;
20        e.reserve(size_t(n) * 4);
21        for (int dir = 0; dir < 4; dir++)
22        {
23            sort(id.begin(), id.end(), [&](int i, int j)
24            {
25                return pair{p[i].first + p[i].second, i} <
26                    pair{p[j].first + p[j].second, j};
27            });
```

活动点扫描与 Kruskal（接上页同一结构体，仍在四轮循环中）：

```

28     map<I, int> sweep;
29     for (int i : id)
30     {
31         auto it = sweep.lower_bound(-p[i].second);
32         while (it != sweep.end())
33         {
34             int j = it->second;
35             I dx = p[i].first - p[j].first;
36             I dy = p[i].second - p[j].second;
37             if (dx < dy) break;
38             e.emplace_back(dx + dy, i, j);
39             it = sweep.erase(it);
40         }
41         sweep[-p[i].second] = i;
42     }
43     for (auto &[x, y] : p)
44         if (dir & 1) x = -x;
45         else swap(x, y);
46 }
47 return e;
48 }
49
50 static Result solve(const vector<Point> &points)
51 {
52     auto e = candidates(points);
53     int n = int(points.size());
54     Result r;
55     if (n < 2) return r;
56     sort(e.begin(), e.end());
57     dsu uf(n);
58     for (auto [w, u, v] : e)
59     {
60         int a = uf.find(u), b = uf.find(v);
61         if (a == b) continue;
62         if (uf.size(a) < uf.size(b)) swap(a, b);
63         uf.merge(a, b);
64         r.weight += w;
65         r.edges.emplace_back(u, v);
66         if (r.edges.size() == points.size() - 1) break;
67     }
68     assert(r.edges.size() == points.size() - 1);
69     return r;
70 }
71 };

```

## 模板题使用：Library Checker Manhattan MST

题意： $1 \leq N \leq 200000$ ， $N$  个点的坐标  $0..10^9$ ，可重合且每个输入位置仍是不同顶点。完整图边权为曼哈顿距离；输出最小总权及  $N-1$  条 0-based 原编号边，任意最优树均可。结果 weight 为 128 位；本题粗上界  $2e9$  乘以  $N-1$ ，严格小于  $4e14$ ，所以此驱动才可转 long long 打印。 $N=1$  输出 0、不输出边；库的空输入和有符号 64 位坐标另属 API 扩展。

所需模板：dsu（第 207 页，D.1.1 节）、ManhattanMST（第 62 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://judge.yosupo.jp/problem/manhattanmst>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     vector<ManhattanMST::Point> p(n);
8     for (auto &[x, y] : p) cin >> x >> y;
9     auto r = ManhattanMST::solve(p);
10    cout << (long long)r.weight << '\n';
11    for (auto [u, v] : r.edges) cout << u << ' ' << v << '\n';
12 }
```

### 4.1.2 严格与非严格次小生成树、换边方案

代码依赖：dsu (第 207 页, D.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

SecondMST( $n$ ),  $n \geq 1$ , 点号  $0..n-1$ 。add( $u,v,w$ ) 返回 0-based 边号；允许自环、重边、完整 signed64 位权。solve(strict=true) 返回 connected、128 位 MST 总权 weight、所选边号 tree，以及可选 128 位次小权 next 和换入/换出边号 in/out。strict=true 要求权值严格增大；false 要求边集与返回的 MST 不同，可同权。不连通返回默认空结果；无次小树时 next 为空且 in/out=-1。存在时将 tree 中的 out 删去并加入 in 即得方案。可加边后重算；每次 solve 重建。依赖 dsu，沿用路径减半及  $y$  根挂  $x$  根；不用按秩合并。时间  $O((n+m)\log(n+m))$ ，空间  $O(m+n \log n)$ ，DFS 递归深度  $O(n)$ 。merge/dfs/path 与工作数组为内部实现；原始  $e$  只读。

```

1 struct SecondMST
2 {
3     using I = __int128_t;
4     using Pair = array<int, 2>;
5
6     struct Edge
7     {
8         int u, v;
9         long long w;
10    };
11
12    struct Result
13    {
14        bool connected = false;
15        I weight = 0;
16        vector<int> tree;
17        optional<I> next;
18        int in = -1, out = -1;
19    };
20
21    int n, h;
22    vector<Edge> e;
23    vector<vector<pair<int, int>>> g;
24    vector<int> dep;
25    vector<vector<int>> up;
26    vector<vector<Pair>> mx;
27
28    SecondMST(int n) : n(n), h(bit_width((unsigned)n)) { assert(n > 0); }
29
30    int add(int u, int v, long long w)
31    {
32        assert(0 <= u && u < n && 0 <= v && v < n);
33        e.push_back({u, v, w});
34        return (int)e.size() - 1;
35    }

```

不同权摘要与递归预处理（接上页同一结构体）：

```

37 // Keep edge IDs of the two largest DISTINCT weights; -1 means absent.
38 Pair merge(Pair a, Pair b) const
39 {
40     for (int x : b)
41     {
42         if (x == -1) continue;
43         if (a[0] == -1 || e[x].w > e[a[0]].w)
44         {
45             a[1] = a[0];
46             a[0] = x;
47         }
48         else if (e[x].w != e[a[0]].w && (a[1] == -1 || e[x].w > e[a[1]].w))
49             a[1] = x;
50     }
51     return a;
52 }
53
54 void dfs(int u, int p)
55 {
56     for (int j = 1; j < h; j++)
57     {
58         int v = up[j - 1][u];
59         up[j][u] = up[j - 1][v];
60         mx[j][u] = merge(mx[j - 1][u], mx[j - 1][v]);
61     }
62     for (auto [v, id] : g[u])
63         if (v != p)
64         {
65             dep[v] = dep[u] + 1;
66             up[0][v] = u;
67             mx[0][v] = {id, -1};
68             dfs(v, u);
69         }
70 }

```

路径查询（接上页同一结构体）：

```
72 Pair path(int u, int v) const
73 {
74     Pair ans{-1, -1};
75     if (dep[u] < dep[v]) swap(u, v);
76     for (int j = h - 1; j >= 0; j--)
77         if ((dep[u] - dep[v]) >> j & 1)
78             {
79                 ans = merge(ans, mx[j][u]);
80                 u = up[j][u];
81             }
82     if (u == v) return ans;
83     for (int j = h - 1; j >= 0; j--)
84         if (up[j][u] != up[j][v])
85             {
86                 ans = merge(ans, merge(mx[j][u], mx[j][v]));
87                 u = up[j][u];
88                 v = up[j][v];
89             }
90     return merge(ans, merge(mx[0][u], mx[0][v]));
91 }
```

Kruskal 与逐边换入（接上页同一结构体）：

```

93     Result solve(bool strict = true)
94     {
95         vector<int> ids(e.size());
96         iota(ids.begin(), ids.end(), 0);
97         sort(ids.begin(),
98             ids.end(),
99             [&](int a, int b) { return tie(e[a].w, a) < tie(e[b].w, b); });
100        dsu ds(n);
101        vector<bool> used(e.size());
102        g.assign(n, {});
103        Result ans;
104        for (int id : ids)
105        {
106            auto [u, v, w] = e[id];
107            if (!ds.merge(u, v)) continue;
108            used[id] = true;
109            ans.weight += (I)w;
110            ans.tree.push_back(id);
111            g[u].push_back({v, id});
112            g[v].push_back({u, id});
113        }
114        if ((int)ans.tree.size() != n - 1) return {};
115        ans.connected = true;
116        dep.assign(n, 0);
117        up.assign(h, vector<int>(n));
118        mx.assign(h, vector<Pair>(n, Pair{-1, -1}));
119        dfs(0, -1);
120        for (int id : ids)
121        {
122            auto [u, v, w] = e[id];
123            if (used[id] || u == v) continue;
124            auto p = path(u, v);
125            int out = p[0];
126            if (strict && e[out].w == w) out = p[1];
127            if (out == -1) continue;
128            I cost = ans.weight + (I)w - (I)e[out].w;
129            if (!ans.next || cost < *ans.next)
130            {
131                ans.next = cost;
132                ans.in = id;
133                ans.out = out;
134            }
135        }
136        return ans;
137    }
138 };

```

## 应用题补充：Luogu P4180 [BJWC2010] 严格次小生成树

题意：无向图求总权严格大于 MST 的最小生成树权值； $n$  至 10 万、 $m$  至 30 万、边权  $0..10^9$ ，允许自环并保证答案存在。1-based 顶点读入后减一，solve 默认严格；题目答案可安全转 long long。比赛应用，不计非比赛模板覆盖。

所需模板：dsu（第 207 页，D.1.1 节）、SecondMST（第 65 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4180>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     SecondMST mst(n);
8     while (m--)
9     {
10         int u, v;
11         long long w;
12         cin >> u >> v >> w;
13         mst.add(u - 1, v - 1, w);
14     }
15     auto ans = mst.solve();
16     // The problem guarantees a strict second tree and a 64-bit answer.
17     assert(ans.connected && ans.next);
18     cout << (long long)*ans.next << '\n';
19 }
```

### 4.1.3 曼哈顿最小生成树：闭锥扫描与平局割证明

接口见第 4.1.1 节 (第 62 页)，正式题用法见第 4.1.1 节 (第 64 页)。完整无向图的边权是  $d(p, q) = |p_x - q_x| + |p_y - q_y|$ 。不枚举二次边集：每个点只保留四个向右闭锥中的最近邻，再对候选边作 Kruskal。候选图含有至少一棵最小生成树，不保证含所有最优边；禁边、颜色等额外约束不能直接套用。

**一个方向的扫描** 变换后的锥取  $q_x - p_x \geq q_y - p_y \geq 0$ 。按  $(x + y, \text{原编号})$  排序，活动点用  $-y$  作 map 键。依次从  $y_p \leq y_q$  的点开始检查，直到  $x_q - x_p < y_q - y_p$ 。活动点的  $x - y$  随  $y$  递减而严格递增，因此首次失败后可停止。第一次满足条件的后继具有最小  $x + y$ ，其距离正是  $(x_q + y_q) - (x_p + y_p)$ 。删掉已找到邻居的活动点，每点每轮最多删除一次，故每轮至多  $n - 1$  条候选边。四轮坐标分别为  $(x, y), (y, x), (-y, x), (x, -y)$ ，覆盖右侧四个闭锥。任意无向边都能从左端点指向其中一个锥；竖直边任选一个方向。总时间  $O(n \log n)$ ，额外空间  $O(n)$ 。

**重复坐标与闭边界** 重复坐标是不同顶点。排序平局按原编号，连续代表之间生成零权链；同一  $x + y$  的其他位置不能以非零距离删掉该代表，最后一个代表继续寻找非重合邻居。可先在证明中收缩这些零链。若一个割拆开重合点类，零链已有最小的过割边；否则割可直接下降到收缩图。闭锥内只能使用  $d(r, q) \leq \max(d(p, r), d(p, q))$ ，不能写严格小于： $p = (0, 0), q = (2, 0), r = (1, 1)$  的三条距离都是 2。

**平局也成立的割证明** 在非重合点图的任意割中，选距离最小为  $\delta$  的过割点对  $p, q$ ，并在平局中令  $p_x + q_x$  最大；令  $p$  为左端点。选择包含  $q$  的右向锥，若边恰在对角线上，选靠近水平轴的浅锥。该锥的最近邻  $r$  满足  $d(p, r) \leq \delta$ 。若  $r$  和  $q$  在割的同侧，则  $pr$  已是候选最小过割边。否则  $rq$  过割且  $d(r, q) \leq \delta$ ：严格小于违反最小性，等于也会违反上述平局选择。

后一个结论可在局部锥坐标  $X \geq Y \geq 0$  中检查。满足  $d(p, r) \leq \delta$  的点位于顶点  $(0, 0), (\delta, 0), (\delta/2, \delta/2)$  的三角形。当  $q$  严格位于锥内时，距离达到  $\delta$  只能取已排除的原点； $q$  在主轴时，非零等号点在  $X = Y$  的射线上； $q$  在对角线时，非零等号点只能是  $(\delta, 0)$ 。后两种情况在所选右向锥中都使  $r_x > p_x$ （对角线选浅锥正是为了保证这一点），于是过割点对  $r, q$  的横坐标和更大，矛盾。所以每个割都保留最小边；在候选图上应用 Prim 的割性质，得到的树也是完整图的最小二生成树 MST。

**整数范围与抄写合同** 输入为 signed 64 位整数对；先整体转换为 signed 128 位，再进行和、差、变号及距离求和。单边最大距离为  $2^{65} - 2$ ；在通常 32 位 int 编号上限内，总权远小于 signed 128 位上限。返回原始 0 基编号和 128 位总权；空集与单点返回 0 和空边集。正式题限制  $1 \leq n \leq 200000, 0 \leq x, y \leq 10^9$ ，仅该驱动可安全转回 long long 输出。并查集只在本调用点按大小选择根的合并顺序，不改普通 dsu 的接口语义。

参考：Zhou-Shenoy-Nicholls (2002)，DOI 10.1016/S0020-0190(01)00232-0；四轮 map 实现参考 KACTL `ManhattanMST.h` (CC0，固定版本与字节哈希见来源记录)。原论文半开锥的严格不等式不能直接代入这里的闭锥代码。

## 4.2 最小树形图

### 4.2.1 有向最小树形图

`solve(n, root, edges)`, 点号  $0..n-1$ ,  $n \geq 1$ ,  $0 \leq \text{root} < n$ ; 返回以 `root` 为根向外可达的最小树形图费用, 无解 `nullopt`。允许负权、重边和自环 (自环不选),  $n=1$  的费用为 0; 费用 `-1` 与无解可区分。输入、缩环后边权及累计费用绝对值须小于  $2^{120}$ , 类型为 `int128`。每个非根点选最小入边; 若成环则缩点, 跨环入边减去原终点已选入边费用, 以计算替代代价。 $O(VE)$ , `edges` 按值传入, 原图保持不变。仅返回费用, 不恢复边方案。双版本已做所有根的入边枚举与 800 点连续缩环核验, 暂无专门 OJ AC。

```

1 struct Arborescence
2 {
3     using I = __int128_t;
4
5     struct Edge
6     {
7         int u, v;
8         I w;
9     };
10
11     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
12     static optional<I> solve(int n, int root, vector<Edge> e)
13     {
14         const I inf = I(1) << 120;
15         I ans = 0;
16         while (true)
17         {
18             vector<I> in(n, inf);
19             vector<int> pre(n, -1), id(n, -1), vis(n, -1);
20             for (auto a : e)
21                 if (a.u != a.v && a.w < in[a.v])
22                 {
23                     in[a.v] = a.w;
24                     pre[a.v] = a.u;
25                 }
26             in[root] = 0;
27             for (I x : in)
28             {
29                 if (x == inf) return nullopt;
30                 ans += x;
31             }

```

判环与缩点（接上页同一函数）：

```
32     int cnt = 0;
33     for (int i = 0; i < n; i++)
34     {
35         int u = i;
36         while (vis[u] != i && id[u] == -1 && u != root)
37         {
38             vis[u] = i;
39             u = pre[u];
40         }
41         if (u != root && id[u] == -1)
42         {
43             id[u] = cnt;
44             for (int v = pre[u]; v != u; v = pre[v]) id[v] = cnt;
45             ++cnt;
46         }
47     }
48     if (!cnt) return ans;
49     for (int i = 0; i < n; i++)
50         if (id[i] == -1) id[i] = cnt++;
51     for (auto &a : e)
52     {
53         int v = a.v;
54         a.u = id[a.u];
55         a.v = id[a.v];
56         if (a.u != a.v) a.w -= in[v];
57     }
58     root = id[root];
59     n = cnt;
60 }
61 }
62 };
```

## 模板题使用：Luogu P4716

题意：给定根的有向最小树形图费用，无解输出-1。n 至 100、m 至  $10^4$ 、正权至  $10^6$ ；顶点和根从 1-based 转 0-based。optional 有值时才解引用，本题总费用可转 long long；核心只返回费用，不恢复选边。

所需模板：Arborescence（第 71 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4716>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, r;
6     cin >> n >> m >> r;
7     vector<Arborescence::Edge> e;
8     while (m--)
9     {
10         int u, v;
11         long long w;
12         cin >> u >> v >> w;
13         e.push_back({u - 1, v - 1, w});
14     }
15     auto ans = Arborescence::solve(n, r - 1, e);
16     // This problem's answer fits in long long.
17     cout << (ans ? (long long)*ans : -1) << '\n';
18 }
```

# 第五章 连通性相关

## 5.1 强连通分量

### 5.1.1 Tarjan 强连通分量与缩点

TarjanSCC(n), 点号 1..n; add(u,v) 加有向边, run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号, 跨分量边总从大编号指向小编号, 可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表, 保留重边; 若 DP 需要对不同邻点去重, 调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储  $O(n+m)$ , 递归 DFS 另需  $O(n)$  调用栈; 本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
1 struct TarjanSCC
2 {
3     int n, timer = 0, cnt = 0;
4     vector<vector<int>> g;
5     vector<int> dfn, low, bel, st;
6
7     TarjanSCC(int n) : n(n), g(n + 1) {}
8
9     void add(int u, int v)
10    {
11        assert(1 <= u && u <= n && 1 <= v && v <= n);
12        g[u].push_back(v);
13    }
14
15    void dfs(int u)
16    {
17        dfn[u] = low[u] = ++timer;
18        st.push_back(u);
19        for (int v : g[u])
20        {
21            if (!dfn[v])
22            {
23                dfs(v);
24                low[u] = min(low[u], low[v]);
25            }
26            else if (!bel[v])
27                low[u] = min(low[u], dfn[v]);
28        }
29        if (low[u] == dfn[u])
30        {
31            ++cnt;
32            while (true)
33            {
34                int v = st.back();
35                st.pop_back();
36                bel[v] = cnt;
37                if (v == u) break;
38            }
39        }
40    }
41 }
```

```
42 // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
43 void run()
44 {
45     timer = cnt = 0;
46     dfn.assign(n + 1, 0);
47     low = dfn;
48     bel = dfn;
49     st.clear();
50     for (int u = 1; u <= n; u++)
51     {
52         if (!dfn[u]) dfs(u);
53     }
54 }
55
56 // Retains parallel edges. Use after run().
57 vector<vector<int>> dag() const
58 {
59     vector<vector<int>> result(cnt + 1);
60     for (int u = 1; u <= n; u++)
61     {
62         for (int v : g[u])
63         {
64             if (bel[u] != bel[v]) result[bel[u]].push_back(bel[v]);
65         }
66     }
67     return result;
68 }
69 };
```

## 模板题使用：Library Checker scc

题意：把有向图分成强连通分量，并按缩点图的拓扑顺序输出各分量。题目点号为  $0..n-1$ ，调用时加 1，输出时减 1。

所需模板：TarjanSCC（第 74 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://judge.yosupo.jp/problem/scc>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     TarjanSCC g(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         g.add(u + 1, v + 1);
13     }
14     g.run();
15     vector<vector<int>> c(g.cnt + 1);
16     for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
17     cout << g.cnt << '\n';
18     for (int i = g.cnt; i >= 1; i--)
19     {
20         cout << c[i].size();
21         for (int u : c[i]) cout << ' ' << u;
22         cout << '\n';
23     }
24 }
```

相关应用：函数图包含尾链与环分解，导航无独立页；需正文定位，不应等同一般 SCC 算法。

### 5.1.2 函数图分解、跳转与到达

FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号  $0..n-1$ ；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。 $O(n \log n)$  预处理及空间，入环前查询  $O(\log n)$ ，入环后取模  $O(1)$ ，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。 $n \leq \text{INT\_MAX}$ ，实际规模须满足内存。

```

1 struct FunctionalGraph
2 {
3     using U = unsigned long long;
4     int n;
5     vector<int> component, position, entry, depth, order;
6     vector<vector<int>> cycles, up;
7
8     explicit FunctionalGraph(vector<int> to = {}) { build(move(to)); }
9
10    void build(vector<int> to)
11    {
```

```
12     assert(to.size() <= INT_MAX);
13     n = to.size();
14     vector<int> degree(n);
15     for (int v : to)
16     {
17         assert(0 <= v && v < n);
18         degree[v]++;
19     }
20     order.clear();
21     for (int u = 0; u < n; u++)
22         if (degree[u] == 0) order.push_back(u);
23     for (int i = 0; i < int(order.size()); i++)
24     {
25         int v = to[order[i]];
26         if (--degree[v] == 0) order.push_back(v);
27     }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```

28     component.assign(n, -1);
29     position.assign(n, -1);
30     entry.resize(n);
31     depth.assign(n, 0);
32     cycles.clear();
33     for (int u = 0; u < n; u++)
34         if (degree[u] && component[u] == -1)
35             {
36                 vector<int> cycle;
37                 int v = u, id = cycles.size();
38                 do
39                     {
40                         component[v] = id;
41                         position[v] = cycle.size();
42                         entry[v] = v;
43                         cycle.push_back(v);
44                         v = to[v];
45                     } while (v != u);
46                 cycles.push_back(move(cycle));
47             }
48     for (int i = int(order.size()) - 1; i >= 0; i--)
49     {
50         int u = order[i], v = to[u];
51         component[u] = component[v];
52         entry[u] = entry[v];
53         depth[u] = depth[v] + 1;
54     }
55     up.assign(1, to);
56     for (int k = 1; (1ULL << k) <= U(n); k++)
57     {
58         vector<int> row(n);
59         for (int u = 0; u < n; u++) row[u] = up[k - 1][up[k - 1][u]];
60         up.push_back(move(row));
61     }
62 }
```

跳转与最少步数接口（接上页同一结构体）:

```

64     int advance(int u, U k) const
65     {
66         assert(0 <= u && u < n);
67         if (k >= U(depth[u]))
68         {
69             k -= depth[u];
70             u = entry[u];
71             const auto &cycle = cycles[component[u]];
72             return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
73         }
74         for (int bit = 0; k; bit++, k >>= 1)
75             if (k & 1) u = up[bit][u];
76         return u;
77     }
78
79     int steps(int u, int v) const
80     {
81         assert(0 <= u && u < n && 0 <= v && v < n);
82         if (component[u] != component[v] || depth[u] < depth[v]) return -1;
83         if (depth[v] > 0)
84         {
85             int d = depth[u] - depth[v];
86             return advance(u, d) == v ? d : -1;
87         }
88         int offset = position[v] - position[entry[u]];
89         if (offset < 0) offset += int(cycles[component[u]].size());
90         return depth[u] + offset;
91     }
92 };

```

## 模板题使用：CSES 1750 Planets Queries I

题意：给每点唯一后继，查询从  $x$  走  $k$  步后的点， $n, q \leq 200000$ ， $0 \leq k \leq 10^9$ 。题目 1-based 点号读入减 1、输出加 1； $k$  用 unsigned long long，模板也支持完整 64 位步数。advance 先处理入环链再对环长取模，不能只建 30 层就宣称支持任意 64 位。预处理  $O(n \log n)$ ，查询  $O(\log n)$ ，入环后  $O(1)$ 。

所需模板：FunctionalGraph (第 76 页，5.1.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://cses.fi/problemset/task/1750>

```
1 int main()
2 {
3     int n, q;
4     scanf("%d%d", &n, &q);
5     vector<int> to(n);
6     for (auto &v : to)
7     {
8         scanf("%d", &v);
9         v--;
10    }
11    FunctionalGraph graph(to);
12    while (q--)
13    {
14        int u;
15        unsigned long long k;
16        scanf("%d%llu", &u, &k);
17        printf("%d\n", graph.advance(u - 1, k) + 1);
18    }
19 }
```

## 模板题使用：CSES 1160 Planets Queries II

题意：同样每点一个后继，查询  $u$  到  $v$  的最少步数，不可达输出 -1； $n, q \leq 200000$ 。1-based 读入转 0-based，steps 返回步数，不能加 1；同一点答案 0。处于同一弱连通分量也可能不可达，例如不同入环分支或从环走回链。预处理  $O(n \log n)$ ，单次  $O(\log n)$ 。

所需模板：FunctionalGraph（第 76 页，5.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://cses.fi/problemset/task/1160>

```

1 int main()
2 {
3     int n, q;
4     scanf("%d%d", &n, &q);
5     vector<int> to(n);
6     for (auto &v : to)
7     {
8         scanf("%d", &v);
9         v--;
10    }
11    FunctionalGraph graph(to);
12    while (q--)
13    {
14        int u, v;
15        scanf("%d%d", &u, &v);
16        printf("%d\n", graph.steps(u - 1, v - 1));
17    }
18 }
```

## 应用题补充：Luogu P2921

题意：每点一个后继，输出从各点出发直到首次重复前访问的不同点数，包含起点， $n \leq 100000$ 。结果为入环链长  $\text{depth}[u]$  加所在环的长度；component 是内部 0-based 环编号，不能当成环顶点。自环答案 1。这份用法只读取分解字段，不调用 advance/steps。当前模板含倍增表，建图  $O(n \log n)$ ，输出  $O(n)$ 。

所需模板：FunctionalGraph（第 76 页，5.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P2921>

```

1 int main()
2 {
3     int n;
4     scanf("%d", &n);
5     vector<int> to(n);
6     for (auto &v : to)
7     {
8         scanf("%d", &v);
9         v--;
10    }
11    FunctionalGraph graph(to);
12    for (int u = 0; u < n; u++)
13        printf("%d\n", graph.depth[u] + int(graph.cycles[graph.component[u]].size()));
14 }
```

### 5.1.3 强连通分量

$\text{add}(u,v)$ 、 $\text{run}()$ 、 $\text{bel}[1..n]$  为分量编号， $\text{cnt}$  为数量；分量编号按缩点图拓扑序递增。递归 Kosaraju，按用户要求保留 DFS 写法； $O(V+E)$ ，同时存正反图。

```
1 struct SCC
2 {
3     int n, cnt = 0;
4     vector<vector<int>> g, rg;
5     vector<int> bel;
6
7     SCC(int n) : n(n), g(n + 1), rg(n + 1) {}
8
9     void add(int u, int v)
10    {
11        g[u].push_back(v);
12        rg[v].push_back(u);
13    }
14
15    void dfs1(int u, vector<int> &vis, vector<int> &order)
16    {
17        vis[u] = 1;
18        for (int v : g[u])
19        {
20            if (!vis[v]) dfs1(v, vis, order);
21        }
22        order.push_back(u);
23    }
24
25    void dfs2(int u)
26    {
27        bel[u] = cnt;
28        for (int v : rg[u])
29        {
30            if (!bel[v]) dfs2(v);
31        }
32    }
```

重算入口与拓扑编号（接上页同一结构体）：

```
34 // Recursive Kosaraju; component IDs are in topological order, 1..cnt.
35 void run()
36 {
37     vector<int> vis(n + 1), order;
38     cnt = 0;
39     bel.assign(n + 1, 0);
40     for (int s = 1; s <= n; s++)
41     {
42         if (!vis[s]) dfs1(s, vis, order);
43     }
44     reverse(order.begin(), order.end());
45     for (int s : order)
46     {
47         if (!bel[s])
48         {
49             ++cnt;
50             dfs2(s);
51         }
52     }
53 }
54 };
```

## 模板题使用：Library Checker scc (Kosaraju)

题意：给有向图，输出强连通分量并按缩点拓扑序排列； $N, M \leq 500000$ ，可有重边和自环。题面零基点号先加 1； $SCC.bel$  为  $1..cnt$  且沿跨分量边递增，按  $1..cnt$  输出并将点号减 1。不要照搬 TarjanSCC 的逆序循环。时间、空间  $O(N+M)$ ，DFS 递归。

所需模板：SCC (第 82 页, 5.1.3 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://judge.yosupo.jp/problem/scc>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     SCC g(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         g.add(u + 1, v + 1);
13     }
14     g.run();
15     vector<vector<int>> c(g.cnt + 1);
16     for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
17     cout << g.cnt << '\n';
18     for (int i = 1; i <= g.cnt; i++)
19     {
20         cout << c[i].size();
21         for (int u : c[i]) cout << ' ' << u;
22         cout << '\n';
23     }
24 }
```

## 5.2 双连通分量

### 5.2.1 点双与边双：共享 lowlink 核心

BiconnectedCore(n), 点号 1..n; add(u,v) 返回原边的 0-based 编号, run() 从头计算, 允许重复运行。

点双结果: blocks 为点双的顶点列表, 桥两端成块, 孤立点单独成块, 割点可出现在多个块; cut[u] 标记割点。

边双结果: bridge[id] 标记桥, bel[u] 是 1..cnt 的边双编号; cnt 是全图边双个数。重边按边号处理; 自环保留边号但不进 DFS, 不是桥, 不额外产生点双。一次共享 DFS 后递归染色,  $O(n+m)$  时间和空间, DFS 保持递归。n=0 合法。run 后才可读取结果或构造森林; g、edges 为原图, 不在 DFS 期间修改。

```

1 struct BiconnectedCore
2 {
3     int n, timer = 0, cnt = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<pair<int, int>> edges;
6     vector<vector<int>> blocks;
7     vector<int> dfn, low, cut, bridge, bel, st;
8
9     BiconnectedCore(int n) : n(n), g(n + 1) {}
10
11     int add(int u, int v)
12     {
13         assert(1 <= u && u <= n && 1 <= v && v <= n);
14         int id = edges.size();
15         edges.push_back({u, v});
16         if (u != v)
17         {
18             g[u].push_back({v, id});
19             g[v].push_back({u, id});
20         }
21         return id;
22     }

```

```

24 void dfs(int u, int parent_edge)
25 {
26     dfn[u] = low[u] = ++timer;
27     st.push_back(u);
28     int children = 0;
29     for (auto [v, id] : g[u])
30     {
31         if (id == parent_edge) continue;
32         if (!dfn[v])
33         {
34             children++;
35             dfs(v, id);
36             low[u] = min(low[u], low[v]);
37             if (low[v] > dfn[u]) bridge[id] = 1;
38             if (low[v] >= dfn[u])
39             {
40                 if (parent_edge != -1 || children > 1) cut[u] = 1;
41                 blocks.push_back({u});
42                 while (true)
43                 {
44                     int x = st.back();
45                     st.pop_back();
46                     blocks.back().push_back(x);
47                     if (x == v) break;
48                 }
49             }
50         }
51         else
52             low[u] = min(low[u], dfn[v]);
53     }
54     if (parent_edge == -1 && children == 0) blocks.push_back({u});
55 }

```

```

57 void paint(int u)
58 {
59     bel[u] = cnt;
60     for (auto [v, id] : g[u])
61     {
62         if (!bridge[id] && !bel[v]) paint(v);
63     }
64 }

```

```
66 void run()
67 {
68     timer = cnt = 0;
69     dfn.assign(n + 1, 0);
70     low = dfn;
71     cut = dfn;
72     bel = dfn;
73     bridge.assign(edges.size(), 0);
74     blocks.clear();
75     st.clear();
76     for (int u = 1; u <= n; u++)
77     {
78         if (!dfn[u])
79         {
80             dfs(u, -1);
81             st.pop_back();
82         }
83     }
84     for (int u = 1; u <= n; u++)
85     {
86         if (bel[u]) continue;
87         ++cnt;
88         paint(u);
89     }
90 }
91 };
```

## 模板题使用：Luogu P8435

题意：给定无向图，输出所有点双连通分量的顶点集合；割点可出现在多个集合中，孤立点单独成块。点号 1..n。

所需模板：BiconnectedCore (第 85 页, 5.2.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P8435>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     BiconnectedCore graph(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         std::cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.run();
15     const auto &blocks = graph.blocks;
16     std::cout << blocks.size() << '\n';
17     for (const auto &block : blocks)
18     {
19         std::cout << block.size();
20         for (int u : block) std::cout << ' ' << u;
21         std::cout << '\n';
22     }
23 }
```

## 模板题使用：Luogu P8436

题意：给定无向图，输出所有边双连通分量的顶点集合；利用 `bel` 按边双编号分组。点号  $1..n$ 。

所需模板：[BiconnectedCore](#) (第 85 页, 5.2.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P8436>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     BiconnectedCore graph(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         std::cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.run();
15     std::vector<std::vector<int>>> blocks(graph.cnt);
16     for (int u = 1; u <= n; u++) blocks[graph.bel[u] - 1].push_back(u);
17     std::cout << blocks.size() << '\n';
18     for (const auto &block : blocks)
19     {
20         std::cout << block.size();
21         for (int u : block) std::cout << ' ' << u;
22         std::cout << '\n';
23     }
24 }
```

### 5.2.2 边双缩点森林（保留桥编号）

代码依赖：[BiconnectedCore](#) (第 85 页, 5.2.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`bridge_component_forest(graph)` 接收已 `run` 的 `BiconnectedCore`，返回按  $1..cnt$  编号的边双森林，邻接项为 (相邻分量, 原桥编号)。每条桥变成一条森林边，重边和自环不会误作桥。0 号空置，不连通输入可产生多棵树，孤立边双保留空邻接表。 $O(n+m)$  时间、 $O(n)$  输出空间，不修改分解结果。

```

1 vector<vector<pair<int, int>>> bridge_component_forest(const BiconnectedCore &graph)
2 {
3     int cnt = graph.cnt;
4     const auto &edges = graph.edges;
5     const auto &bridge = graph.bridge;
6     const auto &bel = graph.bel;
7     vector<vector<pair<int, int>>> tree(cnt + 1);
8     for (int id = 0; id < (int)edges.size(); id++)
9     {
```

```

10     if (!bridge[id]) continue;
11     auto [u, v] = edges[id];
12     u = bel[u];
13     v = bel[v];
14     tree[u].push_back({v, id});
15     tree[v].push_back({u, id});
16 }
17 return tree;
18 }

```

## 应用题补充：Luogu P2860 (bridge forest)

题意：连通无向图， $n \leq 5000$ 、 $m \leq 10000$ ；允许已有重边及新增重边，求最少新增边使任意两点间有两条边不相交路径。run 后边双缩点，编号 1..cnt，邻接项为 (边双号, 原桥号)。度 1 点数为  $L$ ，答案  $(L+1)/2$ ；只剩一个边双时为 0。该公式要求原图连通，不可对任意森林直接套用。 $O(n+m)$  时间、空间。

所需模板：BiconnectedCore (第 85 页, 5.2.1 节)、bridge\_component\_forest (第 89 页, 5.2.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P2860>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     BiconnectedCore graph(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.run();
15     auto tree = bridge_component_forest(graph);
16     int leaves = 0;
17     for (int u = 1; u <= graph.cnt; u++)
18         if (tree[u].size() == 1) leaves++;
19     cout << (leaves + 1) / 2 << '\n';
20     return 0;
21 }

```

### 5.2.3 属于简单奇环的全部顶点

代码依赖：BiconnectedCore (第 85 页, 5.2.1 节)、block\_cut\_forest (第 105 页, 5.4.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

先对 BiconnectedCore 调用 run，再调用 odd\_cycle\_vertices(graph)，返回 answer[1..n]，值为 1 表示该点属于某个长度至少为 3 的简单奇环，answer[0]=0。允许重边、断开图及空图；自环不计入这里的奇环。某点双非二分当且仅当其中存在奇环，此时该点双每个顶点都属于某个奇环；答案为所有非二分点双的顶点并集。通过圆方森林把每条非自环边分到唯一点双：两端中森林深度较大者的父方点即该块，同深时二者父方点相同。随后只对块内边染色，避免在每个块中重复扫描割点的全部邻边。两次 DFS 均递归，分解后额外  $O(n+m)$  时间和空间。可重排 blocks，不能改变分解内容。六点以内所有简单图枚举奇环对拍，另有重边/自环、长链、星形和大奇环测试；POJ2942 补图驱动本地验证，在线待补。依赖 biconnected.hpp。

```

1 // After run(). Simple odd cycles have length >= 3; self-loops are ignored.
2 vector<int> odd_cycle_vertices(const BiconnectedCore &graph)
3 {
4     int n = graph.n;
5     auto forest = block_cut_forest(graph);
6     vector<int> parent(forest.size()), depth(forest.size());
7     auto root_tree = [&](auto &&self, int u, int p) -> void
8     {
9         parent[u] = p;
10        for (int v : forest[u])
11            if (v != p)
12            {
13                depth[v] = depth[u] + 1;
14                self(self, v, u);
15            }
16    };
17    for (int u = 1; u <= n; u++)
18        if (!depth[u])
19        {
20            depth[u] = 1;
21            root_tree(root_tree, u, 0);
22        }

```

```

23     vector<vector<pair<int, int>>> edges(graph.blocks.size());
24     for (auto [u, v] : graph.edges)
25         if (u != v)
26         {
27             int block = parent[depth[u] >= depth[v] ? u : v] - n - 1;
28             assert(0 <= block && block < (int)edges.size());
29             edges[block].push_back({u, v});
30         }
31     vector<vector<int>> g(n + 1);
32     vector<int> color(n + 1), answer(n + 1);
33     for (int block = 0; block < (int)edges.size(); block++)
34     {
35         for (int u : graph.blocks[block])
36         {
37             g[u].clear();
38             color[u] = 0;
39         }
40         for (auto [u, v] : edges[block])
41         {
42             g[u].push_back(v);
43             g[v].push_back(u);
44         }
45         bool bipartite = true;
46         auto paint = [&](auto &&self, int u) -> void
47         {
48             for (int v : g[u])
49                 if (!color[v])
50                 {
51                     color[v] = -color[u];
52                     self(self, v);
53                 }
54             else if (color[v] == color[u])
55                 bipartite = false;
56         };
57         int root = graph.blocks[block][0];
58         color[root] = 1;
59         paint(paint, root);
60         if (!bipartite)
61             for (int u : graph.blocks[block]) answer[u] = 1;
62     }
63     return answer;
64 }

```

## 应用题补充：POJ 2942 Knights of the Round Table

题意：互相憎恨的骑士不能相邻，合法圆桌至少 3 人且人数为奇数；求无法参加任何合法圆桌的人数。输入憎恨关系，顶点  $1..n$ ；对  $u < v$  且不憎恨的二人建补图边，不能把憎恨边直接加入。每组新建矩阵和 `BiconnectedCore`，先 `run` 再调用 `odd_cycle_vertices`，统计 `odd[1..n]` 中的 0，每组输出一行；0 0 终止且不输出。函数只返回各点能否出现在某个简单奇环，不给圆桌座次；无合法圆桌时输出 `n`，不是失败。POJ 原站本次不可达；输入格式和样例另与问题 UVA1364 官方 PDF 核对，未据此声称 POJ 限制或线上 AC 已复核。

所需模板：`BiconnectedCore`（第 85 页，5.2.1 节）、`block_cut_forest`（第 105 页，5.4.1 节）、`odd_cycle_vertices`（第 91 页，5.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://poj.org/problem?id=2942>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     while (cin >> n >> m && (n || m))
7     {
8         vector<vector<bool>> hate(n + 1, vector<bool>(n + 1));
9         while (m--)
10         {
11             int u, v;
12             cin >> u >> v;
13             hate[u][v] = true;
14             hate[v][u] = true;
15         }
16         BiconnectedCore graph(n);
17         for (int u = 1; u <= n; u++)
18             for (int v = u + 1; v <= n; v++)
19                 if (!hate[u][v]) graph.add(u, v);
20         graph.run();
21         auto odd = odd_cycle_vertices(graph);
22         int answer = 0;
23         for (int u = 1; u <= n; u++) answer += !odd[u];
24         cout << answer << '\n';
25     }
26     return 0;
27 }

```

### 5.2.4 边双缩点的大小与内部边数

代码依赖：BiconnectedCore（第 85 页，5.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

在 BiconnectedCore.run() 后构造 EdgeCompression(graph)。n 为边双数量；size[c] 为第 c 块的原顶点数，inside[c] 为块内逻辑无向边数，分量号 1..n，0 号槽忽略。edges 每项为 {块 u, 块 v, 原边 id}，只含跨块的桥，每条桥出现一次，按原边号递增；是无向森林，不承诺端点为父子方向。重边逐条计数，自环在所属块内部计一次，不能用邻接表度数直接计边而漏掉这些约定。空图返回 n=0、长度 1 的零计数数组和空边表。时间  $O(N+M)$ ，输出空间  $O(N+M)$ ，N/M 指原图规模。

```

1 struct EdgeCompression
2 {
3     int n;
4     vector<int> size, inside;
5     vector<array<int, 3>> edges;
6
7     EdgeCompression(const BiconnectedCore &graph)
8         : n(graph.cnt), size(n + 1), inside(n + 1)
9     {
10         for (int u = 1; u <= graph.n; u++) size[graph.bel[u]]++;
11         for (int id = 0; id < (int)graph.edges.size(); id++)
12         {
13             auto [u, v] = graph.edges[id];
14             u = graph.bel[u];
15             v = graph.bel[v];
16             if (u == v)
17                 inside[u]++;
18             else
19                 edges.push_back({u, v, id});
20         }
21     }
22 };

```

## 应用题补充：Codeforces 732F Tourist Reform

题意：连通无向简单图， $2 \leq n \leq 400000$ 、 $1 \leq m \leq 400000$ ，给每条边定向，使各点可达点数的最小值最大（含自身）。输出最优值和按原输入顺序的  $m$  条有向边。先求边双缩点，选 size 最大的块为根；orient\_edges 定向块内边，再按 edges 中的原边编号把桥全部朝根定向。答案是最大块大小，不把缩点边端点顺序当成父子关系。 $O(n+m)$  时间/空间；保留递归核心，需足够大栈，局部 8 MiB 栈在最大链上溢出，评测栈未知。本题是应用，不是正式缩点模板题。

所需模板：BiconnectedCore (第 85 页, 5.2.1 节)、EdgeCompression (第 94 页, 5.2.4 节)、Lowlink (第 102 页, 5.3.3 节)、orient\_edges (第 97 页, 5.2.5 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://codeforces.com/problemset/problem/732/F>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     if (!(std::cin >> n >> m)) return 0;
7     BiconnectedCore core(n);
8     Lowlink graph(n);
9     for (int i = 0; i < m; i++)
10    {
11        int u, v;
12        std::cin >> u >> v;
13        core.add(u, v);
14        graph.add(u, v); // Same insertion order preserves logical edge IDs.
15    }
16    // Existing recursive cores require a sufficiently large process stack.
17    // An 8 MiB stack fails at n=400000; judge stack availability is not assumed.
18    core.run();
19    EdgeCompression compressed(core);
20    int root = 1;
21    for (int c = 2; c <= compressed.n; c++)
22        if (compressed.size[c] > compressed.size[root]) root = c;
23    auto direction = orient_edges(graph); // Does not need graph.run().
24    vector<vector<pair<int, int>>> tree(compressed.n + 1);
25    for (auto [u, v, id] : compressed.edges)
26    {
27        tree[u].push_back({v, id});
28        tree[v].push_back({u, id}); // Compressed endpoints are not parent/child.
29    }

```

继续处理操作（接上页同一 main）：

```
30     vector<int> parent(compressed.n + 1), order{root};
31     parent[root] = -1;
32     for (int i = 0; i < (int)order.size(); i++)
33     {
34         int u = order[i];
35         for (auto [v, id] : tree[u])
36         {
37             if (parent[v]) continue;
38             parent[v] = u;
39             order.push_back(v);
40             auto [a, b] = core.edges[id];
41             if (core.bel[a] != v) swap(a, b);
42             direction[id] = {a, b}; // Original bridge points child -> parent.
43         }
44     }
45     std::cout << compressed.size[root] << '\n';
46     for (auto [u, v] : direction) std::cout << u << ' ' << v << '\n';
47     return 0;
48 }
```

### 5.2.5 无向边定向与强连通定向

代码依赖：Lowlink (第 102 页, 5.3.3 节)。抄写时一并准备；标准库类型不列入算法依赖。

`orient_edges(graph)` 接受 Lowlink 对象。返回长度等于逻辑边数的方向数组，第 `id` 项为该边定向后的 {起点, 终点}，顶点 1-based、边 `id` 为 0-based。每条无向边仅处理一次：DFS 树边向下，非树边指向祖先，自环保留 {u,u}，重边不合并。支持非连通图；每个边双块内定向后强连通，桥只给出某个方向，不保证跨桥互相可达。若要求整个非空图强连通，需另外 `run/Run` 检查 `components==1` 且无桥；不满足则不存在这样的方向。该函数只读取邻接表与边数，不依赖已有 DFS 状态，不修改原图，调用时无须先 `run`。复杂度  $O(N+M)$  时间、 $O(N+M)$  额外空间，DFS 保持递归。

```

1 // Tree edges go down; other edges go to ancestors. Handles each logical edge once.
2 vector<pair<int, int>> orient_edges(const Lowlink &graph)
3 {
4     vector<pair<int, int>> direction(graph.edges);
5     vector<int> seen(graph.n + 1);
6     auto dfs = [&](auto &&self, int u) -> void
7     {
8         seen[u] = 1;
9         for (auto [v, id] : graph.g[u])
10         {
11             if (direction[id].first) continue;
12             direction[id] = {u, v};
13             if (!seen[v]) self(self, v);
14         }
15     };
16     for (int u = 1; u <= graph.n; u++)
17         if (!seen[u]) dfs(dfs, u);
18     return direction;
19 }
```

## 应用题补充：Codeforces 118E Bertown roads

题意：给连通无向简单图，把每条边定向使整图强连通，无解输出 0。 $n \leq 100000$ 、 $m \leq 300000$ 。先用 Lowlink.run 确认连通且无桥，再调用 orient\_edges；返回数组与 add 的原边编号逐项对应，每条逻辑边只加一次。任意可行方向均可。 $O(n+m)$  时间、空间，DFS 递归；比赛应用不替代正式模板题。

所需模板：Lowlink（第 102 页，5.3.3 节）、orient\_edges（第 97 页，5.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://codeforces.com/problemset/problem/118/E>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     Lowlink graph(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         std::cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.run();
15     if (graph.components != 1 ||
16         std::count(graph.bridge.begin(), graph.bridge.end(), 1))
17     {
18         std::cout << 0 << '\n';
19         return 0;
20     }
21     for (auto [u, v] : orient_edges(graph)) std::cout << u << ' ' << v << '\n';
22     return 0;
23 }
```

## 5.3 割点和桥

### 5.3.1 由点双分解计算删点连通块数

代码依赖：BiconnectedCore（第 85 页，5.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

在 BiconnectedCore.run() 后调用 removal\_components(graph)，返回 (原连通块数 before, 删点结果 after)。after[u] 为独立删除原顶点 u 后剩余连通块数，点号 1..n；after[0]=0 不参与计算。before 等于边双数量减去桥数量；after[u] 等于 before-1 加包含 u 的非单点点双数量。单点块必须跳过，因此孤立点删去后会少一个连通块。空图返回 (0,{0})。不修改原分解， $O(n+m)$  时间、 $O(n)$  额外空间。若只需要割点、桥和删点增量，直接用更短的 Lowlink/Low\_Link；已经需要点双分解时可用本函数，避免再次 DFS。

```

1 // Call after run(). Each vertex is deleted independently from the original graph.
2 pair<int, vector<int>> removal_components(const BiconnectedCore &graph)
```

```

3 {
4     int before = graph.cnt - count(graph.bridge.begin(), graph.bridge.end(), 1);
5     vector<int> after(graph.n + 1, before - 1);
6     after[0] = 0;
7     for (const auto &block : graph.blocks)
8         if (block.size() > 1)
9             for (int u : block) after[u]++;
10    return {before, after};
11 }

```

## 模板题使用：AOJ GRL\_3\_A

题意：连通无向简单图，按编号升序输出所有割点， $N, M \leq 100000$ 。输入零基转一基；BiconnectedCore.run 后取 (before, after)，仅当  $\text{after}[u] > \text{before}$  才输出  $u-1$ 。本例展示已有点双结果的复用；只求割点优先用更短的 Lowlink。时间、空间  $O(N+M)$ ，递归 DFS。该题只核验割点集合，完整删点数量另见 UVA10765 应用。

所需模板：BiconnectedCore（第 85 页，5.2.1 节）、removal\_components（第 98 页，5.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL\\_3\\_A&lang=en](https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_3_A&lang=en)

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     BiconnectedCore g(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         g.add(u + 1, v + 1);
13     }
14     g.run();
15     auto [before, after] = removal_components(g);
16     for (int u = 1; u <= n; u++)
17         if (after[u] > before) cout << u - 1 << '\n';
18 }

```

## 应用题补充：UVA 10765 Doves and Bombs

题意：分别删去每个站点，按剩余连通块数降序、原编号升序输出前  $m$  名； $n \leq 10000$ ，原图连通。 $m$  是输出名而非边数，边表以 -1 -1 结束，多测以 0 0 结束，每组末尾空行。after 是剩余总块数而非增量，不重复加 before。 $O(n \log n + E)$  时间、 $O(n + E)$  空间。应用题不计正式模板覆盖。

所需模板：[BiconnectedCore](#)（第 85 页，5.2.1 节）、[removal\\_components](#)（第 98 页，5.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://onlinejudge.org/external/107/10765.pdf>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     while (cin >> n >> m && n)
7     {
8         BiconnectedCore g(n);
9         int u, v;
10        while (cin >> u >> v && u != -1) g.add(u + 1, v + 1);
11        g.run();
12        auto [before, after] = removal_components(g);
13        vector<pair<int, int>> a;
14        for (int u = 1; u <= n; u++) a.push_back({-after[u], u - 1});
15        sort(a.begin(), a.end());
16        for (int i = 0; i < m; i++) cout << a[i].second << ' ' << -a[i].first << '\n';
17        cout << '\n';
18    }
19 }
```

### 5.3.2 连通无向图最少加边方案

代码依赖：BiconnectedCore (第 85 页, 5.2.1 节)、bridge\_component\_forest (第 89 页, 5.2.2 节)。抄写时一并准备；标准库类型不列入算法依赖。

先对非空连通无向图调用 BiconnectedCore.run(), 再调用 bridge\_augmentation(graph)。返回原图点号构成的新增边列表, 不修改原图。允许新增重边, 不产生自环; 不适用于非连通图。已无桥时返回空列表。桥树有  $L$  个叶子时最少加  $(L+1)/2$  条边 (整数除法)。递归 DFS 顺序收集叶子, 奇数时复制首叶, 前后半段配对。每条桥一侧的叶子是循环连续区间, 半周配对必跨过该割, 故所有桥被消除; 每个叶子至少需要一个新边端点, 得到同样的下界。分解后本函数  $O(n+m)$  时间与空间。依赖 bridge\_component\_forest.hpp。P2860 的 vector 用法已通过普通及 ASan/UBSan 本地验证; 新增方案以逐边删除连通性及小图最优解枚举独立核验, 该应用尚无本库在线 AC。

```

1 // Connected, nonempty graph after run(); new parallel edges are allowed.
2 vector<pair<int, int>> bridge_augmentation(const BiconnectedCore &graph)
3 {
4     assert(graph.n > 0 && graph.cnt > 0);
5     int bridges = count(graph.bridge.begin(), graph.bridge.end(), 1);
6     assert(graph.cnt - bridges == 1);
7     if (graph.cnt == 1) return {};
8     auto tree = bridge_component_forest(graph);
9     vector<int> representative(graph.cnt + 1);
10    for (int u = 1; u <= graph.n; u++) representative[graph.bel[u]] = u;
11    vector<int> leaves;
12    auto dfs = [&](auto &&self, int u, int parent) -> void
13    {
14        if (tree[u].size() == 1) leaves.push_back(u);
15        for (auto [v, id] : tree[u])
16            if (v != parent) self(self, v, u);
17    };
18    dfs(dfs, 1, 0);
19    if (leaves.size() % 2) leaves.push_back(leaves[0]);
20    int half = leaves.size() / 2;
21    vector<pair<int, int>> added;
22    for (int i = 0; i < half; i++)
23        added.push_back({representative[leaves[i]], representative[leaves[i + half]]});
24    return added;
25 }
```

## 应用题补充：Luogu P2860 (construct augmentation)

题意：同题的构造接口：连通非空图 run 后调用 `bridge_augmentation`，返回最少补边方案的原点端点对，点号 `1..n`，可直接逐对使用；题目只输出方案长度。已有边和新增边都允许平行边。接口不修改 `graph`，若需更新原图须自行 add 并重新 run。 $O(n+m)$  时间、空间；DFS 顺序收集桥树叶后对半配对，奇数叶子补首叶。

所需模板：[BiconnectedCore](#) (第 85 页, 5.2.1 节)、[bridge\\_component\\_forest](#) (第 89 页, 5.2.2 节)、[bridge\\_augmentation](#) (第 101 页, 5.3.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P2860>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     BiconnectedCore graph(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.run();
15     cout << bridge_augmentation(graph).size() << '\n';
16     return 0;
17 }
```

### 5.3.3 割点与桥

无向多重图，点号 `1..n`；`add/Insert(u,v)` 返回逻辑无向边的 0-based 编号，自环和重边均可输入。run/Run 后 `cut[u]` 为割点标记，`bridge[id]` 为桥标记，`components` 为原图连通块数。`delta[u]` 是独立删除顶点 `u` 及其关联边后增加的连通块数，剩余块数为 `components+delta[u]`；孤立点 `delta=-1`，普通非割点为 0，割点为正。根的增量为 DFS 子树数减 1；非根统计满足 `low[child]>=dfn[u]` 的子树数。`n=1` 的点删除后剩余 0 块。DFS 按父边号跳过，支持重边、非连通图和重复运行。复杂度  $O(n+m)$ ，递归 DFS；新增边后需重新 run。每个删点结果都针对原图，不表示逐次删除。

```

1 struct Lowlink
2 {
3     int n, timer = 0, components = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<int> dfn, low, cut, bridge, delta;
6     int edges = 0;
7
8     Lowlink(int n) : n(n), g(n + 1) {}
9
10    int add(int u, int v)
11    {
12        g[u].push_back({v, edges});
13        g[v].push_back({u, edges});
14        return edges++;
15    }
```

```

17 void dfs(int u, int pe)
18 {
19     dfn[u] = low[u] = ++timer;
20     int children = 0;
21     for (auto [v, id] : g[u])
22     {
23         if (id == pe) continue;
24         if (!dfn[v])
25         {
26             ++children;
27             dfs(v, id);
28             low[u] = min(low[u], low[v]);
29             if (low[v] > dfn[u]) bridge[id] = 1;
30             if (pe != -1 && low[v] >= dfn[u])
31             {
32                 cut[u] = 1;
33                 delta[u]++;
34             }
35         }
36         else
37             low[u] = min(low[u], dfn[v]);
38     }
39     if (pe == -1 && children > 1) cut[u] = 1;
40     if (pe == -1) delta[u] = children - 1;
41 }
42
43 void run()
44 {
45     timer = 0;
46     components = 0;
47     dfn.assign(n + 1, 0);
48     low = dfn;
49     cut = dfn;
50     delta = dfn;
51     bridge.assign(edges, 0);
52     for (int u = 1; u <= n; u++)
53         if (!dfn[u])
54         {
55             components++;
56             dfs(u, -1);
57         }
58 }
59 };

```

## 模板题使用：Luogu P3388

题意：给无向图，按编号递增输出全部割点。图可以不连通；run 后读取 cut[u] 标记。不要把 bridge 的边编号当作顶点标记。点号 1..n。

所需模板：Lowlink（第 102 页，5.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3388>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     Lowlink graph(n);
8     while (m--)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.run();
15     vector<int> answer;
16     for (int u = 1; u <= n; u++)
17         if (graph.cut[u]) answer.push_back(u);
18     cout << answer.size() << '\n';
19     for (int u : answer) cout << u << ' ';
20     cout << '\n';
21 }
```

## 5.4 圆方树

### 5.4.1 由点双构造圆方森林

代码依赖：BiconnectedCore（第 85 页，5.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

block\_cut\_forest(graph) 接收已经 run 的 BiconnectedCore，返回 1-based 邻接表。原点为 1..n，blocks[i] 的方点为 n+i+1；每个点双内的所有原点连向该方点。孤立点也连向自己的单点方块。不连通原图得到森林；0 号空置。O(n+m) 时间与输出空间，不修改分解结果。

```

1 vector<vector<int>> block_cut_forest(const BiconnectedCore &graph)
2 {
3     int n = graph.n;
4     const auto &blocks = graph.blocks;
5     vector<vector<int>> tree(n + blocks.size() + 1);
6     for (int i = 0; i < (int)blocks.size(); i++)
7     {
8         int b = n + i + 1;
9         for (int u : blocks[i])
10        {
11            tree[u].push_back(b);
12            tree[b].push_back(u);
13        }
14    }
15    return tree;
16 }
```

### 应用题补充：Luogu P4630

题意：无向简单图，可不连通， $n \leq 100000$ 、 $m \leq 200000$ ；统计存在经过 c 的简单 s-f 路径的有序三元组 (s,c,f)，三点互异。原点 1..n 权为 -1，方点 n+i+1 权为 blocks[i].size()，树路径权和就是可选 c 数。sz 仅计原点，每棵树独立累计有序端点对，答案用 long long。O(n+m) 时间、空间，递归 DFS。

所需模板：BiconnectedCore（第 85 页，5.2.1 节）、block\_cut\_forest（第 105 页，5.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4630>

## Luogu P4630: 使用代码

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     BiconnectedCore graph(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     graph.run();
15     auto tree = block_cut_forest(graph);
16     vector<int> sz(tree.size());
17     auto dfs = [&](auto &&self, int u, int p) -> void
18     {
19         sz[u] = (u <= n);
20         for (int v : tree[u])
21         {
22             if (v == p) continue;
23             self(self, v, u);
24             sz[u] += sz[v];
25         }
26     };
27     long long ans = 0;
28     auto calc = [&](auto &&self, int u, int p, int total) -> void
29     {
30         long long sum = total - sz[u];
31         long long cnt = u <= n ? 2LL * (total - 1) : 0;
32         for (int v : tree[u])
33         {
34             if (v == p) continue;
35             cnt += 2 * sum * sz[v];
36             sum += sz[v];
37             self(self, v, u, total);
38         }
39         long long w = u <= n ? -1 : (int)tree[u].size();
40         ans += w * cnt;
41     };
42     for (int u = 1; u <= n; u++)
43     {
44         if (sz[u]) continue;
45         dfs(dfs, u, 0);
46         calc(calc, u, 0, sz[u]);
47     }
48     cout << ans << '\n';
49     return 0;
50 }

```

## 第六章 环计数问题

### 6.1 无向简单图三元环枚举

`enumerate_triangles(n, edges, visit)` 使用  $0..n-1$  点号; `edges` 须为无向简单图, 无自环、无重边, 每条边只给一次, 端点顺序任意。每个三角形恰调用一次 `visit(u, v, w)`, 参数按度数和编号组成的次序排列, 不保证编号递增。可在回调中计数、累计权值或收集方案; 计数用 `long long`。按 (度数, 编号) 从小到大定向, 标记 `u` 的出邻点后枚举 `u` 到 `v` 到 `w` 的有向路径。每个出度为  $O(\sqrt{m})$ , 总时间  $O(n+m\sqrt{m})$ , 含常数时间回调, 空间  $O(n+m)$ ; 空图合法。调用期间不修改输入边集。

```
1 template <class F>
2 void enumerate_triangles(int n, const vector<pair<int, int>> &edges, F visit)
3 {
4     assert(n >= 0);
5     vector<int> deg(n), mark(n, -1);
6     vector<vector<int>> g(n);
7     for (auto [u, v] : edges)
8     {
9         assert(0 <= u && u < n && 0 <= v && v < n && u != v);
10        deg[u]++, deg[v]++;
11    }
12    for (auto [u, v] : edges)
13    {
14        if (pair{deg[u], u} > pair{deg[v], v}) swap(u, v);
15        g[u].push_back(v);
16    }
17    for (int u = 0; u < n; u++)
18    {
19        for (int v : g[u]) mark[v] = u;
20        for (int v : g[u])
21            for (int w : g[v])
22                if (mark[w] == u) visit(u, v, w);
23    }
24 }
```

## 模板题使用：Library Checker `enumerate_triangles`

题意：给简单无向图的每个点一个权值，求所有三角形三个点权乘积之和，模 998244353。点号  $0..n-1$ 。

所需模板：`enumerate_triangles`（第 107 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：[https://judge.yosupo.jp/problem/enumerate\\_triangles](https://judge.yosupo.jp/problem/enumerate_triangles)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<long long> x(n);
8     for (auto &v : x) cin >> v;
9     vector<pair<int, int>> edges(m);
10    for (auto &[u, v] : edges) cin >> u >> v;
11    const long long mod = 998244353;
12    long long ans = 0;
13    enumerate_triangles(n,
14                        edges,
15                        [&](int u, int v, int w)
16                        { ans = (ans + x[u] * x[v] % mod * x[w]) % mod; });
17    cout << ans << '\n';
18 }
```

# 第七章 2-SAT

## 7.1 2-SAT 与任意可行方案

代码依赖：SCC (第 82 页, 5.1.3 节)。抄写时一并准备；标准库类型不列入算法依赖。

TwoSAT(n), 变量 1..n; add(x,a,y,b) 添加 (x=a) 或 (y=b), 强制值写 add(x,a,x,a), 蕴含 (x=a)=>(y=b) 写 add(x,!a,y,b)。solve() 返回可满足性, 仅成功时读取 ans[1..n]; 可重复求解和追加子句后重算, 空公式可满足。不保证字典序最小。依赖同文件 SCC: Kosaraju 编号为拓扑序, 故真值分量编号较大时取真; 不能直接套用 Tarjan 逆拓扑编号的不等号。O(n+m) 时间和空间, DFS 递归。

```
1 struct TwoSAT
2 {
3     int n;
4     SCC g;
5     vector<int> ans;
6
7     TwoSAT(int n) : n(n), g(2 * n) {}
8
9     int id(int x, bool value) const { return 2 * x - 1 + value; }
10
11     void add(int x, bool a, int y, bool b)
12     {
13         g.add(id(x, !a), id(y, b));
14         g.add(id(y, !b), id(x, a));
15     }
16
17     bool solve()
18     {
19         g.run();
20         ans.assign(n + 1, 0);
21         for (int x = 1; x <= n; x++)
22         {
23             if (g.bel[id(x, 0)] == g.bel[id(x, 1)]) return false;
24             ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
25         }
26         return true;
27     }
28 };
```

## 模板题使用：Library Checker two\_sat

题意：每个子句是两个文字的或，判断是否可满足并输出一组赋值。输入为 DIMACS：首行  $p \text{ cnf } n \text{ m}$ ，正数表示变量，负数表示取反，每个子句末尾为 0。

所需模板：SCC（第 82 页，5.1.3 节）、TwoSAT（第 109 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：[https://judge.yosupo.jp/problem/two\\_sat](https://judge.yosupo.jp/problem/two_sat)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string p, cnf;
6     int n, m;
7     cin >> p >> cnf >> n >> m;
8     TwoSAT g(n);
9     for (int i = 0; i < m; i++)
10     {
11         int a, b, zero;
12         cin >> a >> b >> zero;
13         g.add(abs(a), a > 0, abs(b), b > 0);
14     }
15     if (!g.solve())
16     {
17         cout << "s UNSATISFIABLE\n";
18         return 0;
19     }
20     cout << "s SATISFIABLE\n";
21     for (int i = 1; i <= n; i++) cout << ' ' << (g.ans[i] ? i : -i);
22     cout << " 0\n";
23 }
```

## 模板题使用：Luogu P4782

题意：n 个布尔变量，m 个  $(x=a)$  或  $(y=b)$  子句，输出任意满足全部子句的赋值或 IMPOSSIBLE。变量号 1..n； $n, m \leq 10^6$ 。同一变量的重复文字照常 add，成功后才读取 ans。

所需模板：SCC (第 82 页, 5.1.3 节)、TwoSAT (第 109 页, 7.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4782>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     TwoSAT sat(n);
8     while (m--)
9     {
10         int x, a, y, b;
11         std::cin >> x >> a >> y >> b;
12         sat.add(x, a, y, b);
13     }
14     if (!sat.solve())
15     {
16         std::cout << "IMPOSSIBLE\n";
17         return 0;
18     }
19     std::cout << "POSSIBLE\n";
20     for (int i = 1; i <= n; i++) std::cout << sat.ans[i] << (i == n ? '\n' : ' ');
21     return 0;
22 }
```

## 7.2 递归染色与回滚求最小赋值

构造 LexTwoSAT( $n$ ), 变量号  $1..n$ 。add( $x,a,y,b$ ) 加入子句  $(x=a) \text{ OR } (y=b)$ , force( $x,v$ ) 强制变量值; solve() 成功后 answer[ $1..n$ ] 是按变量编号、0 优先于 1 的字典序最小赋值。失败时 answer 为空,  $n=0$  时可满足, answer[0] 始终仅为占位。每个未确定变量先尝试 0, 递归传播所有蕴含; 冲突时仅撤销本次新增标记, 再试 1。history 是撤销记录, 不替代递归调用栈。可重复求解, 追加子句会清空旧答案; marked/history 仅为内部工作区, 失败时不可当作方案。内部 id( $x,v$ )= $2(x-1)+v$ , 从 0 开始, 可用异或 1 取反; 不要混用另一份 SCC 2-SAT 的 1-based 字面量编号。最坏  $O(n(n+m))$  时间、 $O(n+m)$  空间, 递归栈  $O(n)$ ; 100000 变量测试是特定蕴含链, 不代表一般大规模输入的性能保证。HDU1814 完整程序本地对拍, 在线待补。

```

1 struct LexTwoSAT
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<int> marked, history, answer;
6
7     LexTwoSAT(int n) : n(n), g(2 * n), marked(2 * n) { assert(n >= 0); }
8
9     int id(int x, bool value) const
10    {
11        assert(1 <= x && x <= n);
12        return 2 * (x - 1) + value;
13    }
14
15    // (x == a) OR (y == b). Internal literal IDs are 0-based.
16    void add(int x, bool a, int y, bool b)
17    {
18        int u = id(x, a), v = id(y, b);
19        g[u ^ 1].push_back(v);
20        g[v ^ 1].push_back(u);
21        answer.clear();
22    }
23
24    void force(int x, bool value) { add(x, value, x, value); }

```

```
26 bool paint(int u)
27 {
28     if (marked[u ^ 1]) return false;
29     if (marked[u]) return true;
30     marked[u] = 1;
31     history.push_back(u);
32     for (int v : g[u])
33         if (!paint(v)) return false;
34     return true;
35 }
36
37 bool solve()
38 {
39     fill(marked.begin(), marked.end(), 0);
40     history.clear();
41     answer.clear();
42     for (int x = 1; x <= n; x++)
43     {
44         int u = id(x, false);
45         if (marked[u] || marked[u ^ 1]) continue;
46         int checkpoint = history.size();
47         if (!paint(u))
48         {
49             while ((int)history.size() > checkpoint)
50             {
51                 marked[history.back()] = 0;
52                 history.pop_back();
53             }
54             if (!paint(u ^ 1)) return false;
55         }
56     }
57     answer.assign(n + 1, 0);
58     for (int x = 1; x <= n; x++) answer[x] = marked[id(x, true)];
59     return true;
60 }
61 };
```

## 应用题补充：HDU 1814 Peaceful Commission

题意：每党两名候选人编号  $2x-1$ 、 $2x$ ，恰选一人且冲突双方不能同时入选，输出字典序最小代表序列。候选人  $u$  转为变量  $x=(u+1)/2$ ，偶号对应值 1；冲突  $(u,v)$  转为  $(x \neq a) \text{ OR } (y \neq b)$ ，即  $\text{add}(x,!a,y,!b)$ 。变量与  $\text{answer}$  均为 1-based；按党号输出  $2*x-1+\text{answer}[x]$ ，每人一行。无解只输出 NIE，不能读取失败后的  $\text{answer}$ 。多组读至 EOF，每组重新构造，无 0 0 结束标记。原站题面本次不可达，题意与格式依据现有驱动及独立枚举测试，不声称已复核官方限制或线上 AC。

所需模板：LexTwoSAT (第 112 页，7.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://acm.hdu.edu.cn/showproblem.php?pid=1814>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     while (cin >> n >> m)
7     {
8         LexTwoSAT graph(n);
9         while (m--)
10         {
11             int u, v;
12             cin >> u >> v;
13             int x = (u + 1) / 2;
14             int y = (v + 1) / 2;
15             bool a = u % 2 == 0;
16             bool b = v % 2 == 0;
17             graph.add(x, !a, y, !b);
18         }
19         if (!graph.solve())
20             cout << "NIE\n";
21         else
22             for (int x = 1; x <= n; x++) cout << 2 * x - 1 + graph.answer[x] << '\n';
23     }
24     return 0;
25 }
```

# 第八章 欧拉图

## 8.1 有向图字典序最小欧拉路

构造 `DirectedEuler(n)`,  $n \geq 1$ , 点号  $1..n$ ; `add(u,v)` 返回从 0 开始的逻辑边编号, 支持自环与重边。`run()` 返回是否存在欧拉路; 参数 `start` 默认 0, `lexicographic` 默认 `true`。`start=0` 自动选择最小可行起点, 否则固定起点; 孤立点不影响有边部分的连通性。成功后 `vertices` 长度为  $m+1$ , `edge_ids` 长度为  $m$ , 第  $i$  条原边从 `vertices[i]` 指向 `vertices[i+1]`; 失败清空两数组。无边时返回单点路径, 自动起点为 1。可重复 `run`, `add` 会清空旧方案。默认求顶点序列字典序最小方案; 关闭排序后只保证任意方案, 邻接顺序可能保留上次排序。开启排序总时间  $O(n+m \log(m+1))$ , 否则  $O(n+m)$ , 空间  $O(n+m)$ 。DFS 递归深度可能达到  $m+1$ , 不是  $n$ 。度数检查后, 以消费边数判定是否遍历全部非孤立部分, 逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号, P7771 双风格完整程序本地验证, 在线待补。

```
1 struct DirectedEuler
2 {
3     int n;
4     vector<vector<pair<int, int>>>> g;
5     vector<pair<int, int>> edges;
6     vector<int> in, out, next, vertices, edge_ids;
7
8     DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
9     {
10         assert(n > 0);
11     }
12
13     int add(int u, int v)
14     {
15         assert(1 <= u && u <= n && 1 <= v && v <= n);
16         int id = edges.size();
17         edges.push_back({u, v});
18         g[u].push_back({v, id});
19         out[u]++;
20         in[v]++;
21         vertices.clear();
22         edge_ids.clear();
23         return id;
24     }
25
26     void dfs(int u, int incoming)
27     {
28         while (next[u] < (int)g[u].size())
29         {
30             auto [v, id] = g[u][next[u]++];
31             dfs(v, id);
32         }
33         vertices.push_back(u);
34         if (incoming != -1) edge_ids.push_back(incoming);
35     }
```

```

37 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
38 bool run(int start = 0, bool lexicographic = true)
39 {
40     assert(0 <= start && start <= n);
41     vertices.clear();
42     edge_ids.clear();
43     int source = 0, sink = 0;
44     for (int u = 1; u <= n; u++)
45     {
46         int delta = out[u] - in[u];
47         if (delta == 1 && !source)
48             source = u;
49         else if (delta == -1 && !sink)
50             sink = u;
51         else if (delta != 0)
52             return false;
53     }
54     if (bool(source) != bool(sink)) return false;
55     if (source)
56     {
57         if (start && start != source) return false;
58         start = source;
59     }
60     if (!start)
61     {
62         start = 1;
63         for (int u = 1; u <= n; u++)
64             if (out[u])
65             {
66                 start = u;
67                 break;
68             }
69     }
70     if (lexicographic)
71         for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
72     fill(next.begin(), next.end(), 0);
73     dfs(start, -1);
74     if (edge_ids.size() != edges.size())
75     {
76         vertices.clear();
77         edge_ids.clear();
78         return false;
79     }
80     reverse(vertices.begin(), vertices.end());
81     reverse(edge_ids.begin(), edge_ids.end());
82     return true;
83 }
84 };

```

## 模板题使用：Luogu P7771

题意：给有向图，求字典序最小的欧拉路径。run 默认选择可行起点并按字典序遍历；成功后 vertices 含  $m+1$  个顶点，失败输出 No。点号  $1..n$ 。

所需模板：DirectedEuler（第 115 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P7771>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     DirectedEuler graph(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     if (!graph.run())
15         cout << "No\n";
16     else
17     {
18         for (int u : graph.vertices) cout << u << ' ';
19         cout << '\n';
20     }
21     return 0;
22 }
```

## 8.2 字典序最小单词链

代码依赖：DirectedEuler (第 115 页, 8.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

word\_chain(words) 返回可空的编号序列 (optional), 有解时为原输入的 0-based 单词编号顺序, 每个单词恰用一次；相邻单词首尾字母相接, 单词序列字典序最小。无解返回 nullopt, 空输入返回有值的空序列。单词必须非空且仅含小写 a-z, 重复词视为不同编号。先对原编号按单词排序, 按此顺序加边, 再调用 DirectedEuler.run(0,false); 不可让底层重新按终点排序。欧拉回路从最小出现首字母开始, 非回路起点由度数差确定。O(m log m) 次字符串比较及 O(L+m) 其他工作, L 为总字符数；空间 O(m), 递归栈 O(m)。依赖 directed\_euler.hpp。全排列对拍、重复词/前缀词、20 万相同词及 POJ2337 完整输出本地验证, 在线待补。

```

1 // Lowercase nonempty words. Returns original 0-based IDs; duplicates stay distinct.
2 optional<vector<int>> word_chain(const vector<string> &words)
3 {
4     int n = words.size();
5     vector<int> order(n);
6     iota(order.begin(), order.end(), 0);
7     for (const auto &word : words)
8     {
9         assert(!word.empty());
10        for (char c : word) assert('a' <= c && c <= 'z');
11    }
12    sort(order.begin(),
13         order.end(),
14         [&](int a, int b)
15         {
16             if (words[a] != words[b]) return words[a] < words[b];
17             return a < b;
18         });
19    DirectedEuler graph(26);
20    for (int id : order)
21        graph.add(words[id].front() - 'a' + 1, words[id].back() - 'a' + 1);
22    // Preserve word order in each adjacency list; do not sort by destination.
23    if (!graph.run(0, false)) return nullopt;
24    vector<int> result;
25    for (int id : graph.edge_ids) result.push_back(order[id]);
26    return result;
27 }
```

## 应用题补充：UVA 10441 Catenyms

题意：多组给出 3..1000 个互不相同的小写单词，长度 1..20。每词恰用一次，后词首字母等于前词尾字母，输出字典序最小的点分隔词链；无解输出 \*\*\*。返回原输入的 0-based 编号，按编号取词输出，末尾不加点。单词排序不可换成按终点排序。重复词和空输入仅属更宽接口范围，不在本题范围内。

所需模板：DirectedEuler（第 115 页，8.1 节）、word\_chain（第 118 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://onlinejudge.org/external/104/10441.pdf>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int tests;
6     cin >> tests;
7     while (tests--)
8     {
9         int n;
10        cin >> n;
11        vector<string> words(n);
12        for (auto &word : words) cin >> word;
13        auto order = word_chain(words);
14        if (!order)
15        {
16            cout << "***\n";
17            continue;
18        }
19        for (int i = 0; i < n; i++)
20        {
21            if (i) cout << '.';
22            cout << words[(order)[i]];
23        }
24        cout << '\n';
25    }
26    return 0;
27 }
```

## 8.3 无向图欧拉路与边方向

构造 `UndirectedEuler(n)`,  $n \geq 1$ , 点号  $1..n$ ; `add(u,v)` 返回从 0 开始的逻辑边号, 自环和重边均保留。`run()` 返回是否存在欧拉路, 参数 `start` 默认 0 (自动), `lexicographic` 默认 `true`。自动选最小可行起点; 指定起点须满足奇度条件且位于有边部分。成功得到  $m+1$  个 `vertices` 和  $m$  个 `edge_ids`, 失败清空两数组。第  $i$  条边方向是 `vertices[i]` 到 `vertices[i+1]`, 与 `edges[edge_ids[i]]` 比较可判断是否反向, 自环任选方向。`used` 按逻辑边号标记, 使两条半边共同只消费一次; 自环贡献度数 2。孤立点不影响有边部分; 无边图返回指定点或 1 的单点序列。默认求顶点序列字典序最小方案; 关闭排序只保证任意方案, 可能保留上次排序。可重复 `run`, 加边清空旧方案。排序开启时  $O(n+m \log(m+1))$ , 否则  $O(n+m)$ , 空间  $O(n+m)$ , DFS 递归深度可能达到  $m+1$ 。全小图走法、原边号与方向独立验证, P2731 字典序顶点序列已线上 AC; 边编号、失败输入及其他模式仍由本地验证。

```

1 struct UndirectedEuler
2 {
3     int n;
4     vector<vector<pair<int, int>>> g;
5     vector<pair<int, int>> edges;
6     vector<int> next, used, vertices, edge_ids;
7
8     UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1) { assert(n > 0); }
9
10    int add(int u, int v)
11    {
12        assert(1 <= u && u <= n && 1 <= v && v <= n);
13        int id = edges.size();
14        edges.push_back({u, v});
15        g[u].push_back({v, id});
16        g[v].push_back({u, id});
17        vertices.clear();
18        edge_ids.clear();
19        return id;
20    }
21
22    void dfs(int u, int incoming)
23    {
24        while (next[u] < (int)g[u].size())
25        {
26            auto [v, id] = g[u][next[u]++];
27            if (used[id]) continue;
28            used[id] = 1;
29            dfs(v, id);
30        }
31        vertices.push_back(u);
32        if (incoming != -1) edge_ids.push_back(incoming);
33    }

```

```

35 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
36 bool run(int start = 0, bool lexicographic = true)
37 {
38     assert(0 <= start && start <= n);
39     vertices.clear();
40     edge_ids.clear();
41     vector<int> odd;
42     for (int u = 1; u <= n; u++)
43         if (g[u].size() % 2) odd.push_back(u);
44     if (!odd.empty() && odd.size() != 2) return false;
45     if (!odd.empty())
46     {
47         if (start && start != odd[0] && start != odd[1]) return false;
48         if (!start) start = odd[0];
49     }
50     if (!start)
51     {
52         start = 1;
53         for (int u = 1; u <= n; u++)
54             if (!g[u].empty())
55             {
56                 start = u;
57                 break;
58             }
59     }
60     if (lexicographic)
61         for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
62     fill(next.begin(), next.end(), 0);
63     used.assign(edges.size(), 0);
64     dfs(start, -1);
65     if (edge_ids.size() != edges.size())
66     {
67         vertices.clear();
68         edge_ids.clear();
69         return false;
70     }
71     reverse(vertices.begin(), vertices.end());
72     reverse(edge_ids.begin(), edge_ids.end());
73     return true;
74 }
75 };

```

## 模板题使用：Luogu P2731

题意：USACO Training：无向多重图保证有欧拉路，输出字典序最小顶点序列。 $m \leq 1024$ ，点号 1..500；直接构造 500 个点，不把孤立点 1 强设为起点。run() 自动选较小奇点，若全偶选最小非孤立点，并按邻点排序。输出 vertices，共  $m+1$  项；本题不输出 edge\_ids，边编号与方向另验。DFS 递归，最深可达  $m+1$ 。

所需模板：UndirectedEuler（第 120 页，8.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P2731>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int m;
6     cin >> m;
7     UndirectedEuler graph(500);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     if (!graph.run()) return 0;
15     for (int u : graph.vertices) cout << u << '\n';
16     return 0;
17 }
```

## 8.4 欧拉回路与指定端点定向

代码依赖：Dinic (第 127 页, 9.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`mixed_euler_orientation(n, edges, start, finish)` 返回可空的定向边序列。每条输入为  $\{u, v, \text{type}\}$ ,  $\text{type}=1$  固定  $u$  到  $v$ ,  $\text{type}=0$  可反转；点号  $1..n$ ,  $n \geq 1$ 。省略两个端点或均传 0 时请求任意起点的欧拉回路；两个正端点请求指定起终点的欧拉路径，相等时为指定起点回路。无解返回 `nullopt`，有解返回与原边顺序对应的端点对；无边图可返回有值空序列。忽略无关孤立点，但正端点须在有边部分。先任意定向可反转边，扣除目标出入度差；每条反转将两端差值分别改变  $-2$  和  $+2$ ，奇数差无解，否则用容量 1 的反转边流调整。检查底层无向连通性后，满流即得到所需方向。最大流图  $n+2$  点、至多  $m+n$  条正向边，总时间可按 Dinic 的  $O((n+2)^2(m+n))$  上界计，空间  $O(n+m)$ 。依赖 `flow.hpp`；返回边交给 `DirectedEuler` 可生成连续走法，指定端点时保持指定起点。定向不保证字典序最小。

```

1 // Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
2 // start=finish=0 requests a circuit; positive endpoints request a specified trail.
3 optional<vector<pair<int, int>>> mixed_euler_orientation(
4     int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
5 {
6     assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
7     assert(bool(start) == bool(finish));
8     vector<int> parent(n + 1), id;
9     vector<long long> degree(n + 1), balance(n + 1);
10    iota(parent.begin(), parent.end(), 0);
11    auto find = [&](auto &&self, int u) -> int
12    {
13        if (parent[u] != u) parent[u] = self(self, parent[u]);
14        return parent[u];
15    };
16    Dinic flow(n + 2);
17    vector<pair<int, int>> direction;
18    for (auto [u, v, type] : edges)
19    {
20        assert(1 <= u && u <= n && 1 <= v && v <= n);
21        assert(type == 0 || type == 1);
22        parent[find(find, u)] = find(find, v);
23        degree[u]++;
24        degree[v]++;
25        balance[u]++;
26        balance[v]--;
27        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
28        direction.push_back({u, v});
29    }

```

```

30     if (!edges.empty())
31     {
32         int root = find(find, edges[0][0]);
33         for (int u = 1; u <= n; u++)
34             if (degree[u] && find(find, u) != root) return nullopt;
35         if (start && (!degree[start] || !degree[finish])) return nullopt;
36     }
37     if (start)
38     {
39         balance[start]--;
40         balance[finish]++;
41     }
42     int source = n + 1, sink = n + 2;
43     long long need = 0;
44     for (int u = 1; u <= n; u++)
45     {
46         if (balance[u] % 2) return nullopt;
47         if (balance[u] > 0)
48         {
49             flow.add(source, u, balance[u] / 2);
50             need += balance[u] / 2;
51         }
52         else if (balance[u] < 0)
53             flow.add(u, sink, -balance[u] / 2);
54     }
55     if (flow.flow(source, sink) != need) return nullopt;
56     for (int i = 0; i < (int)edges.size(); i++)
57         if (id[i] != -1 && flow.used(id[i]))
58             swap(direction[i].first, direction[i].second);
59     return direction;
60 }

```

## 接口用法演示：混合欧拉路径：定向与走法（自定义协议）

演示约定：自定义接口演示，非 OJ 题目。首行 T；每组 mode n m start finish，再读 m 条 u v type。点号 1..n，type=1 固定方向、0 可反转。C 输入端点 0 0 求回路；A 也输入 0 0，由 mixed\_euler\_trail 选择端点；P 指定两个正端点，相等时是指定起点回路。optional 有值空数组是合法空方案。返回方向须按原边顺序加入 DirectedEuler，再 run(start,false) 恢复连续走法。成功输出 YES、VERTICES 及 EDGES 证书，原边号由内部 0-based 加一；无解输出 NO。零边 C/A 输出点 1，P s s 输出点 s，不同正端点无解。非空图忽略无关孤立点，指定端点须在有边部分。1≤T≤10000、1≤n≤200、0≤m≤800 仅为演示边界。定向采用 Dinic 一般上界，任意端点最多求两次；欧拉恢复 O(n+m)，DFS 深度可达 m+1。不保证所有定向中的全局字典序最小。

所需模板：[Dinic](#)（第 127 页，9.1.1 节）、[mixed\\_euler\\_orientation](#)（第 123 页，8.4 节）、[mixed\\_euler\\_trail](#)（第 126 页，8.5 节）、[DirectedEuler](#)（第 115 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：[https://github.com/gsh20040816/CPCTemplate/blob/main/src/compact/mixed\\_euler.hpp](https://github.com/gsh20040816/CPCTemplate/blob/main/src/compact/mixed_euler.hpp)

## 混合欧拉路径：定向与走法（自定义协议）：使用代码

```

1  int main()
2  {
3      ios::sync_with_stdio(false);
4      cin.tie(nullptr);
5      int tests;
6      cin >> tests;
7      assert(1 <= tests && tests <= 10000);
8      while (tests--)
9      {
10         char mode;
11         int n, m, start, finish;
12         cin >> mode >> n >> m >> start >> finish;
13         assert(1 <= n && n <= 200 && 0 <= m && m <= 800);
14         assert(mode == 'C' || mode == 'A' || mode == 'P');
15         assert(mode == 'P' ? 1 <= start && start <= n && 1 <= finish && finish <= n
16             : start == 0 && finish == 0);
17         vector<array<int, 3>> edges(m);
18         for (auto &[u, v, type] : edges) cin >> u >> v >> type;
19         optional<vector<pair<int, int>>> direction;
20         if (mode == 'A')
21             direction = mixed_euler_trail(n, edges);
22         else if (mode == 'C')
23             direction = mixed_euler_orientation(n, edges);
24         else
25             direction = mixed_euler_orientation(n, edges, start, finish);
26         if (!direction)
27         {
28             cout << "NO\n";
29             continue;
30         }
31         // A present empty vector is a valid zero-edge circuit, not failure.
32         DirectedEuler trail(n);
33         for (auto [u, v] : *direction) trail.add(u, v);
34         // Preserve a requested start. The orientation is not globally lexicographic.
35         if (!trail.run(start, false)) return 1;
36         cout << "YES\nVERTICES " << trail.vertices.size();
37         for (int u : trail.vertices) cout << ' ' << u;
38         cout << "\nEDGES " << trail.edge_ids.size();
39         // add() used original input order; its 0-based IDs become 1-based here.
40         for (int id : trail.edge_ids) cout << ' ' << id + 1;
41         cout << '\n';
42     }
43     return 0;
44 }

```

## 8.5 任意起终点的混合欧拉路径

代码依赖：[mixed\\_euler\\_orientation](#) (第 123 页, 8.4 节)。抄写时一并准备；标准库类型不列入算法依赖。

`mixed_euler_trail(n, edges)` 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 `DirectedEuler` 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```

1 // Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
2 // orders.
3 optional<vector<pair<int, int>>> mixed_euler_trail(int n,
4                                           const vector<array<int, 3>> &edges)
5 {
6     assert(n > 0);
7     vector<long long> degree(n + 1);
8     vector<int> odd;
9     for (auto [u, v, type] : edges)
10     {
11         assert(1 <= u && u <= n && 1 <= v && v <= n);
12         degree[u]++;
13         degree[v]++;
14     }
15     for (int u = 1; u <= n; u++)
16         if (degree[u] % 2) odd.push_back(u);
17     if (odd.empty()) return mixed_euler_orientation(n, edges);
18     if (odd.size() != 2) return nullopt;
19     auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
20     if (answer) return answer;
21     return mixed_euler_orientation(n, edges, odd[1], odd[0]);
22 }
```

联合使用示例见第 125 页（混合欧拉路径：定向与走法（自定义协议）），其中直接调用本模板。

# 第九章 网络流

## 9.1 最大流

### 9.1.1 最大流与方案

Dinic( $n$ ), 点号  $1..n$ ; add( $u,v,c$ ) 返回偶数原边编号; flow( $s,t,limit$ ) 返回本次新增流; used( $id$ ) 查询原边净流, cut( $s$ ) 返回残量可达点集。允许重边、自环, 容量非负,  $s \neq t$ ; 重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般  $O(V^2E)$ , 递归增广深度  $O(V)$ 。总流不得超过 long long 上限。get\_edge( $id$ ) 返回 tuple(起点, 终点, 总容量, 当前流), change\_edge( $id, cap, f$ ) 修改总容量与流, 要求  $id$  为 add 返回的偶数原边编号且  $0 \leq f \leq cap$ ; 正向残量变为  $cap-f$ , 反向为  $f$ 。仅保证此残量对一致, 不自动维持所有顶点的流量守恒; 继续 flow 前必须由调用方保证全局可行流及同源汇。增容且保留  $f$  合法, 缩容低于已有流时不能只截断该边。要从零重算须将所有原边一起重置为零流。两个新接口为  $O(1)$ , 尚只有本地验证。

```
1 struct Dinic
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll cap, initial;
9     };
10
11     int n;
12     vector<Edge> e;
13     vector<vector<int>> g;
14     vector<int> dep, cur;
15
16     Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1) {}
17
18     int add(int u, int v, ll c)
19     {
20         assert(c >= 0);
21         int id = (int)e.size();
22         e.push_back({u, v, c, c});
23         e.push_back({v, u, 0, 0});
24         g[u].push_back(id);
25         g[v].push_back(id + 1);
26         return id;
27     }
```

```

29  bool bfs(int s, int t)
30  {
31      fill(dep.begin(), dep.end(), -1);
32      queue<int> q;
33      dep[s] = 0;
34      q.push(s);
35      while (!q.empty())
36      {
37          int u = q.front();
38          q.pop();
39          for (int id : g[u])
40              if (e[id].cap && dep[e[id].to] < 0)
41              {
42                  dep[e[id].to] = dep[u] + 1;
43                  q.push(e[id].to);
44              }
45      }
46      return dep[t] >= 0;
47  }
48
49  ll dfs(int u, int t, ll f)
50  {
51      if (u == t) return f;
52      for (int &i = cur[u]; i < (int)g[u].size(); i++)
53      {
54          int id = g[u][i], v = e[id].to;
55          if (!e[id].cap || dep[v] != dep[u] + 1) continue;
56          ll d = dfs(v, t, min(f, e[id].cap));
57          if (d)
58          {
59              e[id].cap -= d;
60              e[id ^ 1].cap += d;
61              return d;
62          }
63      }
64      return 0;
65  }

```

```

67 // Returns additional flow; reuse only with the same source/sink.
68 ll flow(int s, int t, ll limit = LLONG_MAX)
69 {
70     assert(s != t && limit >= 0);
71     ll ans = 0;
72     while (ans < limit && bfs(s, t))
73     {
74         fill(cur.begin(), cur.end(), 0);
75         while (ans < limit)
76         {
77             ll d = dfs(s, t, limit - ans);
78             if (!d) break;
79             ans += d;
80         }
81     }
82     return ans;
83 }
84
85 ll used(int id) const { return e[id].initial - e[id].cap; }
86
87 tuple<int, int, ll, ll> get_edge(int id) const
88 {
89     assert(0 <= id && id < (int)e.size() && !(id & 1));
90     return {e[id].from, e[id].to, e[id].initial, used(id)};
91 }
92
93 // Changes one residual pair; caller must preserve global flow conservation.
94 void change_edge(int id, ll cap, ll f)
95 {
96     assert(0 <= id && id < (int)e.size() && !(id & 1));
97     assert(0 <= f && f <= cap);
98     e[id].initial = cap;
99     e[id].cap = cap - f;
100     e[id ^ 1].initial = 0;
101     e[id ^ 1].cap = f;
102 }
103
104 vector<int> cut(int s)
105 {
106     bfs(s, s);
107     vector<int> ans;
108     for (int u = 1; u <= n; u++)
109         if (dep[u] >= 0) ans.push_back(u);
110     return ans;
111 }
112 };

```

## 模板题使用：Luogu P3376

题意：给定有向容量网络与源点、汇点，求最大流。输入点号为  $1..n$ ，平行边逐条加入。

所需模板：Dinic（第 127 页，9.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3376>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, s, t;
6     cin >> n >> m >> s >> t;
7     Dinic g(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         long long c;
12         cin >> u >> v >> c;
13         g.add(u, v, c);
14     }
15     cout << g.flow(s, t) << '\n';
16 }
```

### 9.1.2 最大流中单位容量边的可行性与必选性

代码依赖：Dinic（第 127 页，9.1.1 节）、TarjanSCC（第 74 页，5.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

`unit_flow_edges(g,ids)` 在完整普通最大流的残量图上，查询每条指定边的流量是否在至少一个/每一个整数最大流中等于 1，返回 `{possible,forced}`。ids 必须是 `Dinic.add` 返回的原始偶数边号，且每条被查询边初始容量恰为 1；其他边允许任意非负 `long long` 容量，最大流值须可表示。结果保持查询顺序，支持重复编号。已使用或两端残量 SCC 相同则可行，已使用且不同 SCC 则必选。这里比较的是完整流量方案，允许不改变源汇净流量的环流；单位自环可以取 0 或 1，不应解释成必经的简单  $s$ - $t$  路径。不是最小割边分类，也不把容量大于 1 的边强行按布尔流处理。 $O(n+m+ \text{查询数})$  时间与空间，Tarjan 保留递归；分析不修改 `g`。

```

1 vector<array<bool, 2>> unit_flow_edges(const Dinic &g, const vector<int> &ids)
2 {
3     TarjanSCC d(g.n);
4     for (auto e : g.e)
5         if (e.cap) d.add(e.from, e.to);
6     d.run();
7     vector<array<bool, 2>> ans;
8     for (int id : ids)
9     {
10         assert(0 <= id && id < (int)g.e.size() && id % 2 == 0);
11         auto e = g.e[id];
12         assert(e.initial == 1);
13         bool used = g.used(id) == 1, same = d.bel[e.from] == d.bel[e.to];
14         ans.push_back({used || same, used && !same});
15     }
16     return ans;
17 }
```

## 应用题补充：最大流单位边 Y/N/? (本地接口示例)

题意：输入  $n$ 、 $m$ 、 $s$ 、 $t$ 、 $q$ ，随后  $m$  条有向容量边和  $q$  个原边序号；被询问边容量须为 1。先输出最大流值，再输出 Y（所有方案取 1）、N（没有方案取 1）、?（两种都可能）。用 `add` 返回值保存残量边号。本地示例对应附件 2.14 的分类内核，不代替原比赛题的排列约束建图。

所需模板：Dinic（第 127 页，9.1.1 节）、TarjanSCC（第 74 页，5.1.1 节）、`unit_flow_edges`（第 130 页，9.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://github.com/gsh20040816/CPCTemplate/issues/11>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, s, t, q;
6     cin >> n >> m >> s >> t >> q;
7     Dinic g(n);
8     vector<int> edge(m), ids(q);
9     for (int &id : edge)
10     {
11         int u, v;
12         long long cap;
13         cin >> u >> v >> cap;
14         id = g.add(u, v, cap);
15     }
16     for (int &id : ids)
17     {
18         int i;
19         cin >> i;
20         id = edge[i - 1];
21     }
22     cout << g.flow(s, t) << '\n';
23     for (auto [possible, forced] : unit_flow_edges(g, ids))
24         cout << (forced ? 'Y' : possible ? '?' : 'N');
25     cout << '\n';
26 }
```

## 应用题补充：QOJ 10424 / NERC 2024 K

题意：两套排列各给出同一  $k$  元素集合的部分子序列， $-1$  表示未知；输出每个元素必选 Y、必不选 N 或可选？，无解输出 Inconsistent。已知元素切分两侧容量组，每个元素对应一条单位边；最大流必须等于  $k$ ，再作残量 SCC 分类。 $n$  及所有测试的  $n$  之和至 2000。比赛应用，不计作独立模板题覆盖。

所需模板：Dinic（第 127 页，9.1.1 节）、TarjanSCC（第 74 页，5.1.1 节）、unit\_flow\_edges（第 130 页，9.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://qoj.ac/problem/10424>

```

1 bool blocks(const vector<int> &p,
2             const vector<int> &s,
3             vector<int> &id,
4             vector<int> &cap)
5 {
6     int n = p.size(), k = s.size(), last = -1, prev = -1;
7     vector<int> pos(n + 1);
8     for (int i = 0; i < n; i++) pos[p[i]] = i;
9     id.assign(n + 1, -1);
10    cap.clear();
11    for (int i = 0; i <= k; i++)
12    {
13        if (i < k && s[i] == -1) continue;
14        int at = i == k ? n : pos[s[i]];
15        if (at <= last || at - last - 1 < i - prev - 1) return false;
16        int g = cap.size();
17        cap.push_back(i - prev - 1);
18        for (int j = last + 1; j < at; j++) id[p[j]] = g;
19        if (i < k)
20        {
21            id[s[i]] = cap.size();
22            cap.push_back(1);
23        }
24        last = at;
25        prev = i;
26    }
27    return true;
28 }
```

使用入口（接上页配置函数）：

```

30 int main()
31 {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34     int tests;
35     cin >> tests;
36     while (tests--)
37     {
38         int n, k;
39         cin >> n;
40         vector<int> p(n), q(n);
41         for (int &x : p) cin >> x;
42         for (int &x : q) cin >> x;
43         cin >> k;
44         vector<int> a(k), b(k), l, r, x, y;
45         for (int &v : a) cin >> v;
46         for (int &v : b) cin >> v;
47         if (!blocks(p, a, l, x) || !blocks(q, b, r, y))
48         {
49             cout << "Inconsistent\n";
50             continue;
51         }
52         int m = x.size(), source = m + y.size() + 1, sink = source + 1;
53         Dinic g(sink);
54         for (int i = 0; i < m; i++) g.add(source, i + 1, x[i]);
55         for (int i = 0; i < (int)y.size(); i++) g.add(m + i + 1, sink, y[i]);
56         vector<int> ids;
57         for (int v = 1; v <= n; v++) ids.push_back(g.add(l[v] + 1, m + r[v] + 1, 1));
58         if (g.flow(source, sink) != k)
59         {
60             cout << "Inconsistent\n";
61             continue;
62         }
63         for (auto [possible, forced] : unit_flow_edges(g, ids))
64             cout << (forced ? 'Y' : possible ? '?' : 'N');
65         cout << '\n';
66     }
67 }

```

## 9.2 最小割

### 9.2.1 正容量最小割的可行边与必选边

代码依赖：Dinic (第 127 页, 9.1.1 节)、TarjanSCC (第 74 页, 5.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`mincut_edges(g,s,t)` 只读分析 Dinic 已经完整求出的普通  $s \rightarrow t$  最大流；不得传入限流提前停止或人为修改过的残量网络。所有原边初始容量必须严格为正，最大流值须可由 `long long` 表示。返回逐条原边的 `{possible,forced}`，对应 Dinic 原始偶数边号 `id` 的结果是 `ans[id/2]`。只有饱和边可能属于最小割：两端残量 SCC 不同则可行；起点与 `s` 同 SCC、终点与 `t` 同 SCC 则必选。支持平行边和自环，源汇不同，点号 `1..g.n`。O(n+m) 时间与空间，Tarjan 递归栈 O(n)。零容量边没有正流对应的反向残量边，不能套用本函数的同 SCC 简化条件。每次修改图或重新增广后都需重新分析。

```

1 vector<array<bool, 2>> mincut_edges(const Dinic &g, int s, int t)
2 {
3     TarjanSCC d(g.n);
4     for (auto e : g.e)
5         if (e.cap) d.add(e.from, e.to);
6     d.run();
7     vector<array<bool, 2>> ans(g.e.size() / 2);
8     for (int id = 0; id < (int)g.e.size(); id += 2)
9     {
10         auto e = g.e[id];
11         assert(e.initial > 0);
12         if (e.cap) continue;
13         ans[id / 2][0] = d.bel[e.from] != d.bel[e.to];
14         ans[id / 2][1] = d.bel[e.from] == d.bel[s] && d.bel[e.to] == d.bel[t];
15     }
16     return ans;
17 }
```

## 应用题补充：Luogu P4126 / AHOI2009 最小割

题意：求每条正容量有向边是否出现在至少一个/每一个最小割中。先不设额外限制地跑完最大流，再按原边输入顺序输出两项布尔值。容量用 long long；平行道路分别加边。此题源自 AHOI，作为应用补充单列。

所需模板：Dinic (第 127 页, 9.1.1 节)、TarjanSCC (第 74 页, 5.1.1 节)、mincut\_edges (第 134 页, 9.2.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4126>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, s, t;
6     cin >> n >> m >> s >> t;
7     Dinic g(n);
8     while (m--)
9     {
10         int u, v;
11         long long c;
12         cin >> u >> v >> c;
13         g.add(u, v, c);
14     }
15     g.flow(s, t);
16     for (auto [possible, forced] : mincut_edges(g, s, t))
17         cout << possible << ' ' << forced << '\n';
18 }
```

## 9.2.2 最大权闭合子图与选点方案

代码依赖：Dinic (第 127 页, 9.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`maximum_closure(weight, dependencies)` 返回 (最大权值, 选中点列表), `weight` 为 0-based `vector<long long>`, 图中点号 `1..n`。依赖边 `(u,v)` 表示选 `u` 必须选 `v`, 允许环、自环与重边; 允许空选集, 因此答案不小于 0。依赖 `flow.hpp` 中的 Dinic。正权连源、负权连汇, 依赖边容量取正权总和加 1; 要求所有正权之和严格小于 `LLONG_MAX`。负权先提升到 `int128` 再取负, 并截断到依赖边容量, 因此 `LLONG_MIN` 也可处理。返回残量图源点可达的原顶点, 权值为正权总和减最小割; 超大负权边不会出现在最优割中, 截断不改变原问题最优值。每次调用重建网络。复杂度沿用 Dinic: 一般  $O(V^2E)$ , 空间  $O(V+E)$ , 增广保持递归。返回一种最优闭合集, 不承诺最大或最小选点个数。

```

1 pair<long long, vector<int>> maximum_closure(const vector<long long> &weight,
2                                              const vector<pair<int, int>> &dependencies)
3 {
4     using I = __int128_t;
5     int n = weight.size(), s = n + 1, t = n + 2;
6     I positive = 0;
7     for (long long w : weight)
8         if (w > 0) positive += w;
9     assert(positive < LLONG_MAX);
10    long long inf = (long long)positive + 1;
11    Dinic graph(n + 2);
12    for (int u = 1; u <= n; u++)
13    {
14        long long w = weight[u - 1];
15        if (w > 0)
16            graph.add(s, u, w);
17        else if (w < 0)
18            graph.add(u, t, (long long)min(-I(w), I(inf)));
19    }
20    for (auto [u, v] : dependencies)
21    {
22        assert(1 <= u && u <= n && 1 <= v && v <= n);
23        graph.add(u, v, inf);
24    }
25    long long answer = (long long)positive - graph.flow(s, t);
26    vector<int> selected;
27    for (int u : graph.cut(s))
28        if (u <= n) selected.push_back(u);
29    return {answer, selected};
30 }

```

## 应用题补充：Luogu P2762 太空飞行计划问题

题意：选实验获得收益，配置仪器支付费用，输出最优实验/仪器编号和净收益。 $m, n$  至 50，单项费用为正且小于  $2^{31}$ ，累计用 long long。逐行读取变长依赖，istringstream 兼容 CRLF；实验  $i$  依赖仪器  $m+j$ 。应用用法，不计非比赛模板覆盖。

所需模板：Dinic（第 127 页，9.1.1 节）、maximum\_closure（第 136 页，9.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P2762>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int m, n;
6     cin >> m >> n;
7     vector<long long> w(m + n);
8     vector<pair<int, int>> dep;
9     string line;
10    getline(cin, line);
11    for (int i = 1; i <= m; i++)
12    {
13        getline(cin, line);
14        istringstream in(line);
15        in >> w[i - 1];
16        int j;
17        while (in >> j) dep.push_back({i, m + j});
18    }
19    for (int j = 0; j < n; j++)
20    {
21        long long cost;
22        cin >> cost;
23        w[m + j] = -cost;
24    }
25    auto [best, selected] = maximum_closure(w, dep);
26    for (int u : selected)
27        if (u <= m) cout << u << ' ';
28    cout << '\n';
29    for (int u : selected)
30        if (u > m) cout << u - m << ' ';
31    cout << '\n' << best << '\n';
32 }

```

### 9.2.3 Gomory–Hu 最小割树

代码依赖：Dinic (第 127 页, 9.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

gomory\_hu(n,edges) 返回 n-1 条 (u,v, 容量) 树边； $n \geq 1$ ，原图点号 0..n-1，输入为无向边 tuple 列表，重边保留、自环忽略、容量非负。非自环边总容量须严格小于 `LLONG_MAX`。两点最小割是树路径最小边权；删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边， $n=1$  返回空树。运行 n-1 次 Dinic，一般上界  $O(n^3m+n^2)$ ，额外空间  $O(n+m)$ ，DFS 增广保持递归。P4897 输入 n 表示 n+1 个点，驱动传 n+1。双版本 P4897 已 AC，最长单点分别 252、185 ms。

```

1 vector<tuple<int, int, long long>>
2 gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
3 {
4     assert(n >= 1);
5     vector<int> parent(n);
6     vector<long long> value(n);
7     for (int s = 1; s < n; s++)
8     {
9         int t = parent[s];
10        Dinic flow(n);
11        for (auto [u, v, w] : edges)
12            if (u != v)
13            {
14                flow.add(u + 1, v + 1, w);
15                flow.add(v + 1, u + 1, w);
16            }
17        long long cut = flow.flow(s + 1, t + 1);
18        vector<bool> side(n);
19        for (int u : flow.cut(s + 1)) side[u - 1] = true;
20        for (int v = 1; v < n; v++)
21            if (v != s && parent[v] == t && side[v]) parent[v] = s;
22        if (side[parent[t]])
23        {
24            parent[s] = parent[t];
25            parent[t] = s;
26            value[s] = value[t];
27            value[t] = cut;
28        }
29        else
30            value[s] = cut;
31    }
32    vector<tuple<int, int, long long>> tree;
33    for (int v = 1; v < n; v++) tree.push_back({v, parent[v], value[v]});
34    return tree;
35 }
```

## 模板题使用：Luogu P4897

题意：给无向图，多次询问不同两点的最小割。点号为  $0..n$ ，因此建树传  $n+1$ 。再预处理所有点对的答案，查询矩阵额外占  $O(n^2)$  时间和空间。

所需模板：Dinic（第 127 页，9.1.1 节）、gomory\_hu（第 138 页，9.2.3 节）、cut\_tree\_values（第 140 页，9.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4897>

```
1 int main()
2 {
3     int n, m;
4     scanf("%d%d", &n, &m);
5     vector<tuple<int, int, long long>> edges;
6     for (int i = 0; i < m; i++)
7     {
8         int u, v;
9         long long w;
10        scanf("%d%d%lld", &u, &v, &w);
11        edges.push_back({u, v, w});
12    }
13    auto tree = gomory_hu(n + 1, edges);
14    auto answer = cut_tree_values(n + 1, tree);
15    int q;
16    scanf("%d", &q);
17    while (q--)
18    {
19        int u, v;
20        scanf("%d%d", &u, &v);
21        printf("%lld\n", answer[u][v]);
22    }
23    return 0;
24 }
```

### 9.2.4 最小割树全点对查询

`cut_tree_values(n, tree)` 对合法的连通无环加权树计算全点对路径最小边权；点号  $0..n-1$ 、 $n \geq 1$ 、恰有  $n-1$  条边，边权为非负 `long long`。输入可直接使用 `gomory_hu` 的返回值。结果 `ans[u][v]` 在  $u \neq v$  时为两点最小割；对角线为 `LLONG_MAX` 哨兵，不能解释成同点最小割。 $n=1$  返回单个哨兵。 $O(n^2)$  时间与空间，递归 DFS 深度  $O(n)$ ，查询  $O(1)$ 。只需要标准库，不依赖流模板；不是任意图的查询函数。

```

1 vector<vector<long long>>
2 cut_tree_values(int n, const vector<tuple<int, int, long long>> &tree)
3 {
4     assert(n >= 1 && tree.size() == (size_t)n - 1);
5     vector<vector<pair<int, long long>>> g(n);
6     for (auto [u, v, w] : tree)
7     {
8         assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
9         g[u].push_back({v, w});
10        g[v].push_back({u, w});
11    }
12    vector<vector<long long>> answer(n, vector<long long>(n, LLONG_MAX));
13    for (int s = 0; s < n; s++)
14    {
15        auto dfs = [&](auto &&self, int u, int parent, long long value) -> void
16        {
17            answer[s][u] = value;
18            for (auto [v, w] : g[u])
19                if (v != parent) self(self, v, u, min(value, w));
20        };
21        dfs(dfs, s, -1, LLONG_MAX);
22    }
23    return answer;
24 }
```

联合使用示例见第 139 页 (Luogu P4897)，其中直接调用本模板。

## 9.3 费用流

### 9.3.1 势能最小费用最大流

`MinCostFlow(n)`, 点号  $1..n$ ; `add(u,v,cap,cost)` 返回边编号, `flow(s,t,limit)` 返回新增流量及 128 位费用, `used(id)` 返回原边流量。允许负费用, 初始残量网络必须没有负环; 检测到负环抛异常。Bellman-Ford 初始化  $O(VE)$ , 每次增广  $O(E \log V)$ ; 增广次数不能简单假设为  $O(V)$ 。另要求  $\sum(\text{cap} \times |\text{cost}|) < 2^{120}$ , 防止总费用超过 `int128`。`slope(s,t,limit)` 返回从  $(0,0)$  开始的本次新增流量/费用折点, 相邻同斜率段合并, 费用为 `int128`。无增广时只返回起点; 每个整数流量的费用可在线性段内插值。重复调用仍用同源汇, 返回值不是历史累计值。`flow` 不存储曲线; `slope` 额外空间为折点数。内部 `run` 不是用户接口。新曲线接口仅本地验证, P3381 驱动只输出终点, 不能据此验证全部中间折点。

```

1 struct MinCostFlow
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5
6     struct Edge
7     {
8         int from, to;
9         ll cap, cost, initial;
10    };
11
12    int n;
13    vector<Edge> e;
14    vector<vector<int>> g;
15
16    MinCostFlow(int n) : n(n), g(n + 1) {}
17
18    int add(int u, int v, ll cap, ll cost)
19    {
20        assert(cap >= 0 && cost != LLONG_MIN);
21        int id = (int)e.size();
22        e.push_back({u, v, cap, cost, cap});
23        e.push_back({v, u, 0, -cost, 0});
24        g[u].push_back(id);
25        g[v].push_back(id + 1);
26        return id;
27    }
28
29    ll used(int id) const { return e[id].initial - e[id].cap; }

```

```

31 // Negative costs allowed; initial residual graph must have no negative cycle.
32 pair<ll, i128> run(int s, int t, ll limit, vector<pair<ll, i128>> *curve)
33 {
34     assert(s != t && limit >= 0);
35     const i128 inf = i128(1) << 120;
36     vector<i128> h(n + 1, 0), d(n + 1);
37     vector<int> pre(n + 1);
38     // Super-source Bellman-Ford: feasible potentials for every vertex.
39     for (int it = 0; it < n; it++)
40     {
41         bool changed = false;
42         for (auto &a : e)
43             if (a.cap && h[a.to] > h[a.from] + a.cost)
44             {
45                 h[a.to] = h[a.from] + a.cost;
46                 changed = true;
47             }
48         if (!changed) break;
49         if (it == n - 1) throw invalid_argument("negative cycle");
50     }
51     ll f = 0;
52     i128 cost = 0;
53     optional<i128> last;
54     while (f < limit)
55     {
56         fill(d.begin(), d.end(), inf);
57         d[s] = 0;
58         priority_queue<pair<i128, int>,
59             vector<pair<i128, int>>,
60             greater<pair<i128, int>>>
61             q;
62         q.push({0, s});
63         while (!q.empty())
64         {
65             auto [du, u] = q.top();
66             q.pop();
67             if (du != d[u]) continue;
68             for (int id : g[u])
69             {
70                 auto &a = e[id];
71                 i128 nd = du + a.cost + h[u] - h[a.to];
72                 if (a.cap && nd < d[a.to])
73                 {
74                     d[a.to] = nd;
75                     pre[a.to] = id;
76                     q.push({nd, a.to});
77                 }
78             }
79         }

```

```

80         if (d[t] == inf) break;
81         for (int u = 1; u <= n; u++)
82             if (d[u] != inf) h[u] += d[u];
83         ll take = limit - f;
84         for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
85         for (int u = t; u != s; u = e[pre[u]].from)
86         {
87             int id = pre[u];
88             e[id].cap -= take;
89             e[id ^ 1].cap += take;
90             cost += i128(take) * e[id].cost;
91         }
92         f += take;
93         if (curve)
94         {
95             i128 unit = h[t] - h[s];
96             if (last && *last == unit) curve->pop_back();
97             curve->push_back({f, cost});
98             last = unit;
99         }
100     }
101     return {f, cost};
102 }
103
104 pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
105 {
106     return run(s, t, limit, nullptr);
107 }
108
109 vector<pair<ll, i128>> slope(int s, int t, ll limit = LLONG_MAX)
110 {
111     vector<pair<ll, i128>> curve{{0, 0}};
112     run(s, t, limit, &curve);
113     return curve;
114 }
115 };

```

## 模板题使用：Luogu P3381

题意：给有向容量费用网络，求最大流及对应最小费用。点号  $1..n$ ，add 参数依次为起点、终点、容量、单位费用。题目保证费用答案不超过  $2^{31}-1$ ，因此这里可将返回的 `int128` 转为 `long long`；一般题目不能照搬转换。

所需模板：MinCostFlow（第 141 页，9.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3381>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, s, t;
6     cin >> n >> m >> s >> t;
7     MinCostFlow graph(n);
8     while (m--)
9     {
10         int u, v;
11         long long capacity, cost;
12         cin >> u >> v >> capacity >> cost;
13         graph.add(u, v, capacity, cost);
14     }
15     auto [flow, cost] = graph.flow(s, t);
16     cout << flow << ' ' << (long long)cost << '\n';
17 }
```

### 9.3.2 SPFA 最小费用最大流

SpfaFlow(n), 点号 1..n; add(u,v,cap,cost) 返回偶数原边编号, used(id) 查询原边流量。flow(s,t,limit) 返回本次新增流量及 int128 费用; 默认 limit=LLONG\_MAX, s≠t。允许重边、自环、零容量和负费用, 调用前残量网络不得有负环; 先用全点入队 SPFA 检查全图, 含不可达负环也抛 invalid\_argument, 检查阶段不修改流量。重复调用须保持同源汇, 返回值不累计历史。容量非负、费用不能是 LLONG\_MIN, 总流  $\leq$  LLONG\_MAX,  $\sum(\text{cap} \times |\text{cost}|) < 2^{120}$ 。每次增广用 SPFA 最短路, 复杂度  $O((A+1)VE)$ , A 为实际增广次数, 不能假设等于点数; 空间  $O(V+E)$ 。通常优先势能 Dijkstra, 本版作为更直接的 SPFA 备选。e/g 可读, 不能任意修改残量; shortest 为内部方法, 0 仅是全点初始化标记, 不是合法顶点。

```

1 struct SpfaFlow
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Edge
7     {
8         int from, to;
9         ll cap, cost, initial;
10    };
11
12    int n;
13    vector<Edge> e;
14    vector<vector<int>> g;
15
16    SpfaFlow(int n) : n(n), g(n + 1) {}
17
18    int add(int u, int v, ll cap, ll cost)
19    {
20        assert(cap >= 0 && cost != LLONG_MIN);
21        int id = (int)e.size();
22        e.push_back({u, v, cap, cost, cap});
23        e.push_back({v, u, 0, -cost, 0});
24        g[u].push_back(id);
25        g[v].push_back(id + 1);
26        return id;
27    }
28
29    ll used(int id) const { return e[id].initial - e[id].cap; }

```

接上页同一结构体:

```

31 // Internal: s=0 seeds every vertex to detect any residual negative cycle.
32 void shortest(int s, vector<I> &d, vector<int> &pre)
33 {
34     const I inf = I(1) << 120;
35     d.assign(n + 1, inf);
36     pre.assign(n + 1, -1);
37     vector<int> len(n + 1), in(n + 1);
38     queue<int> q;
39     for (int u = 1; u <= n; u++)
40         if (!s || u == s)
41         {
42             d[u] = 0;
43             in[u] = 1;
44             q.push(u);
45         }
46     while (!q.empty())
47     {
48         int u = q.front();
49         q.pop();
50         in[u] = 0;
51         for (int id : g[u])
52         {
53             auto &a = e[id];
54             if (!a.cap || d[a.to] <= d[u] + a.cost) continue;
55             d[a.to] = d[u] + a.cost;
56             pre[a.to] = id;
57             len[a.to] = len[u] + 1;
58             if (len[a.to] >= n) throw invalid_argument("negative cycle");
59             if (!in[a.to])
60             {
61                 in[a.to] = 1;
62                 q.push(a.to);
63             }
64         }
65     }
66 }

```

接上页同一结构体：

```

68 // Returns additional flow/cost; reuse only with the same source/sink.
69 pair<ll, I> flow(int s, int t, ll limit = LLONG_MAX)
70 {
71     assert(s != t && limit >= 0);
72     vector<I> d;
73     vector<int> pre;
74     shortest(0, d, pre);
75     ll f = 0;
76     I cost = 0;
77     while (f < limit)
78     {
79         shortest(s, d, pre);
80         if (pre[t] < 0) break;
81         ll take = limit - f;
82         for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
83         for (int u = t; u != s; u = e[pre[u]].from)
84         {
85             int id = pre[u];
86             e[id].cap -= take;
87             e[id ^ 1].cap += take;
88         }
89         f += take;
90         cost += I(take) * d[t];
91     }
92     return {f, cost};
93 }
94 };

```

## 模板题使用：Luogu P3381

题意：求最小费用最大流。点号  $1..n$ ，输入容量与费用均为  $0..1000$ ，最多  $5000$  点、 $50000$  边；本题保证最大流及最小费用  $\leq 2^{31}-1$ ，可把返回费用转 `long long` 输出。这里实际使用 SPFA 增广，通用接口允许负边但禁止负环；有负环的费用流须用另一套算法。

所需模板：[SpfaFlow](#) (第 145 页, 9.3.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3381>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, s, t;
6     cin >> n >> m >> s >> t;
7     SpfaFlow g(n);
8     while (m--)
9     {
10         int u, v;
11         long long cap, cost;
12         cin >> u >> v >> cap >> cost;
13         g.add(u, v, cap, cost);
14     }
15     auto [flow, cost] = g.flow(s, t);
16     cout << flow << ' ' << (long long)cost << '\n';
17 }
```

### 9.3.3 允许负环的最小费用最大流

代码依赖：MinCostFlow（第 141 页，9.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

NegativeCostFlow(n)，点号 1..n；add(u,v,cap,cost) 返回从 0 开始的原边编号。容量非负，所有原边容量总和  $\leq \text{LLONG\_MAX}$ ；费用为 long long 且不等于  $\text{LLONG\_MIN}$ 。允许重边、自环及任意位置的负环。solve(s, t, limit) 只调用一次，limit 默认  $\text{LLONG\_MAX}$ ， $s \neq t$ 、 $\text{limit} \geq 0$ ，返回（不超过 limit 的最大流量，该流量下的最小费用），费用为 int128。limit=0 可求最小费用循环流；不足 limit 时返回实际最大流。成功后 used(id) 返回原边流量，自动还原负边退流。先满流负费用边并建立反向正费用退流边，再用超级源汇修复平衡；初始残量费用非负，修复后无负环。删除辅助边后继续最短增广，包含不经过源汇的负环收益。依赖 MinCostFlow，使用两次势能预处理；若总增广次数为 K，时间  $O(VE + K(E \log(E+1) + V))$ ，空间  $O(V+E)$ ，整数容量下为伪多项式，不是费用缩放算法。节点和残量边数须可由 int 表示。

```

1 struct NegativeCostFlow
2 {
3     using ll = long long;
4     using I = __int128_t;
5     int n;
6     MinCostFlow g;
7     vector<ll> balance, cap;
8     vector<int> ids;
9     vector<bool> reversed;
10    ll total = 0;
11    I cost = 0;
12    bool solved = false;
13
14    NegativeCostFlow(int n) : n(n), g(n + 2), balance(n + 1) {}
15
16    int add(int u, int v, ll c, ll w)
17    {
18        assert(!solved && 1 <= u && u <= n && 1 <= v && v <= n);
19        assert(0 <= c && c <= LLONG_MAX - total && w != LLONG_MIN);
20        total += c;
21        int id = ids.size();
22        cap.push_back(c);
23        reversed.push_back(w < 0);
24        if (w < 0)
25        {
26            balance[u] -= c;
27            balance[v] += c;
28            cost += I(c) * w;
29            ids.push_back(g.add(v, u, c, -w));
30        }
31        else
32            ids.push_back(g.add(u, v, c, w));
33        return id;
34    }

```

需求修复、增产与原边流量还原（接上页同一结构体）：

```

36 pair<ll, I> solve(int s, int t, ll limit = LLONG_MAX)
37 {
38     assert(!solved && 1 <= s && s <= n && 1 <= t && t <= n && s != t);
39     assert(limit >= 0);
40     solved = true;
41     int first = g.e.size(), back = g.add(t, s, min(total, limit), 0);
42     ll need = 0;
43     for (int u = 1; u <= n; u++)
44         if (balance[u] > 0)
45         {
46             g.add(n + 1, u, balance[u], 0);
47             need += balance[u];
48         }
49         else if (balance[u] < 0)
50             g.add(u, n + 2, -balance[u], 0);
51     auto [sent, correction] = g.flow(n + 1, n + 2, need);
52     assert(sent == need);
53     ll base = g.used(back);
54     cost += correction;
55     for (int id = first; id < (int)g.e.size(); id++) g.e[id].cap = 0;
56     auto [more, extra] = g.flow(s, t, limit - base);
57     cost += extra;
58     return {base + more, cost};
59 }
60
61 ll used(int id) const
62 {
63     assert(solved && 0 <= id && id < (int)ids.size());
64     ll f = g.used(ids[id]);
65     return reversed[id] ? cap[id] - f : f;
66 }
67 };

```

## 模板题使用：Luogu P7173

题意：求允许负费用环的最小费用最大流，独立于源汇的负环也计入费用。无额外流量上限时调用 `solve(s,t)`。题目容量和费用界保证最终费用可转为 `long long`；一般模板返回 `int128`，不能任意照搬此转换。

所需模板：[MinCostFlow](#)（第 141 页，9.3.1 节）、[NegativeCostFlow](#)（第 149 页，9.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P7173>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, s, t;
6     cin >> n >> m >> s >> t;
7     NegativeCostFlow g(n);
8     while (m--)
9     {
10         int u, v;
11         long long cap, cost;
12         cin >> u >> v >> cap >> cost;
13         g.add(u, v, cap, cost);
14     }
15     auto [flow, cost] = g.solve(s, t);
16     cout << flow << ' ' << (long long)cost << '\n';
17 }
```

## 9.4 上下界网络流

### 9.4.1 上下界可行循环流

代码依赖：Dinic (第 127 页, 9.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

BoundedCirculation(n), 点号 1..n. add(u,v,lo,hi) 返回原边编号,  $0 \leq lo \leq hi \leq \text{LLONG\_MAX}$ , 所有下界总和  $\leq \text{LLONG\_MAX}$ . solve() 仅调用一次, 返回可行性; 成功后 used(id) 读取边流量。每点要求流量守恒, 属于循环流; 带源汇最大流用 BoundedMaxFlow。hi-lo 是残量容量, 下界引起 balance; 各边上界不必求和, 只要求下界总和保证 balance 和超级源总需求不溢出。依赖 Dinic。

```

1 struct BoundedCirculation
2 {
3     using ll = long long;
4     int n;
5     Dinic g;
6     vector<ll> balance, lower;
7     vector<int> ids;
8     bool solved = false, feasible = false;
9
10    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1) {}
11
12    int add(int u, int v, ll lo, ll hi)
13    {
14        assert(!solved && 0 <= lo && lo <= hi);
15        int id = ids.size();
16        ids.push_back(g.add(u, v, hi - lo));
17        lower.push_back(lo);
18        balance[u] -= lo;
19        balance[v] += lo;
20        return id;
21    }
22
23    bool solve()
24    {
25        assert(!solved);
26        solved = true;
27        ll need = 0;
28        for (int u = 1; u <= n; u++)
29            if (balance[u] > 0)
30            {
31                g.add(n + 1, u, balance[u]);
32                need += balance[u];
33            }
34            else if (balance[u] < 0)
35                g.add(u, n + 2, -balance[u]);
36        return feasible = (g.flow(n + 1, n + 2) == need);
37    }
38
39    ll used(int id) const
40    {
41        assert(feasible);
42        return lower[id] + g.used(ids[id]);
43    }
44 };

```

## 模板题使用：LibreOJ 115

题意：无源汇有上下界可行流：每个点流入等于流出，按输入顺序输出每条边的合法流量，无解输出 NO。点号 1..n, n 不超过 200、m 不超过 10200,  $0 \leq lo \leq hi < 3000$ 。add 返回 0 起原始边编号，used(i) 不是 Dinic 残量边号；solve 只能调用一次，且只有可行时才读取 used。自环、重边逐条保留，不添加源汇回边。

所需模板：Dinic (第 127 页, 9.1.1 节)、BoundedCirculation (第 152 页, 9.4.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://loj.ac/p/115>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     BoundedCirculation graph(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         long long lo, hi;
12         cin >> u >> v >> lo >> hi;
13         graph.add(u, v, lo, hi);
14     }
15     if (!graph.solve())
16     {
17         cout << "NO\n";
18         return 0;
19     }
20     cout << "YES\n";
21     for (int i = 0; i < m; i++) cout << graph.used(i) << '\n';
22 }
```

### 9.4.2 有源汇上下界最值流与边流量

代码依赖：BoundedCirculation (第 152 页, 9.4.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

BoundedMaxFlow(n), 点号 1..n; add(u,v,lo,hi) 返回从 0 开始的原边编号,  $0 \leq lo \leq hi$ , 所有原边上界总和  $\leq \text{LLONG\_MAX}$ 。solve(s,t) 求最大净流, minimum(s,t) 求最小净流; 同一对象只能任选其一调用一次,  $s \neq t$ 。minimum 交换源汇求最大值后取反, 原边方向及 used 不变; 返回 optional<long long>, nullopt 表示无可行流。目标为源点流出减流入的净流量, 两种查询都允许负答案, 不能把 -1 直接当无解; minimum 不额外强制净流量非负。只在成功后读取 used(id) 的原边方案。先加  $t \rightarrow s$  与  $s \rightarrow t$  两条辅助边兼容负净流, 再求可行循环流; 保存净值后清空全部辅助残量边, 最后在原图从 s 到 t 增广。最优值与增加量均受原边上界总和和总流量控制。允许自环、重边; 构建和清理  $O(n+m)$ , 主体为两次 Dinic, 额外空间  $O(n+m)$ 。依赖 BoundedCirculation 与 Dinic, 二者内部字段只读, 不应在求解后再加边或改流。

```

1 struct BoundedMaxFlow
2 {
3     using ll = long long;
4     BoundedCirculation b;
5     ll total = 0;
6     int m = 0;
7
8     BoundedMaxFlow(int n) : b(n) {}
9
10    int add(int u, int v, ll lo, ll hi)
11    {
12        assert(0 <= lo && lo <= hi && hi <= LLONG_MAX - total);
13        total += hi;
14        b.add(u, v, lo, hi);
15        return m++;
16    }
17
18    optional<ll> solve(int s, int t)
19    {
20        assert(!b.solved && 1 <= s && s <= b.n && 1 <= t && t <= b.n && s != t);
21        int first = b.g.e.size();
22        int back = b.add(t, s, 0, total);
23        int forward = b.add(s, t, 0, total);
24        if (!b.solve()) return nullopt;
25        __int128 ans = (__int128)b.used(back) - b.used(forward);
26        for (int id = first; id < (int)b.g.e.size(); id += 2) b.g.change_edge(id, 0, 0);
27        ans += b.g.flow(s, t);
28        assert(-(__int128)total <= ans && ans <= total);
29        return (ll)ans;
30    }

```

最小净流与原边流量（接上页同一结构体）：

```
32 optional<ll> minimum(int s, int t)
33 {
34     auto ans = solve(t, s);
35     if (ans) *ans = -*ans;
36     return ans;
37 }
38
39 ll used(int id) const
40 {
41     assert(0 <= id && id < m);
42     return b.used(id);
43 }
44 };
```

## 应用题补充：Luogu P5192 Shoot the Bullet

题意：每天至多拍  $D$  张，每次独立拍摄有  $[L,R]$  限制，每位目标至少拍  $G$  张，求最多拍摄数。源连天、天连目标、目标连汇；重复目标记录分别加边。目标原编号加 1 后再偏移  $n$ 。题目总拍摄数至多  $365 \times 30000$ ，用作目标到汇的容量上界。读至 EOF，每组输出后空一行。本题净流非负，因此无解可输出 -1；一般接口允许负净流。作为附件建图应用单列。

所需模板：Dinic（第 127 页，9.1.1 节）、BoundedCirculation（第 152 页，9.4.1 节）、BoundedMaxFlow（第 154 页，9.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P5192>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     while (cin >> n >> m)
7     {
8         int s = n + m + 1, t = s + 1;
9         BoundedMaxFlow f(t);
10        for (int j = 1; j <= m; j++)
11        {
12            long long g;
13            cin >> g;
14            f.add(n + j, t, g, 365LL * 30000);
15        }
16        for (int i = 1; i <= n; i++)
17        {
18            int c, d;
19            cin >> c >> d;
20            f.add(s, i, 0, d);
21            while (c--)
22            {
23                int j, lo, hi;
24                cin >> j >> lo >> hi;
25                f.add(i, n + j + 1, lo, hi);
26            }
27        }
28        auto ans = f.solve(s, t);
29        cout << (ans ? *ans : -1) << "\n\n";
30    }
31 }
```

## 应用题补充：有源汇上下界最小净流（本地接口示例）

题意：输入带上下界的有向网络，求源点流出减流入的最小净流量，并输出每条原边流量。无解输出 NO；有解输出 YES、最小值及方案。净值可以为负，因此不能用 -1 作为无解哨兵；本地接口未附加“净流非负”约束。

所需模板：Dinic（第 127 页，9.1.1 节）、BoundedCirculation（第 152 页，9.4.1 节）、BoundedMaxFlow（第 154 页，9.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://github.com/gsh20040816/CPCTemplate/issues/11>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, s, t;
6     cin >> n >> m >> s >> t;
7     BoundedMaxFlow g(n);
8     for (int i = 0; i < m; i++)
9     {
10         int u, v;
11         long long lo, hi;
12         cin >> u >> v >> lo >> hi;
13         g.add(u, v, lo, hi);
14     }
15     auto ans = g.minimum(s, t);
16     if (!ans)
17         cout << "NO\n";
18     else
19     {
20         cout << "YES " << *ans << '\n';
21         for (int i = 0; i < m; i++) cout << g.used(i) << '\n';
22     }
23 }
```

## 9.5 Stoer–Wagner 算法

### 9.5.1 无向全局最小割

$\text{solve}(w)$ ,  $n \geq 2$ ;  $w$  为  $n \times n$  对称非负权矩阵, 重边先累加, 主对角置零。返回 {割权, 一侧顶点列表}, 编号  $0..n-1$ ; 两侧非空, 不要求原图连通, 不连通时割权为 0。并列最优割返回其中一种, 不承诺点数或字典序。矩阵按值传入, 原图不变。 $O(n^3)$  时间、 $O(n^2)$  空间; 无向边总权 (每条只计一次) 严格小于  $2^{120}$ , 内部  $\text{int}_{128}$ 。负权不适用。已逐条核验返回点集在原图上的跨割费用, 暂无专门 OJ AC。

```

1 struct StoerWagner
2 {
3     using I = __int128_t;
4
5     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
6     static pair<I, vector<int>> solve(vector<vector<I>> w)
7     {
8         int n = w.size();
9         assert(n >= 2);
10        vector<int> active(n);
11        iota(active.begin(), active.end(), 0);
12        vector<vector<int>> group(n);
13        for (int i = 0; i < n; i++) group[i] = {i};
14        I best = I(1) << 120;
15        vector<int> side;
16        while (active.size() > 1)
17        {
18            vector<I> dis(n);
19            vector<bool> used(n);
20            int prev = -1;
21            for (int step = 0; step < (int)active.size(); step++)
22            {
23                int sel = -1;
24                for (int v : active)
25                    if (!used[v] && (sel == -1 || dis[v] > dis[sel])) sel = v;
            }
        }
    }
}

```

本轮割、合并与继续扩张（接上页同一函数）：

```
26         if (step + 1 == (int)active.size())
27         {
28             if (dis[sel] < best)
29             {
30                 best = dis[sel];
31                 side = group[sel];
32             }
33             for (int v : active)
34                 if (v != prev && v != sel)
35                     w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
36             group[prev].insert(
37                 group[prev].end(), group[sel].begin(), group[sel].end());
38             active.erase(find(active.begin(), active.end(), sel));
39             break;
40         }
41         used[sel] = true;
42         prev = sel;
43         for (int v : active)
44             if (!used[v]) dis[v] += w[sel][v];
45     }
46 }
47 return {best, side};
48 }
49 };
```

## 模板题使用：Luogu P5632

题意：无向连通正权图的全局最小割， $n$  至 600、边权总和至  $10^9$ ；矩阵按无向边双向累加、自环忽略。核心返回费用与 0-based 割侧 side，本题只输出费用。核心要求  $n \geq 2$ ；单点没有非平凡割，驱动额外约定输出 0。

所需模板：StoerWagner（第 158 页，9.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P5632>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     using I = StoerWagner::I;
8     vector<vector<I>> w(n, vector<I>(n));
9     while (m--)
10     {
11         int u, v;
12         long long c;
13         cin >> u >> v >> c;
14         u--;
15         v--;
16         if (u == v) continue;
17         w[u][v] += c;
18         w[v][u] += c;
19     }
20     // A singleton has no nontrivial cut; use zero by convention.
21     if (n == 1)
22     {
23         cout << 0 << '\n';
24         return 0;
25     }
26     auto [ans, side] = StoerWagner::solve(w);
27     // The statement bounds the sum of edge weights by 1e9.
28     cout << (long long)ans << '\n';
29 }
```

# 第十章 图的匹配

## 10.1 二分图最大匹配

### 10.1.1 Hopcroft–Karp 最大匹配与最小点覆盖

左右两侧分别编号  $1..n$ 、 $1..m$ ，可有空侧及重边； $\text{add}(u,v)$  加边， $\text{solve}()$  返回最大匹配总数， $l[u]$  与  $r[v]$  为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次  $\text{solve}$ ；不支持删边或手动更改匹配数组。BFS 用  $\text{dep}[0]$  记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。 $V=n+m$ ，单次  $\text{solve}$  最坏  $O((V+E)\sqrt{V})$ ，空间  $O(V+E)$ 。 $\text{cover}()$  在  $\text{solve}$  后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```
1 struct BipartiteMatching
2 {
3     int n, m;
4     vector<vector<int>> g;
5     vector<int> l, r, dep, cur;
6
7     BipartiteMatching(int n, int m)
8         : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
9     {
10    }
11
12    void add(int u, int v)
13    {
14        assert(1 <= u && u <= n && 1 <= v && v <= m);
15        g[u].push_back(v);
16    }
17
18    bool bfs()
19    {
20        queue<int> q;
21        fill(dep.begin(), dep.end(), n + 1);
22        fill(cur.begin(), cur.end(), 0);
23        for (int u = 1; u <= n; u++)
24        {
25            if (!l[u])
26            {
27                dep[u] = 0;
28                q.push(u);
29            }
30        }
31        while (!q.empty())
32        {
33            int u = q.front();
34            q.pop();
35            if (dep[u] >= dep[0]) continue;
36            for (int v : g[u])
37            {
38                int next = r[v];
39                if (dep[next] != n + 1) continue;
40                dep[next] = dep[u] + 1;
```

```

41         if (next) q.push(next);
42     }
43 }
44 return dep[0] != n + 1;
45 }
46
47 bool dfs(int u)
48 {
49     if (!u) return true;
50     for (int &i = cur[u]; i < (int)g[u].size(); i++)
51     {
52         int v = g[u][i];
53         if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
54         {
55             l[u] = v;
56             r[v] = u;
57             return true;
58         }
59     }
60     dep[u] = n + 1;
61     return false;
62 }
63
64 // Retains an existing matching; edges may be added between calls.
65 int solve()
66 {
67     int answer = 0;
68     for (int u = 1; u <= n; u++) answer += (l[u] != 0);
69     while (bfs())
70     {
71         for (int u = 1; u <= n; u++)
72         {
73             if (!l[u] && dfs(u)) answer++;
74         }
75     }
76     return answer;
77 }
78
79 // Call after solve(), before adding more edges. Returns left/right vertex IDs.
80 pair<vector<int>, vector<int>> cover() const
81 {
82     vector<int> left, right;
83     for (int u = 1; u <= n; u++)
84     {
85         if (dep[u] == n + 1) left.push_back(u);
86     }
87     for (int v = 1; v <= m; v++)
88     {
89         if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
90     }
91     return {left, right};
92 }
93 };

```

## 模板题使用：Luogu P3386

题意：给定二分图，求最大匹配数。左右两侧分别编号  $1..n$ 、 $1..m$ ， $\text{add}(u,v)$  的  $u$  属于左侧， $v$  属于右侧； $\text{solve}$  返回匹配数。此题不输出匹配或最小点覆盖方案。

所需模板：BipartiteMatching（第 161 页，10.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3386>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m, e;
6     std::cin >> n >> m >> e;
7     BipartiteMatching graph(n, m);
8     while (e--)
9     {
10         int u, v;
11         std::cin >> u >> v;
12         graph.add(u, v);
13     }
14     std::cout << graph.solve() << '\n';
15 }
```

### 10.1.2 二分图最大匹配的可行边与必选边

代码依赖：BipartiteMatching (第 161 页, 10.1.1 节)、TarjanSCC (第 74 页, 5.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

matching\_edges(n,m,edges) 接收左右两侧点号各为 1..n、1..m 的边表，返回 (最大匹配大小, 逐边分类)。每项为 {possible,forced}：是否属于至少一个/每一个最大匹配；分类保持输入顺序，平行边作为不同边处理。允许空侧、空图和非完备匹配。先用 BipartiteMatching 求解，再建含源汇的残量图跑 TarjanSCC；已匹配或两端同 SCC 为可行，已匹配且两端不同 SCC 为必选。不能在非完备匹配时省略源汇，也不能把可行条件写成已匹配且同 SCC。时间  $O(E\sqrt{(n+m)+n+m})$ ，空间  $O(n+m+E)$ ，节点与残量边数须可用 int 表示。HK 与 SCC 都保留递归，栈深可为  $O(n+m)$ 。

```

1 pair<int, vector<array<bool, 2>>>
2 matching_edges(int n, int m, const vector<pair<int, int>> &edges)
3 {
4     BipartiteMatching b(n, m);
5     for (auto [u, v] : edges)
6     {
7         assert(1 <= u && u <= n && 1 <= v && v <= m);
8         b.add(u, v);
9     }
10    int size = b.solve(), s = n + m + 1, t = s + 1;
11    TarjanSCC d(t);
12    for (int u = 1; u <= n; u++)
13        if (b.l[u])
14            d.add(u, s);
15        else
16            d.add(s, u);
17    for (int v = 1; v <= m; v++)
18        if (b.r[v])
19            d.add(t, n + v);
20        else
21            d.add(n + v, t);
22    vector<bool> chosen(n + 1), used(edges.size());
23    for (int i = 0; i < (int)edges.size(); i++)
24    {
25        auto [u, v] = edges[i];
26        used[i] = b.l[u] == v && !chosen[u];
27        if (used[i])
28        {
29            chosen[u] = true;
30            d.add(n + v, u);
31        }
32        else
33            d.add(u, n + v);
34    }
35    d.run();
36    vector<array<bool, 2>> ans(edges.size());
37    for (int i = 0; i < (int)edges.size(); i++)
38    {
39        auto [u, v] = edges[i];
40        bool same = d.bel[u] == d.bel[n + v];
41        ans[i] = {used[i] || same, used[i] && !same};
42    }
43    return {size, ans};
44 }
```

## 应用题补充：二分图最优匹配边分类（本地接口示例）

题意：输入左右点数、边数及 1-based 边表，输出最大匹配大小，再按原边顺序输出可行/必选标记。非完备匹配和平行边都允许；本例演示接口，不是 OJ 提交记录。

所需模板：BipartiteMatching（第 161 页，10.1.1 节）、TarjanSCC（第 74 页，5.1.1 节）、matching\_edges（第 164 页，10.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://oi-wiki.org/graph/graph-matching/biggraph-match/>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, k;
6     cin >> n >> m >> k;
7     vector<pair<int, int>> edges(k);
8     for (auto &[u, v] : edges) cin >> u >> v;
9     auto [size, labels] = matching_edges(n, m, edges);
10    cout << size << '\n';
11    for (auto [possible, forced] : labels) cout << possible << ' ' << forced << '\n';
12 }
```

## 10.2 二分图最大权匹配

### 10.2.1 矩形二分图最小权匹配

solve(cost) 输入  $n \times m$  完整权值矩阵， $0 \leq n \leq m$ ；每行匹配不同列，返回最小费用及列号（0-based）。允许负费用，使用 int128 势能； $O(n^2m)$ 。禁止边不能用未经证明的有限大常数随意替代，当前接口仅表示完整二分图。

```

1 struct Hungarian
2 {
3     using I = __int128_t;
4
5     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
6     static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
7     {
8         int n = cost.size();
9         if (!n) return {0, {}};
10        int m = cost[0].size();
11        assert(n <= m);
12        vector<I> u(n + 1), v(m + 1);
13        vector<int> p(m + 1), way(m + 1);
14        const I inf = I(1) << 120;
15        for (int i = 1; i <= n; i++)
16        {
17            assert((int)cost[i - 1].size() == m);
18            p[0] = i;
19            int j0 = 0;
20            vector<I> dis(m + 1, inf);
21            vector<bool> used(m + 1);
22            do
23            {
24                used[j0] = true;
25                int i0 = p[j0], j1 = 0;
26                I delta = inf;
27                for (int j = 1; j <= m; j++)
```

```

28         if (!used[j])
29         {
30             I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
31             if (cur < dis[j])
32             {
33                 dis[j] = cur;
34                 way[j] = j0;
35             }
36             if (dis[j] < delta)
37             {
38                 delta = dis[j];
39                 j1 = j;
40             }
41         }
42     for (int j = 0; j <= m; j++)
43     if (used[j])
44     {
45         u[p[j]] += delta;
46         v[j] -= delta;
47     }
48     else
49         dis[j] -= delta;
50     j0 = j1;
51 } while (p[j0]);
52 do
53 {
54     int j1 = way[j0];
55     p[j0] = p[j1];
56     j0 = j1;
57 } while (j0);
58 }
59 vector<int> ans(n);
60 for (int j = 1; j <= m; j++)
61     if (p[j]) ans[p[j] - 1] = j - 1;
62 return {-v[0], ans};
63 }
64 };

```

## 模板题使用：Library Checker assignment

题意：给完整方阵，为每一行分配一个不同的列，使费用和最小。返回列号是 0-based，按行顺序输出。题目  $n \leq 500$ 、单项绝对值  $\leq 10^9$ ，总费用可由 `int128` 转为 `long long`；缺边问题不能直接套本例。

所需模板：Hungarian（第 165 页，10.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://judge.yosupo.jp/problem/assignment>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     vector<vector<long long>> cost(n, vector<long long>(n));
8     for (auto &row : cost)
9         for (auto &x : row) cin >> x;
10    auto [value, columns] = Hungarian::solve(cost);
11    cout << (long long)value << '\n';
12    for (int j : columns) cout << j << ' ';
13    cout << '\n';
14 }
```

### 10.2.2 二分图最大权匹配（缺边与可选匹配）

`WeightedMatching(n,m)`，两侧编号  $1..n, 1..m$ 。`add(x,y,w)` 加边，重边保留最大权；边权支持完整 `long long` 范围，内部势能与答案为 `int128`。`solve()` 要求全部左点匹配不同右点，无解返回 `nullopt`； $n=m$  时即最大权完美匹配。`solve(true)` 允许点不匹配，只最大化总权，不优先最大化边数。两种模式均从空匹配重算；`l[x]`、`r[y]` 为方案，0 表示未匹配，无解时全部清零。缺边用独立于合法边权的  $-2^{120}$  哨兵，仅用 `add` 修改；不把缺边当零权边。允许空侧。全匹配  $O(n^2m)$ ，可选模式增加  $n$  个虚拟零权右点，为  $O(n^2(m+n))$ ；空间  $O(nm+n+m)$ 。按最短松弛量扩展交错树：每次至少访问一个新右点，最多  $n$  次后增广；无有限松弛量即无解。

```

1 struct WeightedMatching
2 {
3     using I = __int128_t;
4     static constexpr I inf = I(1) << 120;
5     int n, m;
6     vector<vector<I>> w;
7     vector<int> l, r;
8
9     WeightedMatching(int n, int m)
10        : n(n), m(m), w(n + 1, vector<I>(m + 1, -inf)), l(n + 1), r(m + 1)
11    {
12    }
13
14    void add(int x, int y, long long weight)
15    {
16        assert(1 <= x && x <= n && 1 <= y && y <= m);
17        w[x][y] = max(w[x][y], I(weight));
18    }
19
20    optional<I> solve(bool allow_unmatched = false)
21    {
22        fill(l.begin(), l.end(), 0);
23        fill(r.begin(), r.end(), 0);
```

```

24     if (!allow_unmatched && n > m) return nullopt;
25     int cols = m + (allow_unmatched ? n : 0);
26     vector<I> u(n + 1), v(cols + 1);
27     vector<int> p(cols + 1), way(cols + 1);
28     for (int i = 1; i <= n; i++)
29     {
30         p[0] = i;
31         int j0 = 0;
32         vector<I> dis(cols + 1, inf);
33         vector<bool> used(cols + 1);
34         do
35         {
36             used[j0] = true;
37             int x = p[j0], j1 = 0;
38             I delta = inf;
39             for (int j = 1; j <= cols; j++)
40             {
41                 if (used[j]) continue;
42                 I weight = j <= m ? w[x][j] : 0;
43                 if (weight != -inf)
44                 {
45                     I cur = -weight - u[x] - v[j];
46                     if (cur < dis[j])
47                     {
48                         dis[j] = cur;
49                         way[j] = j0;
50                     }
51                 }
52                 if (dis[j] < delta)
53                 {
54                     delta = dis[j];
55                     j1 = j;
56                 }
57             }
58             if (delta == inf) return nullopt;
59             for (int j = 0; j <= cols; j++)
60             {
61                 if (used[j])
62                 {
63                     u[p[j]] += delta;
64                     v[j] -= delta;
65                 }
66                 else if (dis[j] != inf)
67                     dis[j] -= delta;
68             }
69             j0 = j1;
70         } while (p[j0]);
71         do
72         {
73             int j1 = way[j0];
74             p[j0] = p[j1];
75             j0 = j1;
76         } while (j0);
77     }
78     I answer = 0;

```

```

79     for (int j = 1; j <= m; j++)
80     {
81         if (!p[j]) continue;
82         l[p[j]] = j;
83         r[j] = p[j];
84         answer += w[p[j]][j];
85     }
86     return answer;
87 }
88 };

```

## 模板题使用：Luogu P6577

题意：求二分图最大权完美匹配，两侧均为  $n$  点，题目保证存在完美匹配。输出  $r[j]$ ，即右部  $j$  对应的左部编号，均为 1-based；题目权值与规模保证总权值可转为 long long。

所需模板：WeightedMatching（第 167 页，10.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P6577>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     WeightedMatching graph(n, n);
8     while (m--)
9     {
10         int x, y;
11         long long w;
12         std::cin >> x >> y >> w;
13         graph.add(x, y, w);
14     }
15     auto answer = graph.solve();
16     assert(answer);
17     std::cout << (long long)*answer << '\n';
18     for (int j = 1; j <= n; j++) std::cout << graph.r[j] << (j == n ? '\n' : ' ');
19 }

```

## 10.3 一般图最大匹配

### 10.3.1 一般图最大基数匹配：带花树

Blossom( $n$ )，点号  $0..n-1$ ；add( $u,v$ ) 加无向边，忽略自环，允许重边。solve() 从空匹配重新计算，返回匹配边数；mate[u] 为配对点，未匹配为 -1。结果支持直接输出方案。缩花维护交错树，单次增广  $O(n^2+m)$ ，总计  $O(n^3+nm)$ ，简单图为  $O(n^3)$ ；空间  $O(n+m)$ 。重复调用 solve 或加边后重算均可。不处理边权，不能代替最大权匹配。

```

1 struct Blossom
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<int> mate, pre, base, q;
6     vector<bool> outer, flower, seen;
7 }

```

```

8 Blossom(int n)
9 : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
10 {
11 }
12
13 void add(int u, int v)
14 {
15     assert(0 <= u && u < n && 0 <= v && v < n);
16     if (u == v) return;
17     g[u].push_back(v);
18     g[v].push_back(u);
19 }
20
21 int lca(int u, int v)
22 {
23     fill(seen.begin(), seen.end(), false);
24     while (true)
25     {
26         u = base[u];
27         seen[u] = true;
28         if (mate[u] == -1) break;
29         u = pre[mate[u]];
30     }
31     while (!seen[base[v]]) v = pre[mate[base[v]]];
32     return base[v];
33 }
34
35 void mark(int u, int b, int v)
36 {
37     while (base[u] != b)
38     {
39         flower[base[u]] = true;
40         flower[base[mate[u]]] = true;
41         pre[u] = v;
42         v = mate[u];
43         u = pre[v];
44     }
45 }
46
47 bool augment(int root)
48 {
49     fill(outer.begin(), outer.end(), false);
50     fill(pre.begin(), pre.end(), -1);
51     iota(base.begin(), base.end(), 0);
52     q.assign(1, root);
53     outer[root] = true;
54     for (int head = 0; head < (int)q.size(); head++)
55     {
56         int u = q[head];
57         for (int v : g[u])
58         {
59             if (base[u] == base[v] || mate[u] == v) continue;
60             if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
61             {
62                 int b = lca(u, v);
63                 fill(flower.begin(), flower.end(), false);

```

```

64         mark(u, b, v);
65         mark(v, b, u);
66         for (int x = 0; x < n; x++)
67         {
68             if (!flower[base[x]]) continue;
69             base[x] = b;
70             if (!outer[x])
71             {
72                 outer[x] = true;
73                 q.push_back(x);
74             }
75         }
76     }
77     else if (pre[v] == -1)
78     {
79         pre[v] = u;
80         if (mate[v] == -1)
81         {
82             while (v != -1)
83             {
84                 int p = pre[v];
85                 int next = mate[p];
86                 mate[v] = p;
87                 mate[p] = v;
88                 v = next;
89             }
90             return true;
91         }
92         int x = mate[v];
93         outer[x] = true;
94         q.push_back(x);
95     }
96 }
97
98 return false;
99 }
100
101 // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
102 int solve()
103 {
104     fill(mate.begin(), mate.end(), -1);
105     int answer = 0;
106     for (int u = 0; u < n; u++)
107     {
108         if (mate[u] == -1 && augment(u)) answer++;
109     }
110     return answer;
111 }
112 };

```

## 模板题使用：Library Checker general\_matching

题意：给简单无向图，输出最大匹配的边数及匹配边。点号  $0..n-1$ ；solve 后  $\text{mate}[u]=-1$  表示未匹配，只在  $u < \text{mate}[u]$  时输出，避免同一条边打印两次。

所需模板：Blossom（第 169 页，10.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://judge.yosupo.jp/problem/general\\_matching](https://judge.yosupo.jp/problem/general_matching)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     Blossom graph(n);
8     while (m--)
9     {
10         int u, v;
11         cin >> u >> v;
12         graph.add(u, v);
13     }
14     cout << graph.solve() << '\n';
15     for (int u = 0; u < n; u++)
16         if (u < graph.mate[u]) cout << u << ' ' << graph.mate[u] << '\n';
17 }
```

# 第十一章 Prüfer 序列

## 11.1 Prüfer 双射与有标号树计数

本节的树是顶点标号固定为  $1, \dots, n$  的简单无向树，不把同构的不同标号树合并。先设  $n \geq 2$ 。反复删除当前**最小标号叶子**，记录它唯一的邻点，直到剩两个点；得到长度  $n-2$ 、每项属于  $[n]$  的 Prüfer 序列。 $n=2$  对应唯一空序列； $n=1$  另有唯一空边树，度数为 0，不代入含  $n-2$  的公式。

### 11.1.1 双射为何成立

解码时保留所有尚未删去的标号。读下一项  $u$ ，取当前未删除且没有出现在**整个剩余序列**中的最小标号  $v$ ，连边  $uv$ ，删除  $v$  并消耗这一项  $u$ ；最后连接剩下的两个标号。当前有  $t$  个点、 $t-2$  个剩余序列项，所以至少两个标号没有出现，可选点总存在。 $u$  自己仍在序列中，故不会选出自环。每个被删点只连接到更晚删去的点，最终得到一棵连通的  $n-1$  边树。

关键不变式是：在当前剩余树中，标号  $i$  在剩余序列里的出现次数等于  $d_i - 1$ 。每个点最后一条关联边在它作为叶子被删除时移除，或在最后两点之间保留到结束；这一条不记录该点。此前每删除一个邻点，就记录一次它。每一步删除一个叶子，会同时使邻点的度数和出现次数减少 1，因此不变式继续成立。反过来看解码得到的剩余树，每个活跃点有一条最终连接边，再加上剩余序列每次出现它时接来的边；其度数同样是剩余出现次数加 1。所以未出现的活跃标号恰好是叶子；两边都取最小者，编码和解码逐步互为逆操作。特别地，原树满足

$$d_i = 1 + \#\{\text{序列中等于 } i \text{ 的项}\}.$$

本节给出计数依据，不新增可调用的编码/解码 API。

### 11.1.2 Cayley、指定度数与叶子数

任意长度  $n-2$  的序列都对应唯一树，故树总数为  $n^{n-2}$ 。给定整数度数  $d_1, \dots, d_n$ ，若某个  $d_i < 1$  或  $\sum_i d_i \neq 2n-2$ ，答案为 0；否则各标号出现次数已确定，答案是多项式系数

$$\frac{(n-2)!}{\prod_{i=1}^n (d_i - 1)!}.$$

例如  $n=4$ ，度数  $(3, 1, 1, 1)$  给出唯一指定中心的星形树，度数  $(2, 2, 1, 1)$  则有 2 棵，而不是把两名度为 2 的顶点视作不可区分。

叶子恰是没有出现的标号。恰有  $\ell$  个叶子的树数为

$$\binom{n}{\ell} (n-\ell)! S(n-2, n-\ell),$$

其中  $S$  为第二类 Stirling 数：先选叶子，再把序列位置满射到其余标号。 $n \geq 3$  时合法范围为  $2 \leq \ell \leq n-1$ ；其他叶子数为 0。 $n=2, \ell=2$  时用  $S(0, 0) = 1$  得 1。若把叶子定义为度数 1，单点树没有叶子，仍单独处理。这里是整数计数恒等式；取模时不能无条件把阶乘分母换成逆元。素数模的预处理与下标限制见第 B.1.1 节（第 201 页），一般模数条件见数学分册“模运算的适用条件”。有根标号树及其生成函数见数学分册“隐式生成函数与拉格朗日反演”，不重复另造树计数模板。

### 11.1.3 加权 Cayley 与连接已有连通块

若完整图的边权有特殊乘积形式  $w_{ij} = x_i x_j$ ，则

$$\sum_T \prod_{ij \in T} w_{ij} = \sum_T \prod_i x_i^{d(i)} = \left( \prod_i x_i \right) \left( \sum_i x_i \right)^{n-2}.$$

这里  $d(i)$  表示点  $i$  的度数。第二步利用出现次数恒等式，把每棵树变成公共因子  $\prod_i x_i$  乘以其序列各项的权；对所有序列求和就是展开右侧的幂。证明没有除以任何  $x_i$ ，所以零权、负权以及任意正模数都可按多项式恒等式使用。 $n = 2$  的零次幂是常数 1，即使  $\sum_i x_i = 0$  也如此。这只适用于上述乘积边权；缺边图或一般边权应使用矩阵树定理，见第 12.3 节（第 180 页）及第 12.2 节（第 177 页）。代数权值和为零也可能来自抵消，不能据此断言没有树。

设一个固定图有  $k \geq 2$  个连通块，大小为正整数  $s_1, \dots, s_k$ ，总顶点数  $N = \sum_i s_i$ 。允许任选不同连通块的顶点对加边，使全图连通，且恰加  $k - 1$  条边。缩点后新增边必须组成树；一条块间边  $ij$  有  $s_i s_j$  种端点选择。因此不同**无序新增边集**的数量是

$$N^{k-2} \prod_{i=1}^k s_i.$$

$k = 1$  时无需加边，答案为 1，不计算负指数。原图的内边固定，不计内部生成树选择。若禁止部分端点对、给每条新增边限制，或数加边的执行顺序，就不能直接使用此式。有标号树也不能简单除以  $n!$  得到无标号树数，因为不同树的自同构数量不同。

双射背景见 [Stanley, Topics in Algebraic Combinatorics](#)，导航与基础公式见 [固定版本 OI Wiki Prüfer 序列](#)。以上用最小叶子的一致约定推导；不将本节计数说明当作整个上游页面的实现覆盖。

## 第十二章 矩阵树定理

### 12.1 带权矩阵树：无向树与内外向树形图

代码依赖：ModInt (第 197 页, A.1.1 节)、det\_prime (第 203 页, C.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

声明 MatrixTree<mod>, 调用 count(n, edges, root, kind) 返回所有合法树的边权乘积之和, mod 必须是素数。Edge=(u,v,Z), 点号 0..n-1,  $n \geq 1$ ; Kind::undirected 为无向生成树, toward\_root 为每点沿边能到根, away\_from\_root 为根沿边能到每点。后两种使用输入边方向, 不能混淆。重边按不同边计数, 自环忽略; 单点答案 1。支持模意义下的零权和负权, 结果 0 不一定表示不存在树。无向图删除哪个根行列均可; 有向图 root 决定答案。依赖 det\_prime 的素数模行列式,  $O(n^3+m)$  时间、 $O(n^2)$  额外空间。

```
1 template <int mod> struct MatrixTree
2 {
3     using Z = ModInt<mod>;
4     using Edge = tuple<int, int, Z>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
13    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
14    {
15        assert(n >= 1 && root >= 0 && root < n);
16        vector<vector<Z>> lap(n - 1, vector<Z>(n - 1));
17        auto add = [&](int u, int v, Z w)
18        {
19            if (u == root) return;
20            int i = u - (u > root), j = v - (v > root);
21            lap[i][i] = lap[i][i] + w;
22            if (v != root) lap[i][j] = lap[i][j] - w;
23        };
24        for (auto [u, v, w] : edges)
25        {
26            assert(u >= 0 && u < n && v >= 0 && v < n);
27            if (u == v) continue;
28            if (kind == Kind::away_from_root) swap(u, v);
29            add(u, v, w);
30            if (kind == Kind::undirected) add(v, u, w);
31        }
32        return det_prime<mod>(move(lap));
33    }
34};
```

## 模板题使用：Luogu P6178

题意：求所有生成树边权乘积之和，模 1000000007。type=0 为无向图，type=1 为以输入点 1 为根的外向树；输入点号减 1 后根为 0。重边分别计数，自环不贡献。

所需模板：ModInt (第 197 页, A.1.1 节)、det\_prime (第 203 页, C.1.1 节)、MatrixTree (第 175 页, 12.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P6178>

```
1 int main()
2 {
3     using M = MatrixTree<1000000007>;
4     int n, m, type;
5     scanf("%d%d%d", &n, &m, &type);
6     vector<M::Edge> edges;
7     for (int i = 0; i < m; i++)
8     {
9         int u, v;
10        long long w;
11        scanf("%d%d%lld", &u, &v, &w);
12        edges.push_back({u - 1, v - 1, w});
13    }
14    auto kind = type ? M::Kind::away_from_root : M::Kind::undirected;
15    printf("%d\n", M::count(n, edges, 0, kind).v);
16 }
```

## 12.2 任意正模数带权矩阵树

代码依赖：`determinant_mod` (第 205 页, C.1.2 节)。抄写时一并准备；标准库类型不列入算法依赖。

`count(n, edges, root, kind, mod)` 返回树的边权乘积之和对 `mod` 的最小非负余数。`mod` 为任意正 signed 64 位数，边权支持完整 signed 64 位。`Kind` 的 `undirected`、`toward_root`、`away_from_root` 分别表示无向、内向和外向树；点号 `0..n-1`。自环忽略，重边按不同边计数，单点返回 `1%mod`，`mod=1` 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 `determinant_mod`； $O(n^3 \log \text{mod} + m)$  时间、 $O(n^2)$  额外空间。不使用模逆元。已知模数是素数时一般优先使用 `MatrixTree<mod>`。

```

1 struct MatrixTreeMod
2 {
3     using ll = long long;
4     using Edge = tuple<int, int, ll>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
13    {
14        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
15        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
16        auto add = [&](int u, int v, ll w)
17        {
18            if (u == root) return;
19            int i = u - (u > root), j = v - (v > root);
20            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
21            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
22        };
23        for (auto [u, v, w] : edges)
24        {
25            assert(u >= 0 && u < n && v >= 0 && v < n);
26            if (u == v) continue;
27            w %= mod;
28            if (w < 0) w += mod;
29            if (kind == Kind::away_from_root) swap(u, v);
30            add(u, v, w);
31            if (kind == Kind::undirected) add(v, u, w);
32        }
33        return determinant_mod(move(lap), mod);
34    }
35 };

```

## 模板题使用：Library Checker Counting Spanning Trees (Undirected)

题意：无向多重图生成树计数，模 998244353；点号  $0..n-1$ ， $1 \leq n \leq 500$ 、 $0 \leq m \leq 500000$ ，允许自环和重边。每条原边权重设 1，自环不参与树，重边是不同选择。删去任意根的拉普拉斯行列，这里取根 0； $n=1$  的空树计 1，断连计 0。此接口也支持合数模数，复杂度  $O(n^3 \log \text{mod})$ ，本题固定质数亦可选更快的 MatrixTree 版本。

所需模板：[determinant\\_mod](#)（第 205 页，C.1.2 节）、[MatrixTreeMod](#)（第 177 页，12.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://judge.yosupo.jp/problem/counting\\_spanning\\_tree\\_undirected](https://judge.yosupo.jp/problem/counting_spanning_tree_undirected)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<MatrixTreeMod::Edge> e;
8     e.reserve(m);
9     for (int i = 0; i < m; i++)
10    {
11        int u, v;
12        cin >> u >> v;
13        e.emplace_back(u, v, 1);
14    }
15    cout << MatrixTreeMod::count(n, e, 0, MatrixTreeMod::Kind::undirected, 998244353)
16         << '\n';
17 }
```

## 模板题使用：Library Checker Counting Spanning Trees (Directed)

题意：有向多重图生成树计数，模 998244353，要求指定根  $r$  能到达所有点，使用 `away_from_root`。 $n$  不超过 500、 $m$  不超过 500000，点号及根为 0 起；每条原边权 1，自环忽略，重边分别计数。每个非根点恰有一个入边，根无入边；`toward_root` 表示所有点到达根，不能替代。 $n=1$  计空树 1，无外向生成树计 0。

所需模板：[determinant\\_mod](#) (第 205 页, C.1.2 节)、[MatrixTreeMod](#) (第 177 页, 12.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：[https://judge.yosupo.jp/problem/counting\\_spanning\\_tree\\_directed](https://judge.yosupo.jp/problem/counting_spanning_tree_directed)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, r;
6     cin >> n >> m >> r;
7     vector<MatrixTreeMod::Edge> e;
8     e.reserve(m);
9     for (int i = 0; i < m; i++)
10    {
11        int u, v;
12        cin >> u >> v;
13        e.emplace_back(u, v, 1);
14    }
15    cout << MatrixTreeMod::count(
16        n, e, r, MatrixTreeMod::Kind::away_from_root, 998244353)
17        << '\n';
18 }
```

## 12.3 带权矩阵树定理与方向约定

实现见第 12.1 节 (第 175 页)。返回的是每棵树的边权乘积, 再对所有树求和, 并对素数取模。无权计数只需把所有边权设为 1。重边是不同选择, 自环不属于任何生成树; 单点图的空树乘积为 1。零权和负权都按域上的运算处理, 因此结果为零可能来自抵消, 不能作为连通性判定。

无向边  $uv$  权值  $w$  对拉普拉斯矩阵贡献为  $L_{uu}, L_{vv}$  各加  $w$ ,  $L_{uv}, L_{vu}$  各减  $w$ 。删除 root 行和列后求行列式, 与 root 的选择无关。对于每个点都能沿有向边走到根的树形图, 边  $u \rightarrow v$  贡献  $L_{uu} += w, L_{uv} -= w$ ; 删除根行列后得到内向树形图的权值和。外向树形图先反转所有边, 再使用同一构造。

例如只有一条边  $0 \rightarrow 1$  时, 内向根 1 的答案为该边权, 内向根 0 为零; 外向根 0 的答案为该边权。P6178 的有向模式 (type=1) 指定以 1 为根的外向树; type=0 为无向图, 完整用法见第 12.1 节 (第 176 页); 驱动把点号减一, 有向模式选择 `away_from_root`。只直接构造  $(n-1) \times (n-1)$  余子式, 时间  $O(n^3 + m)$ 、额外空间  $O(n^2)$ 。行列式复用本书的素数模高斯消元, 不支持将 mod 随意改成合数。

### 12.3.1 为什么这个余子式恰好计数

先证内向情形。删去根  $r$  的坐标, 令  $e_r = 0$ , 其他  $e_i$  为单位行向量。余子式第  $i$  行是  $\sum_{j \neq r} w_{ij}(e_i - e_j)$ 。按行的多线性展开, 每项从每个非根点恰选一条出边, 权为所选边权之积。若这些边含有有向环, 环上向量  $e_i - e_j$  相加为零, 该项行列式为零。若无环, 有限图中每条路径只能在没有出边的根终止。按到根距离从小到大, 同时重排行列, 得到对角全为 1 的三角矩阵, 行列式为 1。因此只有内向生成树保留下来, 每棵树的权恰计一次。

自环的向量为零; 重边在展开时是不同选择; 单点对应空行列式与空乘积 1。无向边替换成权相同的两条反向弧, 每棵无向生成树恰有一种朝根定向; 有向边全部反转则把外向树变成内向树。整个论证是整数系数多项式恒等式, 没有除法, 故允许负权、零权及任意正模数。素数限制来自消元实现, 不来自矩阵树定理本身。

证明依据行列式多线性与有向树的出度刻画, 参见 MIT 18.212 第 29 讲; 这里直接按选边后的环与根路径给出主余子式的证明, 不扩展为任意非主余子式公式。

### 12.3.2 既有接口与验证记录

测试独立枚举  $n-1$  条边的所有选择, 检查连通性或根向约束, 用任意精度整数计算边权乘积之和, 再分别对多个素数取模。两套实现覆盖所有根、三种方向、自环重边、负权及 64 位权值边界, 并通过 220 点完全图的加权 Cayley 公式检查。P6178 完整驱动另测 300 点、十万条边以及输入输出约定。普通与 ASan/UBSan 均通过。两套代码已在 P6178 通过全部 11 个测试点: [动态版记录 297619886](#)、[传统版记录 297620263](#)。单点最大用时分别为 64 ms、62 ms, 内存峰值为 2.85 MB、2.40 MB。在线测试覆盖无向与根为 1 的外向树; 其他根、内向树和负权仍使用本地证据。定理背景可参考 [OI Wiki 矩阵树定理](#)。

## 12.4 任意正模数的矩阵树计数

实现见第 12.2 节（第 177 页）。拉普拉斯余子式表示生成树权值和的恒等式本身不要求模数为素数；需要换掉的是依赖模逆元的行列式算法。本节沿用第 12.3 节（第 180 页）的三种构造，但使用欧几里得行列式消元。输入边权和正模数均支持 signed 64 位，接口名为 `MatrixTreeMod::count`。行列式实现见第 C.1.2 节（第 205 页），时间  $O(n^3 \log(m+1) + |E|)$ 、空间  $O(n^2)$ ；此处  $m$  为模数。

先把边权正规化到  $[0, m)$ ，每次矩阵元素加减仍先扩展至 128 位，再取模存回 64 位。重边的权值和可能超过 64 位范围，不能先累加到普通整数中。只构造删除根后的余子式，并将其存储直接交给行列式函数。单点返回  $1 \bmod m$ ，模 1 返回零。负权与零权仍按代数权值处理，返回零不能直接解释为没有生成树。

独立参考程序逐个枚举边集，用任意精度整数计算权值和。已检查多个合数模数、所有根与三种方向、断图、重边自环及 64 位极值权重；另以大权值完全图的 Cayley 公式验证 160 点规模。普通与 ASan/UBSan 均通过，尚无该接口的在线 AC。此前 P6178 的 AC 属于素数模版本，不自动转移到本接口。

# 第十三章 LGV 引理

## 13.1 LGV: DAG 路径权值行列式

代码依赖: `ModInt` (第 197 页, A.1.1 节)、`det_prime` (第 203 页, C.1.1 节)。抄写时一并准备; 标准库类型不列入算法依赖。

`dag_path_determinant<mod>(g,sources,sinks)` 返回路径权值矩阵的行列式, `mod` 必须为素数。`g[u]` 的项为  $(v, \text{ModInt}<\text{mod}> \text{ 权值})$ , 点号  $0..n-1$ ; 两个端点序列等长  $k$ , 各自不能重复, 但可相互重叠。允许重边、零权、负权; 长度为 0 的路径权为 1。整张图有环则返回 `nullopt`, 空端点族在 DAG 上返回 1。由 LGV, 它是顶点不交路径族按汇点排列带符号的权值和; 只有能证明非恒等排列均不可能时, 才是对应配对的普通计数。顺序不能随意排序, 模结果为 0 也不能据此断言不存在路径族。时间  $O((k+1)(n+m)+k^3+k \log \text{mod})$ , 额外空间  $O(n+k^2)$ , 依赖 `det_prime` 和 `ModInt`。

```
1 template <int mod>
2 optional<ModInt<mod>>
3 dag_path_determinant(const vector<vector<pair<int, ModInt<mod>>>> &g,
4                      const vector<int> &sources,
5                      const vector<int> &sinks)
6 {
7     using Z = ModInt<mod>;
8     int n = g.size(), k = sources.size();
9     assert((int)sinks.size() == k);
10    vector<int> in(n), order;
11    vector<bool> seen_source(n), seen_sink(n);
12    for (int i = 0; i < k; i++)
13    {
14        assert(0 <= sources[i] && sources[i] < n);
15        assert(0 <= sinks[i] && sinks[i] < n);
16        assert(!seen_source[sources[i]] && !seen_sink[sinks[i]]);
17        seen_source[sources[i]] = seen_sink[sinks[i]] = true;
18    }
19    for (const auto &edges : g)
20        for (auto [v, w] : edges)
21        {
22            assert(0 <= v && v < n);
23            in[v]++;
24        }
25    for (int u = 0; u < n; u++)
26        if (!in[u]) order.push_back(u);
27    for (int i = 0; i < (int)order.size(); i++)
28        for (auto [v, w] : g[order[i]])
29            if (!--in[v]) order.push_back(v);
30    if ((int)order.size() != n) return nullopt;
```

```

31     vector<vector<Z>> a(k, vector<Z>(k));
32     for (int i = 0; i < k; i++)
33     {
34         vector<Z> dp(n);
35         dp[sources[i]] = 1;
36         for (int u : order)
37             for (auto [v, w] : g[u]) dp[v] = dp[v] + dp[u] * w;
38         for (int j = 0; j < k; j++) a[i][j] = dp[sinks[j]];
39     }
40     return det_prime<mod>(move(a));
41 }

```

## 应用题补充：DAG 路径行列式（本地接口示例）

题意：输入  $n, m, k$ 、 $m$  条有向带权边、 $k$  个起点和  $k$  个终点，均为 0-based。输出模 998244353 的带符号路径族权值和，整图有环输出 CYCLE。端点顺序原样保留，不能把负号当作错误去掉。

所需模板：ModInt (第 197 页, A.1.1 节)、det\_prime (第 203 页, C.1.1 节)、dag\_path\_determinant (第 182 页, 13.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://github.com/gsh20040816/CPCTemplate/issues/11>

```

1  int main()
2  {
3      using Z = ModInt<998244353>;
4      int n, m, k;
5      cin >> n >> m >> k;
6      vector<vector<pair<int, Z>>> g(n);
7      for (int i = 0; i < m; i++)
8      {
9          int u, v;
10         long long w;
11         cin >> u >> v >> w;
12         g[u].push_back({v, Z(w)});
13     }
14     vector<int> sources(k), sinks(k);
15     for (int &u : sources) cin >> u;
16     for (int &u : sinks) cin >> u;
17     auto ans = dag_path_determinant<998244353>(g, sources, sinks);
18     if (!ans)
19         cout << "CYCLE\n";
20     else
21         cout << ans->v << '\n';
22 }

```

# 第十四章 支配树

## 14.1 支配树：Lengauer–Tarjan

DominatorTree(n), 点号 1..n; add(u,v) 加有向边, 允许重边、自环。build(root) 后 idom 为原点号直接支配点, 根指向自身, 不可达为 0。仅对 root 可达点定义支配关系: 所有 root 到 v 的路径均经过 u, 等价于 u 在支配树上是 v 的祖先。timer 是可达点数, vertex[1..timer] 为 DFS 顺序, dfn 为逆映射; 逆序向 idom 累加可得支配点数, 根不向自身累加。g 保留原点号; pred、bucket、semi 等辅助数组使用 DFS 编号。可更换根重建; 加边使预处理失效, 需重新 build。简单 LT 时间  $O((n+m) \log(n+1))$ , 空间及递归栈  $O(n+m)$  与  $O(n)$ , n+1、边数须可由 int 表示。一般图与 DAG 均适用; 可由 idom 建树组合 LCA。

```
1 struct DominatorTree
2 {
3     int n, timer = 0;
4     vector<vector<int>> g, pred, bucket;
5     vector<int> dfn, vertex, parent, semi, label, ancestor, dom, idom;
6
7     DominatorTree(int n) : n(n), g(n + 1) { assert(n > 0); }
8
9     void add(int u, int v)
10    {
11        assert(1 <= u && u <= n && 1 <= v && v <= n);
12        g[u].push_back(v);
13        idom.clear();
14    }
15
16    void dfs(int u)
17    {
18        dfn[u] = ++timer;
19        vertex[timer] = u;
20        for (int v : g[u])
21            if (!dfn[v])
22            {
23                dfs(v);
24                parent[dfn[v]] = dfn[u];
25            }
26    }
27
28    void compress(int v)
29    {
30        int a = ancestor[v];
31        if (ancestor[a])
32        {
33            compress(a);
34            if (semi[label[a]] < semi[label[v]]) label[v] = label[a];
35            ancestor[v] = ancestor[a];
36        }
37    }
```

路径查询、半支配点与直接支配点计算（接上页同一结构体）：

```

39     int eval(int v)
40     {
41         compress(v);
42         return label[v];
43     }
44
45     void build(int root)
46     {
47         assert(1 <= root && root <= n);
48         timer = 0;
49         dfn.assign(n + 1, 0);
50         vertex.assign(n + 1, 0);
51         parent.assign(n + 1, 0);
52         ancestor.assign(n + 1, 0);
53         dom.assign(n + 1, 0);
54         idom.assign(n + 1, 0);
55         pred.assign(n + 1, {});
56         bucket.assign(n + 1, {});
57         semi.resize(n + 1);
58         iota(semi.begin(), semi.end(), 0);
59         label = semi;
60         dfs(root);
61         for (int u = 1; u <= n; u++)
62             if (dfn[u])
63                 for (int v : g[u]) pred[dfn[v]].push_back(dfn[u]);
64         for (int i = timer; i >= 2; i--)
65         {
66             for (int p : pred[i]) semi[i] = min(semi[i], semi[eval(p)]);
67             bucket[semi[i]].push_back(i);
68             ancestor[i] = parent[i];
69             for (int v : bucket[parent[i]])
70             {
71                 int u = eval(v);
72                 dom[v] = semi[u] < semi[v] ? u : parent[i];
73             }
74             bucket[parent[i]].clear();
75         }
76         for (int i = 2; i <= timer; i++)
77         {
78             if (dom[i] != semi[i]) dom[i] = dom[dom[i]];
79             idom[vertex[i]] = vertex[dom[i]];
80         }
81         idom[root] = root;
82     }
83 };

```

## 模板题使用：Library Checker dominatortree

题意：从指定起点求每个点的直接支配点。输入输出是 0-based，调用模板时加 1；输出减 1 后，不可达点为 -1，根仍指向自身。

所需模板：DominatorTree（第 184 页，14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://judge.yosupo.jp/problem/dominatortree>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, s;
6     cin >> n >> m >> s;
7     DominatorTree t(n);
8     while (m--)
9     {
10         int u, v;
11         cin >> u >> v;
12         t.add(u + 1, v + 1);
13     }
14     t.build(s + 1);
15     for (int u = 1; u <= n; u++) cout << t.idom[u] - 1 << (u == n ? '\n' : ' ');
16 }
```

## 模板题使用：Luogu P5180

题意：从点 1 出发，输出各点支配的可达点数，包含自己。按 DFS 序逆序向直接支配点累加，根不向自身累加；不可达点输出 0。

所需模板：DominatorTree（第 184 页，14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P5180>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     DominatorTree t(n);
8     while (m--)
9     {
10         int u, v;
11         cin >> u >> v;
12         t.add(u, v);
13     }
14     t.build(1);
15     vector<int> ans(n + 1);
16     for (int i = t.timer; i >= 1; i--)
17     {
18         int u = t.vertex[i];
19         ans[u]++;
20         if (u != 1) ans[t.idom[u]] += ans[u];
21     }
22     for (int u = 1; u <= n; u++) cout << ans[u] << (u == n ? '\n' : ' ');
23 }
```

23

}

## 第十五章 配套知识与验证范围

### 15.1 树路径交：投影与单点交

核心见第 18 页，P3398 完整用法见第 19 页。将  $x$  投影到树路径  $a-b$ ，投影是  $\text{lca}(a, b)$ 、 $\text{lca}(a, x)$ 、 $\text{lca}(b, x)$  中深度最大者。三条两点间路径在同一个分叉点相遇；按同一根计算 LCA 即得这个分叉点，与顶点编号无关。设  $c, d$  的投影是  $p, q$ 。若  $p \neq q$ ，路径  $c-d$  必经过  $p-q$ ，后者正是两路径的交集。若  $p = q$ ，两端可能在同一条旁支，不能直接认为有交；须检查  $d(c, p) + d(p, d) = d(c, d)$ 。通过时为单点交，否则为空。这里距离和 depth 都按边数，不能换成带权根距离；零权边会破坏上述距离等式的判别。

接口返回交集端点和顶点数；空交是  $\{0, 0, 0\}$ ，公共边数为  $\max(0, \text{vertices} - 1)$ 。P3398 仅输出是否相交，不验证具体端点。本地接口测试穷举至五点的全部标号树、每个根及四端点，独立构造两条路径的顶点集合求交，核验数量与返回端点间的整条路径。完整驱动另验 146 棵小树、100 棵随机换号树，以及十万点链和十万次查询；两种编译模式均通过。EulerLCA 预处理  $O(n \log n)$ 、每次路径交  $O(1)$ ；若换用 HLD 回调则为  $O(\log n)$ 。完整驱动以 C++20 O2 在 P3398 记录 299972443 通过全部十个测试点，最慢单点 134 ms、峰值 31.89 MB；该 AC 仅对应布尔相交判定，不涵盖端点、精确数量或带权距离。

## 15.2 非负权树的点集直径摘要

代码见第 4 页。一个非空点集只需保存任意一对直径端点  $a, b$ ，即可回答任意树上点  $x$  到该点集的最远距离： $\max(d(x, a), d(x, b))$ 。边权非负，包括零权；该结论不能推广到一般图或负权树。

证明可将点  $u$  投影到路径  $a-b$ ，记投影距  $a$  为  $t$ ，离路径的距离为  $h$ ，直径长为  $D$ 。由于  $d(a, u), d(b, u) \leq D$ ，有  $h \leq \min(t, D - t)$ 。再将查询点投影到同一路径，投影距  $a$  为  $s$ ，离路径的距离为  $g$ 。经两个投影点连接得到  $d(x, u) \leq g + h + |s - t| \leq g + \max(s, D - s)$ ，右侧恰为  $\max(d(x, a), d(x, b))$ 。因此插入新点只需比较到两个旧端点的距离；合并点集时，只插入另一个摘要的两个端点即可。并列直径可任选，不应假设端点编号固定或合并后的端点与合并顺序无关。

Euler LCA 现在同时保存根距离，但其原有 `distance` 接口仍返回边数。带权距离请显式调用 `weighted_distance / Weighted_Distance`；采用  $(D_u - D_z) + (D_v - D_z)$  避免不必要的两倍根距离中间值。所有路径权和须能用 `long long` 表示。独立验证穷举小点集及合并，逐点检查最远距离与返回顶点；带权路径用独立树遍历参照，并覆盖全部根、零边权与 `LLONG_MAX` 单边。原有 Euler LCA 全小树回归，以及 50 万点、边权  $10^9$  的递归链也通过普通与 ASan/UBSan 检查。

**离线添叶仍是固定树度量。** [CF379F New Year Tree](#) 的完整用法见第 6 页。先把全部添叶操作建成最终树，再用 HLD 求距离；只有旧树外接新叶，不会改变两个旧点之间的唯一简单路径。因此可以按出现顺序向摘要插点，尚未出现的点只帮助预处理距离，不提前进入点集。初始四点也必须插入；每次两个新点都插完，再输出 `length`。 $q \leq 500000$  时最终有  $4 + 2q$  个点，采用 HLD 的  $O(q)$  空间、 $O(q \log q)$  总时间。递归 DFS 保留；本地放大测试栈，最大实例包含五十万层链。250 组随机合法添叶过程逐步重建当前树，以独立两次 BFS 求直径，另验满规模链；普通与 ASan/UBSan 均通过。这份用法仅覆盖 `insert` 和 `length`；任意点集合并、空集和最远点见接口独立测试。2026-09-29 洛谷提交入口返回 Codeforces RemoteJudge 暂不可用，故不登记此题线上 AC。

**杭州 2023 F** 中，点权互异。令  $g$  为全树点权的 `mex`，只需按点权  $0, 1, \dots, g - 1$  建立前缀摘要。某个前缀全部位于以  $x$  为中心、半径  $k$  的范围内，当且仅当该摘要的最远距离不超过  $k$ 。二分最长满足条件的前缀长度即为答案； $g = 0$  时直接为零。题面树边带权，不能用边数距离。完整驱动在 `verify/qoj/8235.*.cpp`，本地使用实际权距与 `mex` 集合参照， $n = q = 500000$  的链和星形树保持递归 DFS。两套驱动各 50 个正式测试和 1 个额外测试全部在线 AC：[动态版](#) 666 ms、198076 KB，[传统版](#) 588 ms、198196 KB。OJ 验证题面正权边、前缀插入和最远距离判定；零权边、完整 64 位距离与任意摘要合并仍由独立本地验证支持。

### 15.3 函数图的人环链与到达查询

代码见第 76 页。三份完整用法分别见第 80 页 (CSES1750 跳转)、第 81 页 (CSES1160 最少步数)、第 81 页 (P2921 不同访问点数)。题目点号均从 1 开始, 构造时统一减 1; 只有跳转返回的顶点需要加 1, 最少步数的 0 和不可达的 -1 应原样输出。P2921 只需 depth 加环长, 不代表它测试了跳转或两点可达查询。本轮完整驱动穷举至四点的全部 288 个后继映射, 独立沿后继走到首次重复, 核验步数、跳转与访问数; 另验随机映射和题目最大规模。CSES1750 的题面步数上限为  $10^9$ , 完整 unsigned64 范围另由本地测试覆盖, 不能由该题 AC 推出。

每个点有且仅有一个后继, 故每个弱连通分量由一个有向环和若干指向环的树组成。先不断移除入度为零的点, 剩下的点恰在环上; 沿后继枚举各环。反向处理移除序列时, 后继的环编号、入环点和入环距离均已确定, 可直接传给当前点。这里使用的是入度剥离队列, 不是用显式栈模拟 DFS。

跳转  $k$  步时, 若  $k$  小于入环距离, 用普通倍增; 否则先到入环点, 减去入环步数, 再对环长取模。因此即使  $k$  达到无符号 64 位上界, 也只需  $O(\log n)$  层倍增表。环上位置相加使用无符号宽整数, 避免两个大下标的 int 中间值溢出。

判断从  $u$  到  $v$  的最少步数时: 不同环编号必不可达。若  $v$  在入环树上, 先检查  $u$  的深度至少为  $v$  的深度, 再跳过深度差, 看是否恰好到  $v$ , 从而排除同分量的不同分支。若  $v$  在环上, 答案为  $u$  的入环步数加上沿后继方向的环上距离。不可达返回 -1, 从自己到自己为零步。字段 position 只对环点有意义, 不要将树点的 -1 当作环坐标。

独立参照逐步行走直至首次重复, 同时记录每个点的第一次出现位置。穷举不超过 5 个点的全部后继映射, 并检查跳转、最少步数、入环点与环长; 另验证同对象重建、空图、50 万点长链连环、50 万个自环和完整无符号 64 位步数。普通与 ASan/UBSan 均通过。完整驱动对应 CSES 1750、CSES 1160 和 P2921, 均使用 vector 版驱动。前两题检查跳转与可达步数, 后一题仅使用“入环步数+环长”, 不能替代查询接口的验证。P2921 的当前程序已取得[线上 AC](#); 两道 CSES 查询题的线上评测仍待完成。

## 15.4 杭州 2023 H: 随机事件顺序与函数图

[Sugar Sweet II](#) 的完整驱动为 `verify/qoj/8237.*.cpp`，复用函数图分解与逆阶乘表。事件  $u$  发生前，自己的数量仍为  $a_u$ ；目标  $v = b_u$  只可能为  $a_v$  或  $a_v + w_v$ 。因此  $a_u < a_v$  时必定发生， $a_u \geq a_v + w_v$  时必定不发生。其余情况下，事件  $u$  成功当且仅当事件  $v$  更早执行且成功。

将两类确定事件的后继改为自己，另保存哪些是确定成功的终点。未修改的依赖环没有成功起点：最早执行的环上事件必失败，后续也无法形成成功链。自环同理，自己不可能严格少于自己。沿函数图到达确定失败终点或纯依赖环的节点，成功概率都是零。若到达确定成功终点，且入环距离为  $d$ ，沿途  $d + 1$  个事件必须按唯一的顺序出现。均匀随机全排列对这  $d + 1$  个事件诱导均匀相对顺序，因此成功概率为  $1/(d + 1)!$ 。这些顺序约束有关联，不能将每条依赖当作独立的二分之一概率。期望为  $a_u + w_u/(d + 1)!$ ，在模  $10^9 + 7$  下计算。

题面所有增量为正， $\sum n \leq 500000$ ；阶乘表跨测试组只扩充不缩小，避免大量单点测试时反复求逆。当前驱动复用带倍增表的通用函数图，时间、空间为  $O(n \log n)$ ；概率传播本身只读取 `entry` 和 `depth`，不调用跳转与最少步数接口。

验证穷举小输入及事件全排列，逐步修改真实糖果数量，最后汇总精确总和并除以排列数。另外检查 50 万点的依赖链、纯环，以及 50 万组单点测试，普通与 ASan/UBSan 均通过。两套驱动各 31 个正式测试与 2 个额外测试全过：[动态版](#) 162 ms、84608 KB，[传统版](#) 158 ms、84612 KB。本应用不能验证函数图的 `advance` 或 `steps` 接口。

## 15.5 杭州 2023 G：开放时间的单位边最短路

驱动 `verify/qoj/8236*.cpp` 直接枚举网格邻居，不建立四倍规模的边表。初始蛇身从头到尾编号  $0, \dots, k-1$ ，第  $i \geq 1$  节所在点的最早进入时间为  $k-i$ ，头与其他空格为零。旧蛇尾允许在同一步进入，所以尾格的限制必须是 1。长度为 4 的满  $2 \times 2$  网格样例距离为 0, 3, 2, 1，平方和为 14。

每步只能删除或移动一个尾节，故这些时间限制必要。最短路可以取不重复顶点的路径；行走前把途中等待的总时间全部用于缩短蛇身，随后沿路径连续移动。进入初始蛇身第  $i$  节时，已删除的尾节数量至少为  $k-i$ ；新形成的蛇身不会与无重复顶点的路径相撞。因此这些限制也充分。等待超过  $k-1$  没有必要，因为此时全部初始蛇身均已释放。

转移为  $d_v = \max(d_u + 1, \text{open}_v)$ 。按距离递增处理顶点时，第一个发现  $v$  的前驱就给出最小候选值，故不需要后续松弛。候选等于  $d_u + 1$  的顶点进入普通 FIFO；其余进入对应开放时间的桶。每次取两者中距离较小的一项。桶指针只能扫描到下一个 FIFO 距离，否则可能越过之后才被发现较早开放时间。令  $R = \max \text{open}$ ，复杂度  $O(V + E + R)$ ，空间  $O(V + R)$ 。此性质依赖所有边权为 1；一般非负权仍需优先队列。

本地以独立堆 Dijkstra 对拍，并穷举  $2 \times 3$  网格障碍和全部 744 种初始蛇形，对真实蛇身序列逐步执行移动与缩尾 BFS，核对每个点的最早到达时间。另验证正式样例、满长链和  $3000 \times 3000$  空网格及隔断网格；普通与 ASan/UBSan 均通过。答案用 unsigned long long 累加距离平方，自然实现模  $2^{64}$ 。两套驱动在 QOJ8236 各通过 58 个正式与 3 个额外测试：动态版 324 ms、123080 KB，传统版 294 ms、122768 KB。线上证据对应单位网格、源点时间为零及初始蛇身开放时间；一般有向图与非零源点时间由本地对拍覆盖。

## 15.6 SCC 模板题与速度榜口径

QOJ906 来自 Library Checker，要求输出强连通分量的拓扑序。本库 Tarjan 编号按逆拓扑序产生，驱动将输入点号加一，再按分量编号从大到小输出，输出点号减一。CCF\_NOI 的记录 2940638 通过全部 11 个测试，203 ms、58636 KB。验证覆盖 add、run、bel 和完整分量输出，不调用 dag。本地独立可达性参考核对分量等价关系及每条跨分量边方向，并测试 50 万点链、环和多分量。普通与 ASan/UBSan 均通过；本地测试线程使用 1 GiB 栈容纳 sanitizer 扩大的递归栈帧，提交代码保持普通 main 和递归 DFS。

2026-09-12 核对速度榜时，页面第 66 行处于 203 ms 的第 65–67 行并列区间。但该表合并同一用户的提交，不能将行号与满分提交数 578 配对。以 CCF\_NOI 逐页读取页面链接的满分列表，共取得 143 条，其中 93 条严格更快、3 条同为 203 ms。可见范围内按耗时并列排名为 94 out of 143，位置区间为 94–96。统计页显示所有提交 1053、满分提交 578；与可见列表的计数差异原因尚未核实，因此全站排名仍为待查。记录清单与证据见 docs/ranking-audits.json，不得将该局部排名改写为全站排名。

## 15.7 SPFA 费用流与残量方案

SPFA 备选每次在残量图上直接求最短路，保留负边、限流和继续增广，但不允许负环。每次调用先将全部顶点以零距离入队检查，包括源点不可达的负环；因为不连通的负环同样能降低总费用，不能忽略。检查时不改流量，发现负环抛异常。记录严格改进路径的边数，达到顶点数即拒绝；不要用松弛次数冒充路径边数。零费用环不发生严格改进，不会反复改写前驱。

沿最短残量路增广保持当前流量下的最小费用性。反边可以撤销已经送过的流，最终方案应通过原边编号查询 used，不能累加历史路径当作最终流量。flow 返回本次新增流量与费用，重复调用须同源汇；传零限流仍检查负环。最小费用流允许负边与允许负环是两种不同的契约，后者应使用专门算法。

容量为 long long，距离及费用为 int128；费用不能为 LLONG\_MIN，因反边须取相反数。另要求总流不超过 long long，且  $\sum \text{cap}|\text{cost}| < 2^{120}$ 。P3381 另保证答案不超过  $2^{31} - 1$ ，其示例才可转换为 long long 输出。SPFA 版实际增广  $A$  次的复杂度为  $O((A+1)VE)$ ，不能假设  $A = O(V)$ ；通常优先势能 Dijkstra，本版保留为直接的备选写法。

两版通过 4000 组独立流量枚举及边界测试，完整驱动通过最大规模本地检查。验证细节见仓库 docs/FLOW-ATTACHMENT.md；本地通过不等于线上 AC 或速度排名。

## 15.8 2-SAT 编号方向与双连通定义

TwoSAT 的  $\text{add}(x,a,y,b)$  表示  $(x = a) \vee (y = b)$ 。强制值写  $\text{add}(x,a,x,a)$ ，蕴含  $(x = a) \Rightarrow (y = b)$  写  $\text{add}(x,!a,y,b)$ 。本库使用 Kosaraju 拓扑编号，真文字所在分量编号更大时取真；附件 Tarjan 的分量编号为逆拓扑序，比较方向相反。不能更换 SCC 实现而不检查赋值的不等号。solve 可在追加子句后重算，仅成功时读取 ans；一般复杂度为  $O(n + m)$ ，不保证字典序最小。

点双采用连通且无割点的定义：桥两端也成块，孤立点单独成块，割点允许属于多个块。自环不改变顶点间连通性，不额外产生点双；原边号仍保留，且不标桥。每条逻辑无向边只调用一次 add，不能把两个方向分别当作两条边加入。共享 lowlink 只跳过父边编号，不能跳过所有指向父顶点的重边，否则会误判桥。附件点双单独跳过父顶点虽不改变点双阈值的真假，但它的 low 不可直接复用来判桥。

BiconnectedCore 的点号为  $1 \dots n$ 、边号从零开始。run 后 blocks 是点双顶点集，bridge 是原边桥标记，bel 是  $1 \dots \text{cnt}$  的边双号；边双编号不承诺拓扑方向。不连通输入的圆方图和桥缩点结果均为森林。st 仅保存未归块顶点，不是模拟 DFS 的执行栈；每次 run 重新计算所有结果。

独立子集、删点、删边、真值表与最大规模驱动检查均有普通及 ASan/UBSan 记录。LC 点双 22 组、边双 21 组官方数据双模式本地 checker 接受，不等于线上 AC。百万变量矛盾链在 ASan 下需更大栈，本地以 1 GiB 线程栈运行原递归算法；提交仍用普通 main。细节见仓库 docs/GRAPH-ATTACHMENT.md。当前共享核心的边双输出已由 CCF\_NOI 提交 QOJ999 获 AC（记录 3061643, 87ms, 42352kb）；本次在线结果不核验 blocks、cut、森林或补边方案。

SCC 的完整拓扑序输出见第 84 页，注意其编号方向与 TarjanSCC 相反。固定 LC scc 的 12 组官方数据由本地 checker 接受；枚举三点有向图、独立传递闭包以及 50 万点长链、环、分离二元环核对分组和跨组顺序。同一 Kosaraju 驱动已由 CCF\_NOI 提交 QOJ906 获 AC（记录 3061625, 226ms, 76512kb）；dag 接口及其他未调用的捆绑算法不在本次覆盖范围，速度排名尚未核验。

已有点双分解时，removal\_components 返回原图块数 before 和独立删点后的总块数 after。删除孤立点会使总数减一；非单点点双每包含该顶点一次，给删除后的所在连通分量多贡献一块。AOJ 割点用法见第 99 页，只判  $\text{after}[u] > \text{before}$ ；UVA10765 完整数量及排名见第 100 页，以负块数和原点号排序，别把输入的输出名额  $m$  当作边数，保留每组输出后的空行。独立逐点删除图并重新求连通块，检查数量及同分顺序。

连通无向图能强连通定向当且仅当无桥。必要性：桥不论朝哪边定向，都不能双向通过；充分性：DFS 树边向下，非树边向祖先，每个非根子树都有返回严格祖先的边，逐层可回到根，根又能沿树边到达所有顶点。第 98 页先判桥再定向，每条逻辑边仅 add 一次，输出保留原边编号顺序。完整驱动枚举至五点的 771 个连通简单图，以逐边删除判可行性，再检查每条输出边及正反可达性；10 万点、30 万边边界和重边、自环的额外契约测试均通过。上述测试均有普通及 ASan/UBSan 记录，算法保留递归，不是线上 AC 或速度排名。

圆方森林保留全部原点  $1 \dots n$ ，第  $i$  个点双对应方点  $n + i + 1$  ( $i$  从零开始)。简单路径三元组应用见第 106 页：原点权为  $-1$ ，方点权为块的大小。对于同一连通分量中的不同原点  $s, f$ ，圆方树路径经过的各块，恰好给出可能作为中间点的顶点并集。同一块内任意第三点可放在两个指定端点间的简单路径上；块间只通过割点连接。方点权先计块大小，路径内部割点的  $-1$  去重，两端的  $-1$  排除  $s, f$ ，故路径权和就是可选中间点数，而不是简单路径条数。

设所在树有  $C$  个原点，删去节点  $u$  后各支的原点数为  $a_i$ 。有序原点端点对经过  $u$  的次数为  $2 \sum_{i < j} a_i a_j + 2(C - 1)[u \leq n]$ ；后项计端点等于  $u$  的情形。sz 仅统计原点，父侧大小为  $C - \text{sz}[u]$ ，遍历子树时用前缀和累计两乘积。乘以节点权后求和，使用 long long。每棵树分别求  $C$ ，孤立点贡献零；总时间空间  $O(n + m)$ 。

桥森林邻接项是（边双号，原桥号），不是原点端点；原点所属边双读 bel。第 90 页直接计度一叶子，第 102 页返回原点补边方案。连通且允许新增重边时，每个叶子至少需一个新增边端点，故至少  $\lceil L/2 \rceil$  条边。DFS 顺序收集叶子，奇数时补首叶，再前后半段配对；每条桥两侧的叶子区间都有配对跨过，故达到下界。仅一个边双时不加边。非连通森林不能直接套此公式或调用构造接口。P2860 只检查条数，方案端点及桥编号另以本地逐边删除和多重集检验；小图额外枚举全部新增边多重集验证最优性。P4630、P2860 都是比赛应用，不代替正式模板题覆盖或线上 AC。

## 15.9 排列约束与最大流单位边分类

NERC 2024 K (QOJ 10424) 给两套排列与同一大小为  $k$  的集合所形成的部分子序列。已知元素切开未知段：段内候选来自原排列的对应区间，需求是未知位置数；已知元素单独成需求一的组。检查锚点严格递增与区间长度足够；首尾设虚拟锚点。保留零需求组，使每个元素在两侧都恰好属于一组。

源连左组、右组连汇，容量为需求；每个元素直接对应左右组间的一条单位边。两侧需求总和均为  $k$ ，所以流量达到  $k$  等价于所有组恰好选够，等价于原题存在合法集合。这替代附件的元素拆点路径，减少节点且保留逐元素对应关系。仅区间检查不足以保证两侧同时可行。

保存 add 返回的原边号，满流后调用 `unit_flow_edges`。返回的 forced 为真输出 Y，否则 possible 为真输出问号，否则输出 N。原始网络分层无环，流恰好对应所选集合；残量 SCC 用于判断能否沿环改变某条单位边，不是最小割边分类。网络规模  $O(n)$ ，整数流值至多  $n$ ，因此本题增广求流  $O(n^2)$ ，之后一次 SCC 为  $O(n)$ 。

完整驱动通过 8166 个小输入穷举、2000 个独立子集枚举参考与最大  $n=2000$  边界检查。另按官方资料包 117 个配方本地重建输入，经官方 validator、两份 accepted 参考实现及普通/ASan/UBSan 下的官方 checker 核对；生成器无用越界迭代的修订有单独记录。这是比赛应用与本地证据，不增加正式模板题覆盖，也不表示线上 AC。细节见仓库 docs/KNOWN-UNKNOWN.md。

## 15.10 点分治、启发式合并与虚树用法

P3806 在建完树后只调用一次 build，询问 `count_exact` 是否非零。计数对象是无序不同顶点对，零权路径允许产生零距离，但不能把点与自身配对。预处理  $O(n \log^2 n)$ ，每次查询  $O(n \log n)$ ；边修改后必须重建。

CF600E 输出子树所有最高频颜色编号之和，读 `answer` 而非 `max_freq`。并列颜色全部相加，十万种不同颜色的和超过 32 位；颜色压缩后仍须累加原编号。轻子树先算后清空，重子树保留；清空整份临时统计后再重置最大频率与颜色和。该程序为比赛应用，非比赛模板题仍待补。

P2495 每次显式加入根 1，再构造虚树。虚边和 vertices 都是原树编号，只清零本次 vertices 的 dp；逆序遍历虚边，孩子为关键点时取路径最小边权，否则取路径最小边权与孩子 dp 的较小值。使用 minimum，不能用路径总长 distance。只清除本次 keys 的关键标记；祖先与后代同时关键也允许。题目来自省选，本册将其作为应用用法保留；最大答案可达 24999900000，须用 long long。

三份完整驱动分别通过路径枚举、祖先直方图、删边子集枚举；最大规模长链仍用递归 DFS，本地通过扩大测试进程栈匹配竞赛环境。本地证据与历史线上 AC 快照分开保存。

## 15.11 严格次小生成树与换边

`SecondMST` 用 Kruskal 选定一棵 MST，再用递归 DFS 和倍增维护树路径上两个不同的最大边权及其原边编号。每条非树非自环边换入后，非严格模式删路径最大边；若要求严格增大且换入边恰等于路径最大值，应删路径第二大的不同权边。没有这样的第二值则跳过，而不是把另一棵同权 MST 当作严格次小。路径权为 0,1、换入权为 1 就是两种定义不同的最小反例。

返回的 `tree` 是一份 MST 边号集合；从中删除 `out`、加入 `in` 得到 `next` 的方案。同端点同权重边也使用不同编号，自环永远不进入树。无次小树时 `next` 为空；断图时 `connected=false`，不能把默认总权零当成答案。所有总权和差值用 `int128`，边号 `-1` 表示缺项，避免以最小有符号权当哨兵。普通 `dsu` 保持路径减半和 `y` 根挂 `x` 根，DFS 不改为显式栈。

交换配对说明单边换入即可覆盖次小答案：将另一棵树与 MST 的不同边作合法交换配对，MST 最优性保证各配对差值非负。若总权严格增大，其中至少有一个正差不超过总差；枚举所有环上的最优严格换边就能达到全局最小正差。P4180 为 BJWC 应用题，保证严格次小存在；其权值范围可安全转回 `long long`，但一般接口不可如此假设。6368 份生成树全子集对照、原边方案检查、十万点链和三十万边完整驱动双模式证据见 `docs/SECOND-MST.md`。

## 15.12 闭合集、树形图与全局割的完整用法

P2762 把实验设为正权点、仪器设为负权点，实验到仪器连依赖边。依赖数量不固定，先用 `getline` 读一整行，再用 `istringstream` 分解；这样兼容题面的 CRLF，也不会把下一实验的赞助误读成仪器编号。`maximum_closure` 返回的点号中，实验为  $1..m$ ，仪器为  $m+1..m+n$ ；第二类输出时减去  $m$ 。空方案仍输出两个空行，再输出零收益。共享仪器只计费一次，赞助总额用 `long long`。输出方案可不唯一，不能只比较固定编号。

P4716 的点号和根都减一，调用 `Arborescence::solve`。有值时才解引用；无解输出 `-1`。每个非根点有入边不代表根能到达所有点，必须处理入边环。该接口只返回费用，不恢复边方案；核心用 `optional` 区分无解与合法的负费用 `-1`。P5632 先累加对称矩阵，再调用 `StoerWagner::solve`；`side` 是一侧的原顶点编号，仍为 0-based。核心要求至少两个点、非负权。单点不存在非平凡割，驱动另行约定输出零。两题的费用能转 `long long` 是题面范围保证，不能据此缩小核心的 128 位费用类型。

660 份实验计划、355 份最小树形图和 891 份全局割输入，分别以子集、入边组合、非平凡割枚举核验，另测最大规模及打印程序；普通与 ASan/UBSan 均通过。完整契约与证据见 `docs/CUT-APPLICATIONS.md`。P2762 为应用，P4716/P5632 为正式模板题；本地验证不等于线上 AC。

## 第一部分

### 跨类依赖

# 附录 A 数论

## A.1 模算术简介

### A.1.1 静态模整数

固定 int 模数  $1 \leq \text{mod} \leq \text{INT\_MAX}$ , 可为合数。构造接受 signed64 并正规化, v 保持  $0.. \text{mod}-1$ ; 不要手动破坏正规化。加减乘  $O(1)$ , pow 非负 signed64 指数、时间  $O(\log(e+1))$ 。try\_inv 用欧几里得求逆:  $\text{gcd}(v, \text{mod})=1$  时返回 optional 值, 否则 nullopt;  $\text{mod}=1$  返回代表 0。inv、/、/= 要求分母可逆; 断言开启时诊断违反前提, 关闭断言不承诺拒绝非法除法。逆元  $O(\log(\text{mod}+1))$ 、额外空间  $O(1)$ , 只用 long long, 不新增辅助模板依赖。+=、-=、\*=、/= 返回自身引用, 支持自赋值与链式运算, 可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
1 template <int mod> struct ModInt
2 {
3     static_assert(mod >= 1);
4     int v;
5
6     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
7
8     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
9
10    ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
11
12    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
13
14    ModInt pow(long long e) const
15    {
16        assert(e >= 0);
17        ModInt a = *this, r = 1;
18        for (; e >= 1, a = a * a)
19            if (e & 1) r = r * a;
20        return r;
21    }
```

求逆与除法（接上页同一结构体）：

```

23     optional<ModInt> try_inv() const
24     {
25         long long a = v, b = mod, x = 1, y = 0;
26         while (b)
27         {
28             long long q = a / b;
29             a -= q * b;
30             swap(a, b);
31             x -= q * y;
32             swap(x, y);
33         }
34         if (a != 1) return nullopt;
35         return ModInt(x);
36     }
37
38     ModInt inv() const
39     {
40         auto r = try_inv();
41         assert(r);
42         return *r;
43     } // requires an invertible residue
44
45     ModInt operator/(ModInt b) const { return *this * b.inv(); }
46
47     ModInt &operator+=(ModInt b) { return *this = *this + b; }
48
49     ModInt &operator-=(ModInt b) { return *this = *this - b; }
50
51     ModInt &operator*=(ModInt b) { return *this = *this * b; }
52
53     ModInt &operator/=(ModInt b) { return *this = *this / b; }
54 };

```

联合使用示例见第 202 页（Luogu P1313 计算系数），其中直接调用本模板。

## A.2 卢卡斯定理

### A.2.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p, choose(n,k) 返回  $C(n,k)$  模 p; n、k 为完整 unsigned long long 范围,  $k > n$  返回 0,  $C(0,0)=1$ 。重新赋值 Lucas(p) 可更换模数并重新预处理; 不检测素性, 要求调用者保证 p 为素数且  $p \geq 2$ 。预处理时间  $O(p + \log p)$ , 动态版空间  $O(p)$ ; 单次查询  $O(1 + \log_p(n+1))$ 。适合小素数模数, 通常  $p \leq 10^6$ ; 大素数不能因查询下标小就忽略这里整段  $O(p)$  的预处理。fac、ifac 下标为  $0..p-1$ , 模数和表只读; 中间乘法用 long long。合数模数请使用 exLucas。

```

1 struct Lucas
2 {
3     int mod;
4     vector<int> fac, ifac;
5
6     Lucas(int p)
7     {
8         assert(p >= 2);
9         mod = p;
10        fac.resize(p);
11        ifac.resize(p);
12        fac[0] = 1;
13        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
14        long long a = fac[p - 1], r = 1;
15        for (int e = p - 2; e; e >>= 1)
16        {
17            if (e & 1) r = r * a % p;
18            a = a * a % p;
19        }
20        ifac[p - 1] = r;
21        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
22    }
23
24    int choose(unsigned long long n, unsigned long long k) const
25    {
26        if (k > n) return 0;
27        long long answer = 1;
28        while (n || k)
29        {
30            int a = n % mod, b = k % mod;
31            if (b > a) return 0;
32            answer = answer * fac[a] % mod;
33            answer = answer * ifac[b] % mod;
34            answer = answer * ifac[a - b] % mod;
35            n /= mod;
36            k /= mod;
37        }
38        return answer;
39    }
40 };

```

## 模板题使用：Luogu P3807

题意：多组查询  $C(n+m, n)$  模  $p$ ，题目保证  $p$  为素数且不超过  $10^5$ 。每组构造  $\text{Lucas}(p)$ ，choose 的参数为  $n+m$ 、 $n$ ；此题范围内  $n+m$  不会溢出。

所需模板：Lucas（第 199 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3807>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int t;
6     cin >> t;
7     while (t--)
8     {
9         unsigned long long n, m;
10        int p;
11        cin >> n >> m >> p;
12        Lucas solver(p);
13        cout << solver.choose(n + m, n) << '\n';
14    }
15    return 0;
16 }
```

# 附录 B 组合数学

## B.1 排列组合

### B.1.1 阶乘组合数

代码依赖：ModInt (第 197 页, A.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理  $0..n$ ，要求 prime 为素数且  $0 \leq n < \text{prime}$ 。init(n) 只扩大当前上限，较小或相同的  $n$  不改变表；choose(n,k) 返回  $C(n,k)$ ，permute(n,k) 返回  $n!/(n-k)!$ 。查询前显式预处理到  $n$ ，查询本身不扩表。 $n < 0$ 、 $k < 0$  或  $k > n$  返回 0， $C(0,0)=P(0,0)=1$ 。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询  $O(1)$ ；从 old 扩到  $n$  新增  $O(n-\text{old}+\log \text{prime})$  次域运算，重分配还可能复制旧表。首次预处理  $O(n+\log \text{prime})$ ，空间  $O(n)$ 。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
1 template <int mod> struct Binomial
2 {
3     using Z = ModInt<mod>;
4     vector<Z> fac{Z(1)}, ifac{Z(1)};
5
6     Binomial(int n = 0) { init(n); }
7
8     void init(int n)
9     {
10         assert(0 <= n && n < mod);
11         int old = (int)fac.size() - 1;
12         if (n <= old) return;
13         fac.resize(n + 1);
14         ifac.resize(n + 1);
15         for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
16         ifac[n] = fac[n].inv();
17         for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
18     }
```

```
20 Z choose(int n, int k) const
21 {
22     if (k < 0 || k > n) return 0;
23     assert(n < (int)fac.size());
24     return fac[n] * ifac[k] * ifac[n - k];
25 }
26
27 Z permute(int n, int k) const
28 {
29     if (k < 0 || k > n) return 0;
30     assert(n < (int)fac.size());
31     return fac[n] * ifac[n - k];
32 }
33 };
```

## 应用题补充：Luogu P1313 计算系数

题意：求  $(by+ax)$  的  $k$  次方中指定项的系数，模 10007。 $x$ 、 $y$  的指数分别为  $n$ 、 $m$ ，均非负且  $n+m=k$ 。 $k$  至多 1000， $a$ 、 $b$  均为不超过 1000000 的非负数。组合  $C(k,n)$  与两个幂即可。模数为素数且  $k$  小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：[ModInt](#)（第 197 页，A.1.1 节）、[Binomial](#)（第 201 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P1313>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int a, b, k, n, m;
6     cin >> a >> b >> k >> n >> m;
7     Binomial<10007> c(k);
8     auto x = ModInt<10007>(a).pow(n);
9     auto y = ModInt<10007>(b).pow(m);
10    cout << (c.choose(k, n) * x * y).v << '\n';
11 }
```

# 附录 C 线性代数

## C.1 行列式

### C.1.1 素数模行列式

代码依赖: `ModInt` (第 197 页, A.1.1 节)。抄写时一并准备; 标准库类型不列入算法依赖。

`det_prime<p>(a)` 输入  $n \times n$  方阵, 返回行列式模  $p$ ;  $p$  必须是素数。允许奇异矩阵和零维矩阵, `det(empty)=1`。按行交换改变符号, 主元求逆消元。  $O(n^3)$  时间、传值副本  $O(n^2)$  空间, 可传 `move(a)`。合数模数改用 `determinant_mod`。

```
1 template <int mod> ModInt<mod> det_prime(vector<vector<ModInt<mod>>> a)
2 {
3     using Z = ModInt<mod>;
4     int n = (int)a.size();
5     Z ans = 1;
6     for (int i = 0; i < n; i++)
7     {
8         assert((int)a[i].size() == n);
9         int p = i;
10        while (p < n && !a[p][i].v) ++p;
11        if (p == n) return 0;
12        if (p != i)
13        {
14            swap(a[p], a[i]);
15            ans = Z(0) - ans;
16        }
17        ans = ans * a[i][i];
18        Z inv = a[i][i].inv();
19        for (int j = i + 1; j < n; j++)
20        {
21            Z f = a[j][i] * inv;
22            for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
23        }
24    }
25    return ans;
26 }
```

## 模板题使用：Library Checker matrix\_det

题意：求方阵行列式，模 998244353。输入元素已取模；函数按值接收矩阵，示例用 `move` 交出矩阵避免额外复制。

所需模板：`ModInt`（第 197 页，A.1.1 节）、`det_prime`（第 203 页，C.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：[https://judge.yosupo.jp/problem/matrix\\_det](https://judge.yosupo.jp/problem/matrix_det)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     using Z = ModInt<998244353>;
6     int n;
7     cin >> n;
8     vector<vector<Z>> a(n, vector<Z>(n));
9     for (auto &row : a)
10         for (auto &x : row) cin >> x.v;
11     cout << det_prime<998244353>(move(a)).v << '\n';
12 }
```

### C.1.2 任意正模数行列式：欧几里得消元

`determinant_mod(a,mod)` 返回  $\det(a)$  的最小非负余数。 $a$  为 `long long` 方阵，元素支持完整 signed 64 位； $1 \leq \text{mod} \leq \text{LLONG\_MAX}$ ，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回  $1 \% \text{mod}$ ， $\text{mod}=1$  时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 `int128`。 $O(n^3 \log \text{mod})$  时间、 $O(n^2)$  空间；素数模通常优先用已有高斯行列式。

```

1 long long determinant_mod(vector<vector<long long>> a, long long mod)
2 {
3     assert(mod >= 1);
4     int n = a.size();
5     for (auto &row : a)
6     {
7         assert((int)row.size() == n);
8         for (auto &x : row)
9         {
10             x %= mod;
11             if (x < 0) x += mod;
12         }
13     }
14     long long answer = 1 % mod;
15     for (int i = 0; i < n; i++)
16     {
17         for (int j = i + 1; j < n; j++)
18             while (a[j][i])
19             {
20                 long long quotient = a[i][i] / a[j][i];
21                 for (int k = i; k < n; k++)
22                 {
23                     __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
24                     value %= mod;
25                     if (value < 0) value += mod;
26                     a[i][k] = (long long)value;
27                 }
28                 swap(a[i], a[j]);
29                 answer = answer ? mod - answer : 0;
30             }
31         if (a[i][i] == 0) return 0;
32         answer = (__int128)answer * a[i][i] % mod;
33     }
34     return answer;
35 }

```

## 模板题使用：Library Checker `matrix_det_arbitrary_mod`

题意：给方阵和模数，求行列式的最小非负余数，模数可以为合数。函数无需模逆元，按值接收矩阵，示例用 `move` 交出数据。0 阶方阵的行列式为 1，再对模数取余。

所需模板：`determinant_mod`（第 205 页，C.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：[https://judge.yosupo.jp/problem/matrix\\_det\\_arbitrary\\_mod](https://judge.yosupo.jp/problem/matrix_det_arbitrary_mod)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     long long mod;
7     cin >> n >> mod;
8     vector<vector<long long>> a(n, vector<long long>(n));
9     for (auto &row : a)
10         for (auto &x : row) cin >> x;
11     cout << determinant_mod(move(a), mod) << '\n';
12 }
```

# 附录 D 并查集

## D.1 并查集

### D.1.1 并查集

dsu(n), 点号  $0..n-1$ ,  $n \geq 0$ 。find(x) 路径减半; same(x,y) 判断同集合; merge(x,y) 把 y 的根接到 x 的根, 已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组, 组内点号递增、组间按当前根编号排列; 可能压缩路径, 但不改变分组。fa、sz 可读, sz 仅根处有效; 不要外部改写。本版只做路径减半, 不按大小或秩合并; 单次 find 最坏  $O(n)$ 。空间  $O(n)$ , groups 另需  $O(n)$  输出与工作空间, 并执行 n 次 find。空结构可调用 groups, 不能查询顶点。

```
1 struct dsu
2 {
3     vector<int> fa, sz;
4
5     dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }
6
7     int find(int x)
8     {
9         while (x != fa[x]) x = fa[x] = fa[fa[x]];
10        return x;
11    }
12
13    bool same(int x, int y) { return find(x) == find(y); }
14
15    bool merge(int x, int y)
16    {
17        x = find(x), y = find(y);
18        if (x == y) return false;
19        sz[x] += sz[y], fa[y] = x;
20        return true;
21    }
22
23    int size(int x) { return sz[find(x)]; }
24
25    vector<vector<int>> groups()
26    {
27        vector<vector<int>> res(fa.size());
28        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
29        res.erase(
30            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
31            res.end());
32        return res;
33    }
34};
```

## 模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号  $1..n$ ，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 207 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3367>

```
1 int main()
2 {
3     int n, m;
4     scanf("%d%d", &n, &m);
5     dsu d(n);
6     while (m--)
7     {
8         int op, x, y;
9         scanf("%d%d%d", &op, &x, &y);
10        --x, --y;
11        if (op == 1)
12            d.merge(x, y);
13        else
14            puts(d.same(x, y) ? "Y" : "N");
15    }
16 }
```

# 附录 E 树状数组

## E.1 树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b; b >>= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };
```

## 模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第  $k$  小和严格前驱/后继。预读全部操作并离散化非第  $k$  小查询的值域；频数下标为 1-based,  $k$ th 的返回值减 1 后映射回原值。排名查询允许  $x$  不在集合中；第  $k$  小与前驱/后继按题目保证存在。 $n \leq 100000$ ,  $|x| \leq 10000000$ ；总复杂度  $O(n \log n)$ ，空间  $O(n)$ 。

所需模板：Fenwick（第 209 页，E.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3369>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     if (!(cin >> n)) return 0;
7     vector<pair<int, int>> q(n);
8     vector<int> a;
9     for (auto &[op, x] : q)
10    {
11        cin >> op >> x;
12        if (op != 4) a.push_back(x);
13    }
14    sort(a.begin(), a.end());
15    a.erase(unique(a.begin(), a.end()), a.end());
16    Fenwick<int> f(a.size());
17    for (auto [op, x] : q)
18    {
19        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
20        if (op == 1)
21            f.add(p + 1, 1);
22        else if (op == 2)
23        {
24            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
25        }
26        else if (op == 3)
27            cout << f.sum(p) + 1 << '\n';
28        else if (op == 4)
29            cout << a[f.kth(x) - 1] << '\n';
30        else if (op == 5)
31            cout << a[f.kth(f.sum(p)) - 1] << '\n';
32        else
33        {
34            p = upper_bound(a.begin(), a.end(), x) - a.begin();
35            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
36        }
37    }
38 }

```

# 附录 F 线段树

## F.1 线段树基础

### F.1.1 通用线段树与左右边界搜索

`segtree<S,op,e>(a)` 从 0-based vector 构建, 或 `segtree(n)` 初始化为单位元。  $0 \leq n \leq 2^{29}$ , 实际受内存限制。`op(S,S)` 须满足结合律, `e()` 是双侧单位元; 无需交换律。`set(p,x)`、`get(p)` 操作单点, `prod(l,r)` 返回半开区间  $[l,r)$  从左到右的合并, 空区间返回 `e`; `all()` 返回全区间结果。`max_right(l,ok)` 返回最大的  $r$  使 `ok(prod(l,r))` 为真; `min_left(r,ok)` 返回最小的  $l$ 。要求 `ok(e())` 为真, `ok` 无副作用, 并随对应方向扩展从真变假后不再变真; 例如非负和不超过阈值, 不能对任意负数和这样搜索。 $l=n$ 、 $r=0$  支持空端点。构建  $O(n)$ , 单点/区间/边界搜索  $O(\log n)$ , `all`  $O(1)$ , 空间  $O(n)$ , 假设 `op`、`e`、`ok` 及 `S` 复制为  $O(1)$ 。`d`、`sz`、`pull` 为内部实现, 不直接修改。函数复合时 `op(f,g)` 表示 `g(f(x))`, 对应先执行左区间再执行右区间; `min_left` 不能照抄 `max_right` 的合并顺序。

```
1 template <class S, auto op, auto e> struct segtree
2 {
3     int n, sz = 1;
4     vector<S> d;
5
6     segtree(int n = 0) : segtree(vector<S>(n, e())) {}
7
8     segtree(const vector<S> &a) : n(a.size())
9     {
10         while (sz < n) sz <= 1;
11         d.assign(2 * sz, e());
12         copy(a.begin(), a.end(), d.begin() + sz);
13         for (int i = sz - 1; i; i--) pull(i);
14     }
15
16     void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }
17
18     void set(int p, S x)
19     {
20         assert(0 <= p && p < n);
21         d[p += sz] = x;
22         while (p >>= 1) pull(p);
23     }
24
25     S get(int p) const
26     {
27         assert(0 <= p && p < n);
28         return d[p + sz];
29     }
30
31     S all() const { return d[1]; }
```

```

33  S prod(int l, int r) const
34  {
35      assert(0 <= l && l <= r && r <= n);
36      S a = e(), b = e();
37      for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
38      {
39          if (l & 1) a = op(a, d[l++]);
40          if (r & 1) b = op(d[--r], b);
41      }
42      return op(a, b);
43  }
44
45  template <class F> int max_right(int l, F ok) const
46  {
47      assert(0 <= l && l <= n && ok(e()));
48      if (l == n) return n;
49      S a = e();
50      l += sz;
51      do
52      {
53          while (!(l & 1)) l >>= 1;
54          if (!ok(op(a, d[l])))
55          {
56              while (l < sz)
57              {
58                  l *= 2;
59                  if (ok(op(a, d[l]))) a = op(a, d[l++]);
60              }
61              return l - sz;
62          }
63          a = op(a, d[l++]);
64      } while ((l & -1) != 1);
65      return n;
66  }

```

```
68     template <class F> int min_left(int r, F ok) const
69     {
70         assert(0 <= r && r <= n && ok(e()));
71         if (r == 0) return 0;
72         S a = e();
73         r += sz;
74         do
75         {
76             --r;
77             while (r > 1 && (r & 1)) r >>= 1;
78             if (!ok(op(d[r], a)))
79             {
80                 while (r < sz)
81                 {
82                     r = r * 2 + 1;
83                     if (ok(op(d[r], a))) a = op(d[r--], a);
84                 }
85                 return r + 1 - sz;
86             }
87             a = op(d[r], a);
88         } while ((r & -r) != r);
89         return 0;
90     }
91 };
```

## 模板题使用：Library Checker point\_set\_range\_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于  $x$  的结果。 $op(f,g)$  表示先  $f$  后  $g$ ，单位元为  $x$ ； $prod(l,r)$  使用 0-based 半开区间，空区间返回单位元。配置函数须和 `main` 一起抄写。

所需模板：`segtree`（第 211 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：[https://judge.yosupo.jp/problem/point\\_set\\_range\\_composite](https://judge.yosupo.jp/problem/point_set_range_composite)

```

1 using S = pair<long long, long long>;
2 const int mod = 998244353;
3
4 S op(S f, S g)
5 {
6     return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
7 }
8
9 S e()
10 {
11     return {1, 0};
12 }
13
14 int main()
15 {
16     ios::sync_with_stdio(false);
17     cin.tie(nullptr);
18     int n, q;
19     cin >> n >> q;
20     vector<S> a(n);
21     for (auto &[x, y] : a) cin >> x >> y;
22     segtree<S, op, e> s(a);
23     while (q--)
24     {
25         int t;
26         cin >> t;
27         if (t == 0)
28         {
29             int p;
30             long long c, d;
31             cin >> p >> c >> d;
32             s.set(p, {c, d});
33         }
34         else
35         {
36             int l, r;
37             long long x;
38             cin >> l >> r >> x;
39             auto [c, d] = s.prod(l, r);
40             cout << (c * x + d) % mod << '\n';
41         }
42     }
43 }
```

### F.1.2 线段树：区间乘、加、赋值与模意义求和

`AffineSegTree(a,mod)` 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。`update(l,r,mul,add)` 将区间元素  $x$  改为  $mul \cdot x + add$ ；乘法用 `(mul,0)`，加法用 `(1,add)`，赋值用 `(0,value)`。`query(l,r)` 返回区间和。 $1 \leq mod \leq INT\_MAX$ ，模数无需为素数，初值和修改参数允许 `long long` 全范围，入口统一归一化。旧标记 `(m,b)` 后应用新标记 `(a,c)`，复合为  $(m \cdot a, b \cdot a + c)$ ，乘法取模后再相加。建树  $O(n)$ ，单次  $O(\log n)$ ，空间  $O(n)$ 。内部 `build`、`apply` 等函数不作为公共接口；动态版重建新对象。

```

1 struct AffineSegTree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         ll sum = 0, mul = 1, add = 0;
8     };
9
10    int n, mod;
11    vector<Node> t;
12
13    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
14    {
15        assert(n > 0 && mod > 0);
16        build(1, 1, n, a);
17    }
18
19    ll norm(ll x) const
20    {
21        x %= mod;
22        if (x < 0) x += mod;
23        return x;
24    }
25
26    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }
27
28    void build(int p, int l, int r, const vector<ll> &a)
29    {
30        t[p] = Node{};
31        if (l == r)
32        {
33            t[p].sum = norm(a[l - 1]);
34            return;
35        }
36        int m = (l + r) / 2;
37        build(p * 2, l, m, a);
38        build(p * 2 + 1, m + 1, r, a);
39        pull(p);
40    }
41
42    // New transform follows the old pending transform: x -> x*a+b.
43    void apply(int p, int len, ll a, ll b)
44    {
45        t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
46        t[p].mul = t[p].mul * a % mod;
47        t[p].add = (t[p].add * a % mod + b) % mod;
48    }

```

```

49
50 void push(int p, int l, int r)
51 {
52     if (t[p].mul == 1 && t[p].add == 0) return;
53     int m = (l + r) / 2;
54     apply(p * 2, m - l + 1, t[p].mul, t[p].add);
55     apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
56     t[p].mul = 1;
57     t[p].add = 0;
58 }
59
60 void update(int l, int r, ll a, ll b)
61 {
62     assert(1 <= l && l <= r && r <= n);
63     update(1, 1, n, l, r, norm(a), norm(b));
64 }
65
66 void update(int p, int l, int r, int ql, int qr, ll a, ll b)
67 {
68     if (ql <= l && r <= qr)
69     {
70         apply(p, r - l + 1, a, b);
71         return;
72     }
73     push(p, l, r);
74     int m = (l + r) / 2;
75     if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
76     if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
77     pull(p);
78 }
79
80 ll query(int l, int r)
81 {
82     assert(1 <= l && l <= r && r <= n);
83     return query(1, 1, n, l, r);
84 }
85
86 ll query(int p, int l, int r, int ql, int qr)
87 {
88     if (ql <= l && r <= qr) return t[p].sum;
89     push(p, l, r);
90     int m = (l + r) / 2;
91     ll answer = 0;
92     if (ql <= m) answer += query(p * 2, l, m, ql, qr);
93     if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
94     return answer % mod;
95 }
96 };

```

## 模板题使用：Luogu P3373

题意：对数组执行区间乘、区间加和区间求和，结果对给定模数取模。区间为 1-based 闭区间；update 的两个参数表示先乘再加。

所需模板：AffineSegTree（第 215 页，F.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3373>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, q, mod;
6     std::cin >> n >> q >> mod;
7     std::vector<long long> a(n);
8     for (auto &x : a) std::cin >> x;
9     AffineSegTree tree(a, mod);
10    while (q--)
11    {
12        int op, l, r;
13        std::cin >> op >> l >> r;
14        if (op == 3)
15            std::cout << tree.query(l, r) << '\n';
16        else
17        {
18            long long k;
19            std::cin >> k;
20            if (op == 1)
21                tree.update(l, r, k, 0);
22            else
23                tree.update(l, r, 1, k);
24        }
25    }
26 }
```

# 算法索引

AffineSegTree, 215  
Arborescence, 71  
  
BiconnectedCore, 85  
Binomial, 201  
BipartiteMatching, 161  
block\_cut\_forest, 105  
Blossom, 169  
BoundedCirculation, 152  
BoundedMaxFlow, 154  
bridge\_augmentation, 101  
bridge\_component\_forest, 89  
  
Cayley formula, 173  
CentroidDiameter, 49  
CentroidNearest, 45  
CentroidPairs, 36  
CentroidSum, 40  
cut\_tree\_values, 140  
  
dag\_path\_determinant, 182  
det\_prime, 203  
determinant\_mod, 205  
Dijkstra, 58  
Dinic, 127  
DirectedEuler, 115  
DominatorTree, 184  
dsu, 207  
  
EdgeCompression, 94  
enumerate\_triangles, 107  
EulerLCA, 10  
  
Fenwick, 209  
FunctionalGraph, 76  
  
gomory\_hu, 138  
  
HLD, 20  
Hungarian, 165  
  
Johnson, 55  
  
LexTwoSAT, 112  
LiftingLCA, 13  
Lowlink, 102

Lucas, 199

Manhattan MST, 70

ManhattanMST, 62

matching\_edges, 164

MatrixTree, 175

MatrixTreeMod, 177

maximum\_closure, 136

MinCostFlow, 141

mincut\_edges, 134

minimum spanning tree, 70

mixed\_euler\_orientation, 123

mixed\_euler\_trail, 126

ModInt, 197

NegativeCostFlow, 149

odd\_cycle\_vertices, 91

odd\_induced\_partition, 1

OfflineLCA, 7

orient\_edges, 97

path\_intersection, 18

Prüfer sequence, 173

release\_bfs, 60

removal\_components, 98

SCC, 82

SecondMST, 65

segtree, 211

SpfaFlow, 145

StoerWagner, 158

SubtreeColors, 29

TarjanSCC, 74

TreeDiameter, 4

TwoSAT, 109

UndirectedEuler, 120

unit\_flow\_edges, 130

VirtualTree, 32

WeightedMatching, 167

word\_chain, 118

开放时间最短路, 192

树上路径交, 188

贪吃蛇, 192